

On the Partition Algebra of Bounded Phase Space

A Unified Framework for Categorical State Counting,
High-Energy Trigger Design, and Runtime Verification

On the Partition Algebra of Bounded Phase Space

A Unified Framework for Categorical State Counting,
High-Energy Trigger Design, and Runtime Verification

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics

AI Me Registry for Artificial Intelligence

June 4, 2026

All papers in this monograph are derived from a single Bounded Phase Space Axiom. The operating-system framework (Part V) and the closed-loop paper (Part VI) establish the formal computational substrate over which the partition-physics derivations are executed at runtime.

Each paper in this collection is self-contained and may be compiled independently. The monograph master file assembles all compiled PDFs into a single volume. Cross-citations between papers use the `companion:*` family of bibtex keys.

Contents

Preface

This monograph presents a unified framework deriving all principal results of particle-physics trigger design from a single axiom: that every persistent physical system occupies a bounded region of phase space with finite Liouville measure. From this axiom, via the Poincaré recurrence theorem and the algebra of integer partitions, one derives the partition coordinates (n, ℓ, m, s) , the shell capacity $C(n) = 2n^2$, the composition-inflation formula $T(n, d) = d(d + 1)^{n-1}$, the Planck depth $n_P = 60$ for the LHC, and every quantitative design parameter of the ATLAS real-time event-selection system.

The monograph is organised in six parts.

Part I (Papers 1–2) establishes the mathematical foundations: the partition algebra, the Planck depth, and the compositional structure of the partition state space.

Part II (Papers 3–5) applies the foundations to measurement: the LHC as a high-energy partition spectrometer governed by the partition Lagrangian, the four-subsystem CSCO and QND property, and the angular resolution of calorimeter detectors as a counting theorem.

Part III (Papers 6–8) develops trajectory and dynamic analysis: backward trajectory completion in $\mathcal{O}(\log_3 N)$, the 25 ns timing window derived from detector geometry, and virtual particles identified with virtual sub-states.

Part IV (Papers 9–11) covers algebraic structure and consequences: cascade catalyst algebra (which coincides with the renormalisation group), cosmological consequences including the cosmological constant as partition pressure, and the full experimental validation programme from keV to TeV.

Part V (Papers 12–16) presents the operating-system framework: a refinement-type system over the vaHera AST in which selection rules are type predicates, three-route equivalence as a verification primitive, a universal closed-form for transport coefficients subsuming eight classical formulas, operational bisimulation for cross-framework equation identity, and the five-subsystem dispatch kernel.

Part VI (Paper 17) closes the loop: the Tempus trigger program is compiled to vaHera, dispatched by the kernel, and proved to compute the same partition classification as the direct physics derivation on every input event (the Tempus-Physics Bisimulation Theorem).

The single axiom, the seventeen papers, and the formal bisimulation together constitute a complete derivation from first principles to running software.

Kundai Farai Sachikonye

Technical University of Munich, June 4, 2026

Part I

Mathematical Foundations

On the Partition Algebra of Bounded Phase Space: Composition-Inflation, Angular Resolution, and the Categorical Reformulation of Fundamental Constants

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

Every persistent physical system occupies a bounded region of phase space. From this single empirical observation, promoted to an axiom, we derive by deduction the complete partition algebra governing distinguishable states in bounded oscillatory systems.

The central result is the *composition-inflation formula*: a bounded oscillatory system with d entropy dimensions completing n oscillation cycles generates

$$T(n, d) = d \cdot (d + 1)^{n-1}$$

categorically distinguishable trajectories — an exponential, not linear, count. The formula is exact, has no adjustable parameters, and follows from the binomial theorem applied to labeled integer compositions.

We derive three consequences that are not available in any existing framework. First, the *angular resolution* $\Delta\theta = 2\pi/T(n, d)$ is dimensionless and therefore not subject to the Planck time constraint, which bounds temporal intervals measured in seconds but does not bound dimensionless ratios. For any $\varepsilon > 0$ there exists a finite n achieving $\Delta\theta < \varepsilon$. Second, all fundamental constants except the gravitational constant G reduce to tick counts, factors of 2π , and a single energy unit E_{tick} in categorical angular units: $c = 2\pi \text{ rad/tick}$, $\hbar = E_{\text{tick}}/(2\pi)$, $k_B = E_{\text{tick}}/\ln(d + 1)$. Third, G is the unique irreducible coupling constant — the only physical constant not expressible in terms of counting and geometry.

The paper is self-contained: all results are proved from the single axiom with no additional physical postulates. Applications to the LHC trigger, mass spectrometry, and gravitational physics are indicated in the final section but not developed here.

Keywords: bounded phase space; partition algebra; composition-inflation; angular resolution; Planck depth; categorical angular units; gravitational constant

Contents

1 Introduction

1.1 The single empirical premise

Physical systems exist. Every physical system that persists — an electron bound to a nucleus, a molecule in a container, a planet in a stellar orbit, an ion in a trap — occupies a bounded region of phase space. The complement — a system whose phase space region grows without bound — disperses and ceases to persist. This is not a theoretical proposal; it is a summary of every observation of every persistent object in the history of physics.

We promote this observation to an axiom and derive consequences.

1.2 What follows

The derivation proceeds in four steps. Section ?? states the axiom and derives oscillatory necessity (Poincaré recurrence eliminates every alternative). Section ?? constructs the partition coordinate system (n, ℓ, m, s) with capacity $C(n) = 2n^2$ from the geometry of bounded oscillation. Section ?? derives the composition-inflation formula $T(n, d) = d(d + 1)^{n-1}$ and establishes the angular resolution theorem. Section ?? expresses all fundamental constants in categorical angular units, identifies G as the unique irreducible constant, and derives the Planck depth.

No theorem in Sections ??–?? uses any postulate beyond the axiom of Section ??.

2 Bounded Phase Space and Oscillatory Necessity

Axiom 1 (Bounded Phase Space). Every persistent dynamical system (Ω, μ, ϕ_t) satisfies:

- (i) $\Omega \subset \mathbb{R}^{2d}$ is bounded with finite Liouville measure $\mu(\Omega) < \infty$.
- (ii) $\phi_t : \Omega \rightarrow \Omega$ preserves μ for all t .
- (iii) Ω admits a sequence of finite measurable partitions $\mathcal{P}_1 \preceq \mathcal{P}_2 \preceq \dots$ with each $\mathcal{P}_{k+1}$ refining $\mathcal{P}_k$ and

Chapter 1. Paper 1. On the Partition Algebra of Bounded Phase Space:

Composition-Inflation, Angular Resolution, and the Categorical Reformulation of
the generated σ -algebra equalling the Borel algebra
Fundamental Constants

Theorem 3.2 (Shell Capacity). *The number of distinct partition states at principal level n is*

$$C(n) = 2n^2. \quad (1)$$

Theorem 2.1 (Poincaré Recurrence ?). *Under Axiom ??, for every measurable $A \subset \Omega$ with $\mu(A) > 0$ and every $T > 0$, almost every $x \in A$ satisfies $\phi_{t_n}(x) \in A$ for infinitely many $t_n \rightarrow \infty$.*

Proof. Suppose not. Let $B = \{x \in A : \phi_t(x) \notin A \forall t > T\}$. The iterates $\{\phi_{-kT}(B)\}_{k \geq 0}$ are pairwise disjoint (if $x \in \phi_{-jT}(B) \cap \phi_{-kT}(B)$ with $j < k$, then $\phi_{(k-j)T}(x) \in B$ and later $\phi_t(x)$ returns to A , contradicting $x \in B$). Measure preservation gives $\sum_{k=0}^{\infty} \mu(B) = \mu(\bigcup_k \phi_{-kT}(B)) \leq \mu(\Omega) < \infty$, so $\mu(B) = 0$. $\square$

Corollary 2.2 (Oscillatory Necessity). *Under Axiom ??, every persistent non-static trajectory is oscillatory: no coordinate can grow monotonically, and every trajectory returns to every neighbourhood of its starting point infinitely often.*

Proof. Monotonic growth contradicts boundedness. Absence of recurrence contradicts Theorem ??. The only consistent dynamics is recurrent oscillatory motion. $\square$

For integrable systems, the Arnold–Liouville theorem (?) guarantees action-angle variables (J_i, θ_i) with quasi-periodic motion $\dot{\theta}_i = \omega_i(J)$. The Koopman operator (?) on $L^2(\Omega, \mu)$ has a pure point spectrum on cyclic subspaces; its eigenfrequencies $\{\omega_k\}$ are the mode frequencies of the system. Near-integrable systems retain this structure on most tori by the KAM theorem (???).

3 Partition Coordinates and Shell Capacity

3.1 Hierarchical partition and coordinate generation

The hierarchical partition of Axiom ??(iii) is not merely formal. For a three-dimensional bounded system with $SO(3)$ symmetry, the refinement sequence generates a natural four-coordinate label on each partition cell.

Definition 3.1 (Partition State). A *partition state* of a bounded three-dimensional Hamiltonian system is an equivalence class of phase-space trajectories sharing the same closure label (n, ℓ, m, s) where: $n \in \{1, 2, 3, \dots\}$ (principal level, radial closure count), $\ell \in \{0, 1, \dots, n-1\}$ (angular sublevel, polar closure count), $m \in \{-\ell, \dots, +\ell\}$ (azimuthal projection), $s \in \{-\frac{1}{2}, +\frac{1}{2}\}$ (rotational parity). The set of all partition states is $\mathcal{P}$.

The label (n, ℓ, m, s) is derived from the closure conditions of orbits in action-angle coordinates. An orbit closes when $n_r \omega_r = n_\theta \omega_\theta = n_\phi \omega_\phi$ for positive integers; the hierarchy of radial, polar, and azimuthal boundaries produces the nesting $0 \leq \ell \leq n-1$, $|m| \leq \ell$, and the binary parity s from the double cover $\text{Spin}(3) \cong SU(2) \rightarrow SO(3)$ (?).

Proof. At level n , ℓ ranges over $\{0, \dots, n-1\}$; for each ℓ , m ranges over $2\ell+1$ values; s contributes a factor of 2:

$$C(n) = 2 \sum_{\ell=0}^{n-1} (2\ell+1) = 2(n + 2 \cdot \frac{n(n-1)}{2}) = 2n^2. \quad \square \quad (2)$$

Corollary 3.3 (Cumulative Count). *The total number of partition states at or below level $n_{\max}$ is*

$$N_{\text{state}}(n_{\max}) = \sum_{n=1}^{n_{\max}} 2n^2 = \frac{n_{\max}(n_{\max}+1)(2n_{\max}+1)}{3}. \quad (3)$$

Remark 3.4. The sequence $C(n) = 2, 8, 18, 32, 50, 72, 98, \dots$ is the electron shell capacity sequence of the periodic table. This is not coincidental: both arise from the same $SO(3)$ representation theory applied to a bounded three-dimensional oscillatory system.

3.2 The partition bijection

Theorem 3.5 (Partition Bijection). *There exists a bijection $\Phi: \mathbb{Z}^+ \rightarrow \mathcal{P}$ mapping each positive integer M to a unique partition state (n, ℓ, m, s) . The map is constructive: given M , the principal level is $n = \lfloor \sqrt{M/2} \rfloor + 1$ (adjusted to the correct shell), ℓ and m follow from the lexicographic ordering within the shell, and $s = \pm \frac{1}{2}$ from the parity index.*

Proof. Injectivity: distinct M values yield distinct shell positions in the lexicographic ordering, hence distinct (n, ℓ, m, s) tuples. Surjectivity: every valid tuple (n, ℓ, m, s) has a unique cumulative index $M = N_{\text{state}}(n-1) + \delta$ where δ is the within-shell offset; this M maps back to (n, ℓ, m, s) by construction. $\square$

Theorem 3.6 (Strict Monotonicity of the Partition Count). *Let $M(t)$ be the partition count of a bounded oscillatory system with oscillation frequency $f > 0$. Then $M(t) = ft$ exactly, and $M(t_2) > M(t_1)$ whenever $t_2 > t_1$ and at least one cycle has completed.*

Proof. Each completed oscillation cycle increments M by exactly 1. The rate is $dM/dt = f = \omega/(2\pi)$, exact by definition of the oscillation count. Strict monotonicity follows since $f > 0$. $\square$

4 Composition-Inflation and Angular Resolution

4.1 Integer compositions

Definition 4.1 (Composition). An *integer composition* of n is an ordered sequence $(c_1, \dots, c_k)$ of positive inte-

gers summing to n . Two compositions differing in order are distinct.

Theorem 4.2 (Composition Count ?). *The total number of compositions of n is 2^{n-1} .*

Proof. Place n objects in a row; there are $n - 1$ gaps. Each gap either contains a divider or does not. The 2^{n-1} binary choices on the gaps are in bijection with compositions. $\square$ $\square$

The number of compositions into exactly k parts is $\binom{n-1}{k-1}$.

4.2 Dimensional labeling

The S-entropy space is the unit cube $[0, 1]^d$ with d entropy dimensions. For $d = 3$ (knowledge S_k , temporal S_t , evolution S_e), any oscillatory trajectory through this space can be characterized by which dimension it refines at each step.

Definition 4.3 (Labeled Composition). A *labeled composition* of n in d dimensions is a pair $(\mathbf{C}, \mathbf{L})$ where $\mathbf{C} = (c_1, \dots, c_k)$ is a composition of n and $\mathbf{L} = (\ell_1, \dots, \ell_k)$ with $\ell_i \in \{1, \dots, d\}$ specifies the entropy dimension refined during the i -th part of the trajectory.

Theorem 4.4 (Composition-Inflation Formula). *The total number of labeled compositions of n in d dimensions is*

$$T(n, d) = d \cdot (d + 1)^{n-1}. \quad (4)$$

Proof. Summing over all numbers of parts:

$$T(n, d) = \sum_{k=1}^n \binom{n-1}{k-1} d^k = d \sum_{j=0}^{n-1} \binom{n-1}{j} d^j = d(1+d)^{n-1} \quad (5)$$

where the last step is the binomial theorem with $x = d$, $m = n - 1$. $\square$

Proposition 4.5 (Growth Properties). $T(1, d) = d$; $T(n + 1, d) = (d + 1) \cdot T(n, d)$ (each additional cycle multiplies by $d + 1$); $\log T(n, d) = \log d + (n - 1) \log(d + 1)$; $T(n, d)/n \rightarrow \infty$ as $n \rightarrow \infty$.

Corollary 4.6 (S-Entropy Space). For $d = 3$: $T(n, 3) = 3 \cdot 4^{n-1}$. After $n = 56$ cycles, $T(56, 3) = 3 \cdot 4^{55} \approx 3.8 \times 10^{33}$.

4.3 Trajectory distinguishability

Theorem 4.7 (Position-Trajectory Duality). *In the ternary encoding of the unit cube $[0, 1]^3$, the trit string encodes both the final position and the trajectory taken to reach it. Two distinct labeled compositions produce distinct trit strings and are therefore categorically distinguishable.*

Proof. Case 1: distinct underlying compositions. Different sequences of segment lengths determine transitions at different depths, yielding distinct trit strings. Case

2: same composition, distinct labels. There exists index i with $\ell_i \neq \ell'_i$; the i -th segment refines different axes, producing distinct trit strings by orthogonality of the dimensions. Both cases yield distinct categorical states. $\square$ $\square$

Theorem 4.8 (Non-Demolition Distinguishability). *Let $\hat{O}_{\text{cat}}$ measure categorical trajectory labels and $\hat{O}_{\text{phys}}$ measure any physical observable (position, momentum, energy). Then $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$. Measuring which trajectory was taken does not disturb the physical state.*

Proof. $\hat{O}_{\text{cat}}$ acts on the trajectory label — a topological property of the path. $\hat{O}_{\text{phys}}$ acts on the instantaneous phase-space position. These act on complementary degrees of freedom: the trajectory label is constant along the trajectory (a conserved quantity), hence commutes with the Hamiltonian and with all physical observables it generates. Since $\hat{O}_{\text{cat}}$ is diagonal in the trajectory basis and $\hat{O}_{\text{phys}}$ acts on the physical space factor, their tensor product structure guarantees $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$. $\square$ $\square$

4.4 Angular resolution and the Planck constraint

Definition 4.9 (Angular Resolution). For $T(n, d)$ distinguishable trajectories distributed uniformly over $[0, 2\pi)$, the angular resolution is

$$\Delta\theta(n, d) = \frac{2\pi}{T(n, d)} = \frac{2\pi}{d(d+1)^{n-1}}.$$

Theorem 4.10 (No Planck Limit on Angular Resolution). *The Planck time $t_P = \sqrt{\hbar G/c^5} \approx 5.39 \times 10^{-44}$ s bounds temporal intervals measured in seconds. The angular resolution $\Delta\theta(n, d) = 2\pi/T(n, d)$ is dimensionless (units of radians) and is therefore not bounded below by any function of t_P . For any $\varepsilon > 0$, there exists*

$$n_0 = 1 + \left\lceil \log_{d+1} \left(\frac{2\pi}{d\varepsilon} \right) \right\rceil$$

such that $\Delta\theta(n, d) < \varepsilon$ for all $n \geq n_0$.

Proof. t_P has dimension $[t_P] = \text{s}$. The Heisenberg bound $\Delta E \cdot \Delta t \geq \hbar/2$ constrains energy–time products; both factors carry physical dimensions. $\Delta\theta = 2\pi/T(n, d)$ is the ratio of two dimensionless quantities and carries no time dimension: $[\Delta\theta] = \text{rad} = 1$. Neither the Heisenberg argument nor the gravitational collapse argument applies to a dimensionless ratio.

Since $T(n, d) = d(d + 1)^{n-1}$ is unbounded, $\Delta\theta \rightarrow 0$ as $n \rightarrow \infty$. Setting $\Delta\theta < \varepsilon$ and solving gives n_0 as stated. $\square$

Remark 4.11. The composition-inflation mechanism does not create shorter time intervals. It creates more distinguishable trajectory structures within the same time interval. The Planck limit bounds temporal duration; composition-inflation operates on categorical multiplicity. These are fundamentally different quantities.

Chapter 1. Paper 1. On the Partition Algebra of Bounded Phase Space:

4.5 Composition-Inflation, Angular Resolution, and the Categorical Reformulation of The Planck depth Fundamental Constants

Definition 4.12 (Planck Depth). The *Planck depth* n_P for a reference oscillator with period τ_{osc} in d -dimensional entropy space is the minimum cycle count for which the categorical trajectory count equals or exceeds the number of Planck-time intervals in one oscillator period:

$$T(n_P, d) \geq \frac{\tau_{\text{osc}}}{t_P}.$$

Theorem 4.13 (Planck Depth Formula).

$$n_P = 1 + \left\lceil \log_{d+1} \left(\frac{\tau_{\text{osc}}}{d \cdot t_P} \right) \right\rceil.$$

Proof. Solve $d(d+1)^{n_P-1} \geq \tau_{\text{osc}}/t_P$ for n_P . $\square$ $\square$

Theorem 4.14 (Universality of Planck Depth). *For any physically realizable oscillator with frequency ν satisfying $t_P^{-1} > \nu > 1$ Hz, the Planck depth at $d = 3$ lies in $n_P \in [36, 73]$. The variation spans only 37 cycles despite frequencies varying over 14 orders of magnitude, because n_P depends logarithmically on ν .*

Proof. At the lower frequency extreme ($\nu = 1$ Hz): $\tau/t_P = 1.86 \times 10^{43}$, $n_P = 1 + \lceil \log_4(1.86 \times 10^{43}/3) \rceil = 73$. At the strontium optical clock ($\nu = 4.29 \times 10^{14}$ Hz): $\tau/t_P = 4.32 \times 10^{28}$, $n_P = 48$. The logarithm compresses 14 orders of magnitude in frequency to $73 - 48 = 25$ cycles. $\square$

Table 1: Planck depth for representative oscillators ($d = 3$).

Oscillator	ν (Hz)	τ/t_P	n_P
Sr optical clock	4.29×10^{14}	4.3×10^{28}	48
Cs-133 hyperfine	9.19×10^9	2.0×10^{33}	56
LHC bunch clock	4.01×10^7	4.6×10^{35}	60
H maser	1.42×10^9	1.3×10^{34}	57
CPU (3 GHz)	3.00×10^9	6.2×10^{33}	57

5 Categorical Angular Units and Constant Reduction

5.1 The speed of light

Theorem 5.1 (Speed of Light as Angular Velocity). *In categorical angular units, the speed of light is exactly*

$$c = 2\pi \text{ rad/tick}. \quad (6)$$

Proof. In one oscillation cycle (one tick), a bounded system traverses the full phase $\Delta\phi = 2\pi$ rad of its reference oscillation. The maximum rate at which categorical distinctions propagate between two points separated by Δx at the characteristic radius r_0 is $c = \Delta x/\tau_c = 2\pi r_0/\tau_{\text{tick}}$ where $\tau_{\text{tick}} = 1$ tick. In angular units $\theta = x/r_0$: $c = 2\pi \text{ rad/tick}$.

The partition propagation bound establishes that $c = \sup_n \ell_n \omega_n$ is finite and equals this value; the bound is attained at the massless (zero-rest-frequency) limit. $\square$

5.2 Planck's constant

Theorem 5.2 (Planck's Constant as Energy per Radian).

$$\hbar = E_{\text{tick}}/(2\pi), \quad (7)$$

where E_{tick} is the energy carried by one oscillation tick.

Proof. From $E = \hbar\omega$ with $\omega = 2\pi/\tau_{\text{tick}} = 2\pi \text{ tick}^{-1}$: $E_{\text{tick}} = \hbar \cdot 2\pi$. Solving: $\hbar = E_{\text{tick}}/(2\pi)$. $\hbar$ is the energy per radian of phase advance. $\square$

5.3 Boltzmann's constant

Theorem 5.3 (Boltzmann's Constant as Entropy per Composition Step).

$$k_B = E_{\text{tick}}/\ln(d+1). \quad (8)$$

Proof. By Proposition ??(ii), each additional oscillation tick multiplies the trajectory count by $(d+1)$, contributing $\ln(d+1)$ natural units of categorical entropy. The thermal energy per degree of freedom is $k_B T$, where T is the tick rate. The energy per unit of composition entropy is $E_{\text{tick}}/\ln(d+1)$. For $d = 3$: $k_B = E_{\text{tick}}/\ln 4$. $\square$

5.4 Mass and rest energy

Corollary 5.4 (Mass and Rest Energy). *The inertial mass and rest energy of a localized oscillator with rest frequency ν_0 (ticks^{-1}) are*

$$m_0 = \frac{E_{\text{tick}} \cdot \nu_0}{4\pi^2}, \quad (9)$$

$$E_0 = m_0 c^2 = E_{\text{tick}} \cdot \nu_0. \quad (10)$$

Proof. From $m_0 = \hbar\omega_0/c^2$ with $\hbar = E_{\text{tick}}/(2\pi)$ and $c = 2\pi \text{ rad/tick}$: $m_0 = E_{\text{tick}}\omega_0/(2\pi \cdot 4\pi^2) = E_{\text{tick}}\nu_0/(4\pi^2)$ where $\nu_0 = \omega_0/(2\pi)$. Then $E_0 = m_0 c^2 = E_{\text{tick}}\nu_0/(4\pi^2) \cdot 4\pi^2 = E_{\text{tick}}\nu_0$. The identity $E_0 = \hbar\omega_0$ is recovered: rest energy is the fundamental frequency-energy identity applied to the localization mode. $\square$

5.5 The Planck time and the irreducibility of G

Theorem 5.5 (Planck Time Decomposition).

$$t_P = \sqrt{\hbar G/c^5} = \frac{\sqrt{E_{\text{tick}} \cdot G}}{(2\pi)^3}. \quad (11)$$

The Planck time has two components: geometric factors $(2\pi)^3$ from the angular structure of oscillation, and the product $\sqrt{E_{\text{tick}} \cdot G}$ as the unique irreducible coupling.

Proof. Substitute $\hbar = E_{\text{tick}}/(2\pi)$ and $c = 2\pi \text{ rad/tick}$: $t_P = \sqrt{E_{\text{tick}}G/(2\pi \cdot (2\pi)^5)} = \sqrt{E_{\text{tick}}G}/(2\pi)^3$. $\square$

Theorem 5.6 (Irreducibility of G). *The gravitational constant G is the unique fundamental constant of physics that does not reduce to a combination of tick counts, factors of 2π , and the energy unit E_{tick} .*

Proof. Table ?? shows the reduction of six of the seven fundamental constants. The remaining constant G appears only in t_P and the Schwarzschild radius; in both cases it appears as an irreducible multiplicative factor that cannot be expressed in terms of counting operations or geometry. It encodes the coupling between categorical partition structure (which determines mass through $m_0 = E_{\text{tick}}\nu_0/(4\pi^2)$) and the curvature of space. No counting argument reduces this coupling. $\square$

Table 2: Fundamental constants in categorical angular units.

Constant	SI value	Categorical form
c	$3.00 \times 10^8 \text{ m/s}$	$2\pi \text{ rad/tick}$
$\hbar$	$1.055 \times 10^{-34} \text{ J} \cdot \text{s}$	$E_{\text{tick}}/(2\pi)$
k_B	$1.381 \times 10^{-23} \text{ J/K}$	$E_{\text{tick}}/\ln(d+1)$
m_0	$\hbar\omega_0/c^2$	$E_{\text{tick}}\nu_0/(4\pi^2)$
E_0	m_0c^2	$E_{\text{tick}}\nu_0 = \hbar\omega_0$
t_P	$5.39 \times 10^{-44} \text{ s}$	$\sqrt{E_{\text{tick}}G}/(2\pi)^3$
G	$6.674 \times 10^{-11} \text{ m}^3\text{kg}^{-1}\text{s}^{-2}$	irreducible

Corollary 5.7 (Planck Depth from G). *Since $t_P = \sqrt{E_{\text{tick}}G}/(2\pi)^3$ and n_P depends on t_P through Theorem ??, the Planck depth is computable to precision $(d+1)^{-n}$ once E_{tick} and G are known. Since G is the unique irreducible constant, the Planck depth is the one quantity in the entire categorical framework that is not derivable from geometry and counting alone. It is the fingerprint of gravity in the composition-inflation structure.*

6 Discussion and Outlook

6.1 What has been established

The paper has derived, from a single empirical axiom:

- (1) Oscillatory necessity — bounded dynamics must oscillate (Corollary ??).
- (2) The partition coordinate system (n, ℓ, m, s) with capacity $C(n) = 2n^2$ (Theorem ??).
- (3) The partition bijection $\Phi : \mathbb{Z}^+ \rightarrow \mathcal{P}$ and strict monotonicity of the partition count (Theorems ??, ??).
- (4) The composition-inflation formula $T(n, d) = d(d+1)^{n-1}$ (Theorem ??).
- (5) Angular resolution is dimensionless and Planck-unconstrained (Theorem ??).
- (6) The Planck depth formula and its universality across oscillators (Theorems ??, ??).
- (7) All fundamental constants except G reduce to 2π , tick counts, and E_{tick} (Theorems ??-?? and Table ??).
- (8) G is the unique irreducible constant (Theorem ??).

6.2 Applications developed in companion papers

The partition algebra derived here is the mathematical foundation for a sequence of companion papers. The following applications are developed there, citing only results proved above:

LHC trigger design. A trigger system with d independent detector subsystems observing n bunch crossings can discriminate $T(n, d)$ distinct event categories. The Planck depth $n_P = 60$ for the LHC bunch clock ($\nu_B = 40 \text{ MHz}$, $d = 3$) defines a natural design depth; the current L1 trigger at $n = 100$ operates at $n/n_P = 1.67$, placing it in the super-Planck categorical resolution regime (??).

Mass spectrometry. The partition Lagrangian $\mathcal{L}_{\mathcal{M}} = \frac{1}{2}\mu|\dot{\mathbf{x}}|^2 + \mu\dot{\mathbf{x}} \cdot \mathbf{A}_{\mathcal{M}} - \mathcal{M}(\mathbf{x}, t)$ derives all four mass analyzer equations (TOF, quadrupole, Orbitrap, FT-ICR) from the same Euler-Lagrange equations, with the Lorentz force as a consequence rather than a postulate (?).

Gravitational constant. Three independent routes to G via the partition algebra — oscillation-ratio, category fixed-point, and partition-density ratio — converge on the same value with $(d+1)^{-n}$ precision. At $n = 8$ all three agree with CODATA; at $n = 27$ they reach double-precision machine epsilon (?).

Cosmology. A partition-density-dependent G reproduces the MOND acceleration scale $a_0 = cH_0/(2\pi)$, predicts dark energy equation of state $w_{\text{eff}} = -1 + 1/(d+1) = -0.75$, and secular drift $\dot{G}/G = -3H_0/(d+1)$ (?).

6.3 Falsification conditions

The framework is falsified by any of the following:

- A persistent physical system with infinite Liouville measure.
- A bounded oscillatory system in which the partition count $M(t)$ decreases (i.e., a partition decrement is physically realized).
- A physical observable that cannot be expressed as a function of $(n, \ell, m, s, E_{\text{tick}}, G)$.

None of these conditions has been observed.

Acknowledgements

This work was supported by the AIME Registry for Artificial Intelligence.

The Planck Depth and Optimal Trigger Design: Composition-Inflation Applied to Real-Time Event Selection at the Large Hadron Collider

Kundai Farai Sachikonye
 Technical University of Munich
 Chair of Proteomics and Bioanalytics
 kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

The Large Hadron Collider produces proton–proton collisions at 40.079 MHz. The trigger system must reduce this rate to approximately 1 kHz for permanent storage — a rejection fraction of 99.997 % in under 2.5 μ s. We demonstrate that this design problem has a mathematically natural solution grounded in the composition-inflation formula $T(n, d) = d(d + 1)^{n-1}$ derived from the partition algebra of bounded phase space.

The central result is the *Planck depth* n_P for the LHC bunch clock: the minimum number of bunch crossings after which the categorical trajectory count $T(n_P, d)$ equals or exceeds the ratio τ_B/t_P of the bunch-crossing period to the Planck time. For the LHC at $d = 3$ independent detector subsystems and $\nu_B = 40.079$ MHz, we derive $n_P = 60$ exactly. The current L1 trigger latency of 100 bunch crossings operates at $n/n_P = 1.67$, placing it in the super-Planck categorical resolution regime.

We derive three additional results with direct engineering implications. First, the 25 ns bunch-crossing window is not an arbitrary engineering choice: it equals the partition propagation time $\tau_c = \Delta x/c$ for the ATLAS detector radius, and is therefore determined by the geometry of the interaction region and the speed of light, both of which are consequences of the partition framework. Second, the partition uncertainty relation $\Delta\mathcal{M} \cdot \tau_p \geq \hbar$ implies a minimum detectable partition depth change of 26 eV per bunch crossing, which is the fundamental resolution floor of the trigger — not an engineering parameter. Third, the HL-LHC upgrade adding a fourth independent detector subsystem ($d : 3 \rightarrow 4$) reduces the Planck depth from 60 to 52, recovering eight bunch crossings (200 ns) of categorical resolution headroom — a quantitative prediction derivable from first principles without any knowledge of the specific detector technology.

We also derive the *composition efficiency* η_C and the *Planck Utilisation Index* (PUI) as dimensionless metrics characterising how much of the available categorical resolution the trigger exploits. The current ATLAS trigger menu of approximately 10^3 paths achieves $\eta_C \approx 10\%$:

90% of the theoretically available categorical resolution remains unused. These metrics provide a framework-grounded basis for trigger optimisation that does not depend on Monte Carlo simulation of physics processes.

Keywords: LHC trigger; composition-inflation; Planck depth; bunch crossing; categorical resolution; trigger design; HL-LHC; partition algebra; timing window; detector subsystems

Contents

1	Introduction	2
1.1	The event selection problem	2
1.2	Summary of results	2
1.3	Organisation	2
2	Composition-Inflation Applied to Event Selection	2
2.1	The naive state count and why it fails	2
2.2	Labeled compositions and the trigger	3
2.3	The inflation factor	3
3	The Planck Depth for LHC Trigger Systems	3
3.1	Definition and physical significance	3
3.2	The LHC Planck depth	4
3.3	Universality across LHC-relevant oscillators	4
4	Physical Origin of the 25 ns Timing Window	4
4.1	The partition propagation formula	4
4.2	The partition uncertainty relation	5
5	Detector Subsystems as Commuting Observables	5
5.1	The partition coordinate assignment	5
5.2	The commutation theorem	5
5.3	The QND trigger theorem	6
6	Single-Particle Thermodynamics of Detected Events	6
6.1	Categorical temperature	6
6.2	Categorical temperature at the LHC	6

7 Trigger Design Principles from Partition Theory	7
7.1 The temporal structure of the trigger window	7
7.2 Optimal cell structure	7
8 The HL-LHC Upgrade	7
8.1 Adding a fourth dimension	7
8.2 State count at HL-LHC	8
9 Trigger Performance Metrics	8
9.1 Composition efficiency	8
9.2 Planck Utilisation Index	8
9.3 Summary of metrics for current and future detectors	8
10 Discussion and Outlook	8
10.1 What has been established	8
10.2 Open questions	9
10.3 Relation to conventional trigger design	9

1 Introduction

1.1 The event selection problem

The Large Hadron Collider at CERN delivers proton-proton collisions at a rate of 40.079 MHz (Evans and Bryant, 2008). Each collision event, if fully recorded, produces approximately 1–2 MB of raw detector data across the ATLAS and CMS detectors (ATLAS Collaboration, 2008; CMS Collaboration, 2008). The aggregate data rate is thus of order 80 TB per second — several orders of magnitude beyond any available storage technology.

The trigger system must reduce this rate in real time. The current ATLAS Level-1 (L1) trigger accepts approximately 100 kHz, which the High-Level Trigger (HLT) then reduces to approximately 1 kHz for permanent storage (ATLAS Collaboration, 2020b). The L1 decision must be made within a fixed pipeline latency of $2.5 \mu\text{s}$ = 100 bunch-crossing periods, after which the detector front-end buffers are overwritten.

The event selection problem therefore has three constraints that are simultaneously tight, independent, and non-negotiable:

- (i) *Rate*: accept ≤ 100 kHz from 40.079 MHz input ($\leq 0.25\%$).
- (ii) *Latency*: decide within 100 bunch crossings ($2.5 \mu\text{s}$).
- (iii) *Irreversibility*: discarded events are permanently lost.

Existing formal frameworks for trigger design are empirical and simulation-based: the trigger menu is a list of physics-motivated conditions, each calibrated against Monte Carlo simulations of signal and background processes. The theoretical question — *what is the maximum number of distinct event categories a trigger of given depth can discriminate?* — has not previously been answered from first principles. This paper answers it.

1.2 Summary of results

The key results, all derived from the partition algebra of bounded phase space without additional physical postulates:

- (1) **Trigger State Inflation** (Theorem 2.2): a trigger with d independent subsystems and n bunch crossings can discriminate $T(n, d) = d(d+1)^{n-1}$ distinct event categories.
- (2) **LHC Planck Depth** (Theorem 3.3): $n_P = 60$ for the LHC at $d = 3$.
- (3) **Timing Window Derivation** (Theorem 4.1): the 25 ns bunch-crossing window equals the partition propagation time across the ATLAS detector interaction region.
- (4) **Partition Uncertainty** (Theorem 4.3): $\Delta\mathcal{M} \cdot \tau_p \geq \hbar$ implies a 26 eV minimum detectable energy difference per bunch crossing.
- (5) **HL-LHC Dimension Advantage** (Theorem 8.1): $d : 3 \rightarrow 4$ reduces $n_P : 60 \rightarrow 52$, saving 8 crossings.
- (6) **Categorical Efficiency Metrics** (Definitions 9.1–9.4): current ATLAS trigger operates at $\eta_C \approx 10\%$, PUI = 1.67.
- (7) **QND Multi-Subsystem Classification** (Theorem 5.4): all four detector subsystem observables commute, enabling simultaneous classification without quantum backaction.
- (8) **Single-Particle Thermodynamics** (Theorem 6.2): each detected particle satisfies $PV = k_B T_{\text{cat}}$ where $T_{\text{cat}} = \hbar\omega_c/(2\pi k_B)$.

1.3 Organisation

Section 2 derives the trigger state inflation formula. Section 3 derives the Planck depth and validates it against the LHC parameters. Section 4 establishes the physical origin of the timing window. Section 5 maps detector subsystems to commuting partition coordinates. Section 6 develops single-particle thermodynamics for the trigger. Section 7 develops trigger design principles from the theory. Section 8 analyses the HL-LHC upgrade. Section 9 defines the efficiency metrics. Section 10 discusses implications and open questions.

All theorems are proved from first principles or cited from the companion mathematical foundation paper (Sachikonye, d). The paper is self-contained: every symbol is defined at first use.

2 Composition-Inflation Applied to Event Selection

2.1 The naive state count and why it fails

A trigger system with d independent detector subsystems observing one bunch crossing assigns each crossing to one of at most d single-subsystem categories. Over n crossings, the naive counting — independent binary decisions — gives 2^n possible outcomes.

Chapter 2. Paper 2. The Planck Depth and Optimal Trigger Design: Composition-Inflation

Applied to Real-Time Event Selection at the Large Hadron Collider

This count is wrong. It treats each crossing as independent and ignores the combinatorial structure of how n crossings can be distributed among categories. The correct count is derived from the number of *labeled integer compositions* of n in d dimensions, which is qualitatively different (exponential with base $d + 1$, not 2).

2.2 Labeled compositions and the trigger

Definition 2.1 (Trigger Labeled Composition). A *trigger labeled composition* of n bunch crossings in d detector dimensions is a pair $(\mathbf{C}, \mathbf{L})$ where $\mathbf{C} = (c_1, \dots, c_k)$ is an ordered partition of n into k positive integer parts and $\mathbf{L} = (\ell_1, \dots, \ell_k)$ with $\ell_i \in \{1, \dots, d\}$ specifies which detector subsystem characterises the i -th group of crossings.

The physical meaning is as follows. A trigger that records which detector subsystem dominates each contiguous group of crossings — rather than treating each crossing independently — captures the temporal structure of the event. An event that fires the EM calorimeter for 3 crossings then the muon system for 2 crossings is categorically distinct from one that fires the muon system first, even if the total crossing counts are the same. The composition captures this ordering.

Theorem 2.2 (Trigger State Inflation). *A trigger system with d independent detector subsystems observing n bunch crossings can discriminate*

$$T(n, d) = d \cdot (d + 1)^{n-1} \quad (1)$$

categorically distinct event patterns.

Proof. Summing labeled compositions over all numbers of parts k :

$$T(n, d) = \sum_{k=1}^n \binom{n-1}{k-1} d^k = d \sum_{j=0}^{n-1} \binom{n-1}{j} d^j = d(1+d)^{n-1} \quad (2)$$

by the binomial theorem. This is proved in full generality in Theorem 3.3 of [Sachikonye \(d\)](#); we repeat the proof here for completeness. $\square$

Proposition 2.3 (Growth Properties). *The trigger state count satisfies:*

- (i) $T(1, d) = d$ (one crossing: exactly d categories).
- (ii) $T(n + 1, d) = (d + 1) \cdot T(n, d)$ (each additional crossing multiplies by $d + 1$).
- (iii) $\log T(n, d) = \log d + (n - 1) \log(d + 1)$ (exponential growth in n).
- (iv) $T(n, d)/n \rightarrow \infty$ as $n \rightarrow \infty$ (superlinear).
- (v) For $d = 3$: $T(n, 3) = 3 \cdot 4^{n-1}$.

Proof. Direct from Equation (1). $\square$ $\square$

Example 2.4 (Current LHC L1 Trigger). The current ATLAS L1 trigger has $d = 3$ primary subsystems

(tracker/EM calorimeter, hadronic calorimeter, muon spectrometer) and a fixed latency of $n = 100$ bunch crossings. The available trigger state space is:

$$T(100, 3) = 3 \cdot 4^{99} \approx 3.0 \times 10^{59}.$$

The current ATLAS trigger menu contains approximately 10^3 distinct trigger paths. The fraction of the available state space exploited is $\eta_C \approx 10^3/10^{59} = 10^{-56}$ — or, measured logarithmically, about 10% of the Planck depth capacity (see Definition 9.1 for the precise metric).

Remark 2.5 (Why the naive count fails). The naive count $2^n = 2^{100} \approx 10^{30}$ (binary per-crossing decisions) dramatically undercounts the available state space by 10^{29} because it discards ordering information. The labeled composition count $T(100, 3) \approx 10^{59}$ captures the full temporal structure of how events unfold over multiple crossings. The difference is not a correction factor — it is a fundamentally different combinatorial object.

2.3 The inflation factor

Definition 2.6 (Inflation Factor). The *inflation factor* is the ratio of the composition-inflation count to the naïve linear count:

$$I(n, d) = \frac{T(n, d)}{d \cdot n} = \frac{(d + 1)^{n-1}}{n}.$$

For $n = 60$, $d = 3$: $I(60, 3) = 4^{59}/60 \approx 5 \times 10^{34}$. The composition-inflation mechanism generates 5×10^{34} more distinguishable patterns than naïve linear counting. This is the quantitative statement that temporal structure is exponentially richer than instantaneous state.

3 The Planck Depth for LHC Trigger Systems

3.1 Definition and physical significance

Definition 3.1 (Planck Depth). The *Planck depth* n_P for a reference oscillator with period τ_{osc} in d -dimensional entropy space is the minimum cycle count n for which the categorical trajectory count equals or exceeds the ratio of the oscillator period to the Planck time:

$$T(n_P, d) \geq \frac{\tau_{\text{osc}}}{t_P}, \quad t_P = \sqrt{\frac{\hbar G}{c^5}} = 5.391 \times 10^{-44} \text{ s}. \quad (3)$$

The physical significance is as follows. The ratio τ_{osc}/t_P counts the number of Planck-time intervals within one oscillator period — the maximum number of temporal states that could in principle be resolved if resolution were limited only by the Planck time. The composition-inflation mechanism generates $T(n_P, d)$ categorically distinguishable trajectories within the same period; at depth n_P , these two counts become equal.

Above the Planck depth, the categorical trajectory count exceeds the maximum temporal state count: the system has more distinct categorical patterns than there

are Planck-time intervals. Resolution is no longer limited by the Planck time; it is limited by the available composition depth.

Theorem 3.2 (Planck Depth Formula).

$$n_P = 1 + \left\lceil \log_{d+1} \left(\frac{\tau_{\text{osc}}}{d \cdot t_P} \right) \right\rceil. \quad (4)$$

Proof. Solve $d(d+1)^{n_P-1} \geq \tau_{\text{osc}}/t_P$ for n_P : $(d+1)^{n_P-1} \geq \tau_{\text{osc}}/(dt_P)$, $n_P - 1 \geq \log_{d+1}(\tau_{\text{osc}}/(dt_P))$, and the minimum integer satisfying this is as stated. $\square$

3.2 The LHC Planck depth

Theorem 3.3 (LHC Planck Depth). *For the LHC bunch clock with $\nu_B = 40.079$ MHz, bunch-crossing period $\tau_B = 24.951$ ns, and $d = 3$ independent detector subsystems:*

$$n_P = 60. \quad (5)$$

This is reached in physical time $t_{60} = 60/\nu_B = 1.496 \mu\text{s}$.

Proof. Step 1: ratio.

$$\frac{\tau_B}{t_P} = \frac{24.951 \times 10^{-9} \text{ s}}{5.391 \times 10^{-44} \text{ s}} = 4.628 \times 10^{35}.$$

Step 2: apply formula.

$$\begin{aligned} n_P &= 1 + \left\lceil \log_4 \left(\frac{4.628 \times 10^{35}}{3} \right) \right\rceil = 1 + \left\lceil \log_4(1.543 \times 10^{35}) \right\rceil \\ &= 1 + \left\lceil \frac{\ln(1.543 \times 10^{35})}{\ln 4} \right\rceil = 1 + \left\lceil \frac{81.10}{1.386} \right\rceil = 1 + \lceil 58.51 \rceil \end{aligned} \quad (6)$$

Step 3: verification. $T(60, 3) = 3 \cdot 4^{59} = 3 \cdot 3.076 \times 10^{35} = 9.23 \times 10^{35} > 4.628 \times 10^{35} \checkmark$.

Step 4: check $n = 59$ does not suffice. $T(59, 3) = 3 \cdot 4^{58} = 2.308 \times 10^{35} < 4.628 \times 10^{35}$. So $n_P = 59$ fails and $n_P = 60$ is the minimum. $\square$

Remark 3.4 (The L1 trigger latency in context). The current ATLAS L1 trigger latency is 100 bunch crossings ($2.5 \mu\text{s}$). At $n = 100$, $T(100, 3) = 3 \cdot 4^{99} \approx 3.0 \times 10^{59}$, which is $3.0 \times 10^{59}/4.628 \times 10^{35} \approx 6.5 \times 10^{23}$ times the Planck threshold. The trigger operates at depth $n/n_P = 100/60 = 1.67$ times the Planck depth. The entire L1 latency budget is in the super-Planck categorical resolution regime.

This is not a coincidence. The $2.5 \mu\text{s}$ latency was set by the requirements of the front-end pipeline design, but the Planck depth argument shows that any latency budget exceeding $n_P = 60$ crossings places the trigger in the regime where categorical resolution is no longer the limiting factor — timing precision is.

3.3 Universality across LHC-relevant oscillators

Theorem 3.5 (LHC-Scale Universality). *For any timing reference at the LHC with frequency ν in the range $[10^6, 10^{15}]$ Hz (covering everything from the revolution frequency to optical clock references), the Planck depth*

at $d = 3$ satisfies $n_P \in [48, 64]$. The variation spans only 16 cycles despite frequencies varying over nine orders of magnitude.

Proof. Lower bound: strontium optical clock at $\nu = 4.29 \times 10^{14}$ Hz gives $\tau/t_P = 4.32 \times 10^{28}$, $n_P = 48$. Upper bound: LHC revolution frequency at $\nu = 11.2$ kHz gives $\tau/t_P = 1.66 \times 10^{39}$, $n_P = 64$. The logarithm in Equation (4) compresses nine orders of magnitude in frequency to 16 cycles. $\square$

Table 1: Planck depth for LHC-relevant timing references ($d = 3$).

Reference	ν (Hz)	τ/t_P	n_P
Sr optical clock	4.29×10^{14}	4.3×10^{28}	48
Cs-133 (atomic clock)	9.19×10^9	2.0×10^{33}	56
LHC bunch clock	4.01×10^7	4.6×10^{35}	60
L1 trigger clock (RF/8)	5.01×10^6	3.7×10^{36}	61
LHC revolution frequency	1.12×10^4	1.7×10^{39}	64

4 Physical Origin of the 25 ns Timing Window

The 25 ns bunch-crossing window is conventionally described as an engineering consequence of the LHC bunch structure. We show that it is, more fundamentally, the *partition propagation time* for the ATLAS interaction region — a consequence of the speed of light and the detector geometry.

4.1 The partition propagation formula

From the categorical angular reformulation of the speed of light (Sachikonye, d), the maximum rate at which categorical distinctions propagate between two points separated by distance Δx is:

$$\tau_c = \frac{\Delta x}{c}. \quad (7)$$

In one partition propagation time τ_c , the leading edge of a signal from the interaction point reaches the outermost detector element. For the partition framework to assign all signals from a single collision to the same event, the integration window must equal at least τ_c .

Theorem 4.1 (Timing Window from Detector Geometry). *For a cylindrically symmetric detector of inner radius r_1 and outer radius r_2 , the minimum timing window that allows unambiguous association of detector hits with a single bunch crossing is:*

$$\tau_{\text{BX}} = \frac{r_1 + r_2}{c} + \delta_{\text{jitter}}, \quad (8)$$

where δ_{jitter} accounts for the finite timing resolution of the detector electronics.

Chapter 2. Paper 2. The Planck Depth and Optimal Trigger Design: Composition-Inflation

Applied to Real-Time Event Selection at the Large Hadron Collider

Proof. A particle produced at the interaction point at $t = 0$ reaches the outer detector surface at $t = r_2/c$ (neglecting the particle's travel time, which equals r_2/c for relativistic particles moving at $\approx c$). A signal propagating back through electronics to the trigger reaches the L1 processor at $t = (r_2 + r_{\text{cable}})/c$ where r_{cable} is the equivalent cable length. For the ATLAS detector with inner tracking radius $r_1 = 0.05$ m, outer muon radius $r_2 = 11$ m, and cable runs of approximately 40 m equivalent path length: $\tau_c = (r_1 + r_2 + r_{\text{cable}})/c = 51.05 \text{ m}/(3 \times 10^8 \text{ m/s}) \approx 170 \text{ ns}$. However, the L1 trigger uses only first-level primitives that travel over approximately $r_2 = 11$ m in optical fibre: $\tau_c = 11 \text{ m}/(2 \times 10^8 \text{ m/s}) \approx 55 \text{ ns}$ (fibre propagation at $\approx 2c/3$). The fundamental 25 ns window is set by the requirement that $\tau_{\text{BX}} = 1/\nu_B = c^{-1} \times 2 \times r_{\text{eff}}$ where $r_{\text{eff}} = c/(2\nu_B) = 3 \times 10^8/(2 \times 4 \times 10^7) = 3.75 \text{ m}$. This is the effective detector half-radius over which partition information must propagate within one bunch crossing period. $\square$

Corollary 4.2 (The 25 ns Window is Fundamental). *The LHC bunch-crossing period $\tau_B = 25 \text{ ns}$ equals the partition propagation time for the LHC interaction region geometry. It is not arbitrary: any shorter window would be insufficient to collect signals from the entire interaction volume; any longer window would cause signals from adjacent bunch crossings to overlap (pileup).*

Proof. From Theorem 4.1, $\tau_B = c^{-1} \times 2r_{\text{eff}}$ with $r_{\text{eff}} = 3.75 \text{ m}$. Changing τ_B is equivalent to changing the effective detector radius. The specific value $\tau_B = 25 \text{ ns}$ is the unique solution satisfying: (i) sufficient time for signals from $r \leq r_{\text{eff}}$ to reach the trigger; (ii) shorter than the minimum time for in-time pileup overlap at the LHC design luminosity. These constraints are set by physics (particle velocities and interaction rates), not by electronics design. $\square$

4.2 The partition uncertainty relation

Theorem 4.3 (Partition Uncertainty Relation). *For a bounded oscillatory system with bunch-crossing period τ_B , the minimum detectable partition depth change $\Delta\mathcal{M}$ per crossing satisfies:*

$$\Delta\mathcal{M} \cdot \tau_B \geq \hbar. \quad (9)$$

Proof. The partition count $\mathcal{M}(t)$ advances by $d\mathcal{M}/dt = 1/\langle\tau_p\rangle$ where $\tau_p = \hbar/\Delta E$ is the partition lag (Sachikonye, c). A detectable partition depth change $\Delta\mathcal{M}$ requires distinguishing two adjacent partition states, which requires energy uncertainty $\Delta E \geq \hbar/\tau_B$ (time-energy Heisenberg bound applied to the measurement window τ_B). Hence $\Delta\mathcal{M} \cdot \tau_B \geq \Delta\mathcal{M} \cdot (\hbar/\Delta E) = \hbar \cdot \Delta\mathcal{M}/\Delta E$. Since $\Delta\mathcal{M} \geq \Delta E/\Delta E = 1$ state per detection event, the bound $\Delta\mathcal{M} \cdot \tau_B \geq \hbar$ holds. $\square$

Corollary 4.4 (Minimum Energy Resolution per Bunch Crossing). *For $\tau_B = 25 \text{ ns}$, the minimum detectable*

$$\Delta E_{\text{min}} = \frac{\hbar}{\tau_B} = \frac{1.055 \times 10^{-34} \text{ J} \cdot \text{s}}{25 \times 10^{-9} \text{ s}} = 4.22 \times 10^{-27} \text{ J} \approx 26 \text{ eV}. \quad (10)$$

This is the fundamental resolution floor of the LHC trigger: no trigger condition based on energy measurement can resolve energy differences below 26 eV per bunch crossing, regardless of detector technology.

Remark 4.5 (Context of the energy floor). The 26 eV floor is far below the practical trigger thresholds used in ATLAS and CMS (typically $\geq 1 \text{ GeV}$ for electron/photon triggers, $\geq 20 \text{ GeV}$ for jet triggers). The gap between the fundamental floor and practical thresholds is ten orders of magnitude, so the partition uncertainty is not a practical limitation at current luminosities. However, it becomes relevant when considering highly optimized low-threshold triggers for future experiments.

5 Detector Subsystems as Commuting Observables

5.1 The partition coordinate assignment

The partition coordinates (n, ℓ, m, s) derived in Sachikonye (d) are not merely abstract labels: they correspond to measurable physical quantities in the LHC detector.

Definition 5.1 (LHC Detector Partition Coordinates). The partition coordinates of a detected particle at the LHC are:

- (i) *Principal number n* : measured by the inner tracker as the momentum-to-energy scale of the track. n determines which energy shell the particle occupies: $n = \lfloor \sqrt{p_T/p_{\text{ref}}} \rfloor + 1$ where p_T is the transverse momentum and $p_{\text{ref}} = 1 \text{ GeV}/c$.
- (ii) *Angular complexity ℓ* : measured by the EM calorimeter as the shower shape. Electromagnetic showers from electrons/photons have $\ell = 0$ (spherically symmetric); hadronic showers have $\ell \geq 1$ (angular structure in the shower profile).
- (iii) *Orientation m* : measured by the hadronic calorimeter as the energy partition between EM and hadronic components. $m = 0$ for purely EM; $m = \pm 1$ for mixed; $m = \pm \ell$ for hadronic.
- (iv) *Chirality s* : measured by the muon spectrometer as the charge sign. $s = +\frac{1}{2}$ for positive muons; $s = -\frac{1}{2}$ for negative.

Remark 5.2 (Parity and HL-LHC). The fifth coordinate (parity, associated with the timing detector in HL-LHC) encodes the phase of particle arrival relative to the bunch clock, measured by the High-Granularity Timing Detector (HGTD) at 20–30 ps precision. This provides the fifth dimension in the HL-LHC upgrade ($d = 4 \rightarrow d = 5$ eventually).

5.2 The commutation theorem

The crucial property distinguishing partition coordinates from standard quantum observables (position, momentum) is that all four commute.

Theorem 5.3 (Complete Commutation of Detector Observables). *The four detector observables $\hat{O}_n$ (tracker), $\hat{O}_\ell$ (ECAL), $\hat{O}_m$ (HCAL), $\hat{O}_s$ (muon) satisfy:*

$$[\hat{O}_n, \hat{O}_\ell] = [\hat{O}_\ell, \hat{O}_m] = [\hat{O}_m, \hat{O}_s] = [\hat{O}_n, \hat{O}_m] = [\hat{O}_n, \hat{O}_s] = [\hat{O}_\ell, \hat{O}_s] = 0. \quad (11)$$

They constitute a Complete Set of Commuting Observables (CSCO) on the partition coordinate space.

Proof. Each partition coordinate is a function of a conserved quantity: $\hat{O}_n = f(\hat{H})$ (energy), $\hat{O}_\ell = g(\hat{L}^2)$ (angular momentum magnitude), $\hat{O}_m = h(\hat{L}_z)$ (angular momentum projection), $\hat{O}_s = k(\hat{S}_z)$ (charge sign, an internal degree of freedom). The commutation follows from:

- $[f(\hat{H}), g(\hat{L}^2)] = 0$ since $[\hat{H}, \hat{L}^2] = 0$ for rotationally symmetric interactions.
- $[g(\hat{L}^2), h(\hat{L}_z)] = 0$ since $[\hat{L}^2, \hat{L}_z] = 0$ (standard angular momentum algebra).
- $[\hat{H}, \hat{L}_z] = 0$ for z -axis-aligned measurement.
- All four commute with $\hat{S}_z$ since charge is an internal degree of freedom independent of kinematic coordinates.

Commuting observables share a common eigenbasis, so they can be simultaneously diagonalized and simultaneously measured. $\square$ $\square$

5.3 The QND trigger theorem

Theorem 5.4 (Quantum Non-Demolition Trigger Classification). *The trigger’s classification of each detected particle into partition coordinates (n, ℓ, m, s) is quantum non-demolition (QND): repeated measurements of any partition coordinate yield the same result, and measuring any one coordinate does not disturb the others.*

Proof. QND measurement requires $[\hat{O}, \hat{H}] = 0$, which holds for all four coordinates by Theorem 5.3 (each is a function of a conserved quantity). By the QND criterion, the post-measurement state is an eigenstate of the measured observable, which is also an eigenstate of the Hamiltonian, so it evolves unchanged. Repeated measurements yield the same eigenvalue. Since all four observables commute, measuring any one does not collapse the eigenstate of the others. $\square$

Corollary 5.5 (Simultaneous Multi-Subsystem Classification). *The trigger can determine all four partition coordinates simultaneously from one particle’s interaction with the detector. The tracker, ECAL, HCAL, and muon system can all fire within the same 25 ns window and their measurements are internally consistent — no quantum backaction between subsystems.*

Remark 5.6 (Why this matters for trigger design). The QND property is not merely formal. It means: (i) There is no fundamental obstacle to combining tracker,

calorimeter, and muon information in the L1 decision — the measurements commute. (ii) The trigger decision is not disturbed by the measurement process itself. (iii) Subsequent measurements (at HLT) will agree with the L1 classification to within the resolution of each detector element. This is the partition-theoretic justification for the ATLAS “combined performance” trigger approach, which uses all subsystems simultaneously in the L1 decision.

Corollary 5.7 (Measurement Backaction Bound). *The fractional momentum disturbance from trigger classification is bounded by:*

$$\frac{\Delta p}{p} \leq \frac{\hbar}{L \cdot p} = \frac{\lambda_{\text{dB}}}{2\pi L} \quad (12)$$

where $\lambda_{\text{dB}} = h/p$ is the de Broglie wavelength and L is the characteristic detector cell size. For typical detector cell sizes $L \sim 5$ cm and pion momenta $p \sim 10$ GeV/c:

$$\frac{\Delta p}{p} \approx \frac{10^{-34}}{5 \times 10^{-2} \times 5.3 \times 10^{-18}} \approx 4 \times 10^{-16},$$

which is 14 orders of magnitude below the detector’s intrinsic momentum resolution of $\sim 10^{-2}$. The quantum backaction is negligible at any foreseeable LHC energy scale.

6 Single-Particle Thermodynamics of Detected Events

Each detected particle traversing the LHC detector is, from the partition framework’s perspective, a single bounded oscillatory system. Classical thermodynamics requires $N \rightarrow \infty$ for temperature and pressure to be defined. For single particles ($N = 1$), a different formulation is required.

6.1 Categorical temperature

Definition 6.1 (Categorical Temperature). *The categorical temperature of a particle with oscillation (cyclotron) frequency ω_c in the detector is:*

$$T_{\text{cat}} = \frac{\hbar\omega_c}{2\pi k_B}. \quad (13)$$

This is the information temperature: the rate at which the particle explores categorical states per unit time.

Theorem 6.2 (Single-Particle Ideal Gas Law). *A single detected particle with $M = C(n) = 2n^2$ accessible categorical states in effective interaction volume V at categorical temperature T_{cat} satisfies:*

$$PV = k_B T_{\text{cat}}, \quad N = 1. \quad (14)$$

Proof. For a single particle with M accessible categorical states, the canonical partition function is $Z = M$ (high-temperature limit, $k_B T_{\text{cat}} \gg \Delta E$). The Helmholtz free energy is $F = -k_B T_{\text{cat}} \ln M$. The pressure from volume dependence: $P = -\partial F / \partial V|_{T_{\text{cat}}} = k_B T_{\text{cat}} \partial \ln M / \partial V$. Since $M \propto V$ (more volume $\rightarrow$ more accessible partition cells), $\partial \ln M / \partial V = 1/V$, giving $P = k_B T_{\text{cat}} / V$, hence $PV = k_B T_{\text{cat}}$. $\square$

Chapter 2. Paper 2. The Planck Depth and Optimal Trigger Design: Composition-Inflation

6.2 Applied to Real-Time Event Selection at the Large Hadron Collider

For the trigger's reference oscillator at $\omega_{\text{BX}} = 2\pi\nu_B$:

$$T_{\text{cat,trigger}} = \frac{\hbar \cdot 2\pi\nu_B}{2\pi k_B} = \frac{\hbar\nu_B}{k_B} = \frac{1.055 \times 10^{-34} \times 4.0 \times 10^7}{1.381 \times 10^{-23}} \approx 3.1 \times 10^{-10} \text{ K} \quad (15)$$

This ~ 0.3 nK categorical temperature is the information temperature of the LHC trigger system — not a physical kinetic temperature but the rate at which the trigger explores categorical states per bunch crossing.

For an individual pion with $p_T = 1$ GeV/ c in $B = 2$ T: $\omega_c = qB/m = (1.6 \times 10^{-19} \times 2)/(140 \times 10^6/c^2) \approx 6.9 \times 10^8$ rad/s, $T_{\text{cat,pion}} = \hbar\omega_c/(2\pi k_B) \approx 5.3$ μ K.

Remark 6.3 (Consistency check). From Equation (9), $\hbar/(k_B T_{\text{cat}}) = 2\pi/\omega_c = \tau_c$ — the partition propagation time. This is a self-consistency check: the categorical temperature, defined through the ideal gas law, reproduces the fundamental timing constraint $\hbar/(k_B T_{\text{cat}}) = \tau_{\text{BX}} = 25$ ns for the bunch clock. The thermodynamics and the timing analysis are consistent.

7 Trigger Design Principles from Partition Theory

7.1 The temporal structure of the trigger window

A trigger that makes decisions based on $\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k)$ — the deviation of the k -th detector signal from the bunch clock — is operating in the natural language of the partition framework.

Definition 7.1 (Timing Deviation). For the LHC bunch clock providing reference times $T_{\text{ref}}(k)$ (the k -th bunch crossing time), and a detector signal arriving at $t_{\text{rec}}(k)$, the *timing deviation* is:

$$\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k). \quad (16)$$

Definition 7.2 (Timing Cell). A *timing cell* C_i is a Borel-measurable subset of the ΔP -space $\mathbb{R}$, with associated action a_i . The collection $\{C_i\}$ forms the *trigger cell registry* Γ .

A trigger condition is a cell membership predicate: the trigger fires when $\Delta P(k) \in C_i$. For a prompt signal from a collision at the interaction point, $|\Delta P(k)| < \tau_{\text{BX}}/2 = 12.5$ ns; signals outside this window are backgrounds.

Theorem 7.3 (Structural Incorruptibility of Timing Triggers). *A trigger that conditions entirely on $\Delta P(k)$ values — with no content parser applied to signal amplitude, charge, or identity — has no injection surface: any signal whose timing does not originate from the LHC bunch crossing cannot satisfy any physically calibrated cell condition.*

Proof. The cell registry partitions ΔP -space. A real collision signal satisfies $|\Delta P(k)| < \delta$ where δ is the detector timing jitter (~ 1 ns for calibrated detectors). A background signal (cosmic ray, beam halo, noise)

arrives with a $\Delta P(k)$ drawn from a distribution with different mean and variance. Since the cell conditions are defined by ΔP membership alone, there is no content layer on which a forger can operate: the only way to satisfy a timing cell condition is to produce a signal with the correct timing relative to T_{ref} , which requires physical presence at the interaction point during the bunch crossing. An external attacker who wants to inject a fake event must produce a signal synchronized to the LHC bunch clock with sub-nanosecond precision. This requires physical access to the reference clock. $\square$

Remark 7.4 (Security is topological, not cryptographic). Theorem 7.3 establishes that timing-only triggers are *structurally* incorruptible — their security derives from the physics of timing constraints, not from cryptographic algorithms. This is the particle physics analogue of the structural incorruptibility of temporal programming (Sachikonye, a): no parser means no injection surface.

7.2 Optimal cell structure

Proposition 7.5 (Optimal Timing Window). *For a trigger system at the LHC Planck depth $n_P = 60$, the optimal cell half-width is:*

$$\delta_{\text{opt}} = \frac{\tau_{\text{BX}}}{2 \cdot T(n_P, d)^{1/d}} = \frac{12.5 \text{ ns}}{2 \cdot (9.23 \times 10^{35})^{1/3}} \approx 6.5 \times 10^{-13} \text{ ns} = 0.65 \text{ fs} \quad (17)$$

Proof. At the Planck depth, the angular resolution is $\Delta\theta = 2\pi/T(n_P, d)$. This corresponds to a temporal resolution $\Delta t = \tau_{\text{BX}} \cdot \Delta\theta/(2\pi) = \tau_{\text{BX}}/T(n_P, d)$, and a spatial cell half-width $\delta_{\text{opt}} = \tau_{\text{BX}}/(2T(n_P, d)^{1/d})$ after distributing the resolution across d dimensions. $\square$

Remark 7.6. The 0.65 fs optimal cell width is far below the current HGTD timing precision of 20–30 ps. This means the HL-LHC timing detector operates at a fraction ($\sim 3 \times 10^{-2}$) of the fundamental Planck-depth resolution. There remains an enormous categorical resolution reserve available for future detector improvements.

8 The HL-LHC Upgrade

8.1 Adding a fourth dimension

The High-Luminosity LHC (HL-LHC) upgrade includes the ATLAS High-Granularity Timing Detector (HGTD) and the CMS Endcap Timing Layer (ETL), both providing timing precision of 20–30 ps per track (ATLAS Collaboration, 2020a; CMS Collaboration, 2020). These constitute a fourth independent detector subsystem: $d: 3 \rightarrow 4$.

Theorem 8.1 (HL-LHC Dimension Advantage). *Adding the precision timing detector ($d: 3 \rightarrow 4$) reduces the Planck depth from 60 to 52, recovering eight bunch crossings (200 ns) of categorical resolution headroom.*

Proof. For $d = 4$:

$$\begin{aligned} n_P(d=4) &= 1 + \left\lceil \log_5 \left(\frac{4.628 \times 10^{35}}{4} \right) \right\rceil = 1 + \left\lceil \log_5 (1.157 \times 10^5) \right\rceil \\ &= 1 + \left\lceil \frac{\ln(1.157 \times 10^5)}{\ln 5} \right\rceil = 1 + \left\lceil \frac{80.52}{1.609} \right\rceil = 1 + \left\lceil 50.04 \right\rceil = 52 \end{aligned} \quad (18)$$

Saving: $n_P(d=3) - n_P(d=4) = 60 - 52 = 8$ crossings
 $= 8/\nu_B = 200$ ns. $\square$

Remark 8.2 (Engineering interpretation). Each saved bunch crossing represents 25 ns of additional processing time available for the trigger. Eight crossings = 200 ns is significant: it allows more complex HLT algorithms to be applied before the L1 latency budget is exhausted. The dimension advantage is entirely a consequence of the composition-inflation base changing from $d+1=4$ to $d+1=5$, increasing the per-crossing state count factor.

Corollary 8.3 (General Dimension Advantage). *For the LHC bunch clock, the Planck depth savings from each additional independent subsystem are:*

d	$(d+1)$	n_P	$n_P(d) - n_P(d+1)$
3	4	60	8
4	5	52	7
5	6	45	6
6	7	39	5

Diminishing returns are evident: the savings per added dimension decrease as d increases, because the logarithm in Equation (4) becomes less sensitive to the base.

8.2 State count at HL-LHC

Example 8.4 (HL-LHC L1 trigger at $d = 4$). For HL-LHC with $d = 4$ and the same $n = 100$ crossing latency:

$$T(100, 4) = 4 \cdot 5^{99} \approx 6.3 \times 10^{69}.$$

This is $6.3 \times 10^{69} / 4.628 \times 10^{35} \approx 1.4 \times 10^{34}$ times the Planck threshold, compared to 6.5×10^{23} at $d = 3$. The state space grows by 10^{10} by adding one detector dimension.

9 Trigger Performance Metrics

We introduce two dimensionless performance metrics that characterise how efficiently the trigger exploits the available categorical resolution.

9.1 Composition efficiency

Definition 9.1 (Composition Efficiency). The *composition efficiency* of a trigger menu is:

$$\eta_C = \frac{\log_{d+1} N_{\text{menu}}}{n_P}, \quad (19)$$

where N_{menu} is the number of distinct trigger paths in the menu and n_P is the Planck depth. $\eta_C \in [0, 1]$ measures what fraction of the available categorical resolution the menu actually exploits.

Example 9.2 (Current ATLAS trigger). $N_{\text{menu}} \approx 10^3$, $n_P = 60$: $\eta_C = \log_4(10^3)/60 = (3/\log_{10} 4)/60 = 4.98/60 \approx 0.083$. The current trigger menu exploits approximately 8% of the available categorical resolution.

Remark 9.3. The low η_C is not a design failure. The trigger menu contains $\sim 10^3$ paths because that is sufficient to capture all physics processes of interest at current luminosities. The remaining 92% of the Planck depth capacity is a resource reserve for future use — new physics searches requiring finer event topology discrimination.

9.2 Planck Utilisation Index

Definition 9.4 (Planck Utilisation Index). The *Planck Utilisation Index* is:

$$\text{PUI} = \frac{n_{\text{operating}}}{n_P}, \quad (20)$$

where $n_{\text{operating}}$ is the number of bunch crossings in the trigger's integration window.

Example 9.5 (ATLAS L1 trigger). $n_{\text{operating}} = 100$, $n_P = 60$: $\text{PUI} = 100/60 = 1.67$. The trigger operates at 1.67 times the Planck depth: it has accumulated more categorical resolution than the Planck threshold requires.

Proposition 9.6 (Optimal Operating Point). *The optimal trigger design satisfies $\text{PUI} = 1$ (operating exactly at the Planck depth) and η_C as large as the physics requirements allow.*

Proof. $\text{PUI} < 1$: the trigger has not yet accumulated Planck-level categorical resolution; more crossings would improve discrimination. $\text{PUI} > 1$: the trigger has exceeded the Planck threshold; additional crossings provide diminishing returns in categorical resolution (the base $(d+1)^n$ growth continues, but the Planck comparison has already been satisfied). Operating at $\text{PUI} = 1$ minimises the latency budget while maximising the categorical resolution per crossing. $\square$

Remark 9.7 (Why the current design uses $\text{PUI} = 1.67$). The current ATLAS L1 latency of 100 crossings ($\text{PUI} = 1.67$) exceeds the optimal $\text{PUI} = 1$ by 40%. This is not waste: the extra 40 crossings are used to provide a timing safety margin for signal propagation from the outermost detector elements, for pipeline stability, and for the L1 accept/reject decision logic. The partition framework predicts that the minimum physically motivated latency is $n_P = 60$ crossings; the engineering reality adds $\sim 40\%$ overhead for these practical considerations.

Chapter 2. Paper 2. The Planck Depth and Optimal Trigger Design: Composition-Inflation Applied to Real-Time Event Selection at the Large Hadron Collider

Table 2: Trigger performance metrics for current and future LHC detectors.

Detector	d	n_{op}	n_P	PUI	η_C
ATLAS Run 3	3	100	60	1.67	~ 0.08
ATLAS HL-LHC	4	100	52	1.92	~ 0.10
FCC-hh (design)	4	200	52	3.85	~ 0.15

9.3 Summary of metrics for current and future detectors

10 Discussion and Outlook

10.1 What has been established

We have derived, from the composition-inflation formula $T(n, d) = d(d+1)^{n-1}$ and the Planck depth definition, a complete set of first-principles results for LHC trigger design:

- (1) The trigger state space is $T(n, d)$, not 2^n (exponential with base $d+1$, not 2) — Theorem 2.2.
- (2) The Planck depth $n_P = 60$ is the natural design depth for the LHC at $d = 3$ — Theorem 3.3.
- (3) The 25 ns window is the partition propagation time, not an engineering choice — Theorem 4.1.
- (4) The 26 eV minimum energy resolution is fundamental, not instrumental — Corollary 4.4.
- (5) All four detector subsystems measure commuting observables and can be combined simultaneously — Theorem 5.3.
- (6) The trigger classification is QND: it does not disturb the state it classifies — Theorem 5.4.
- (7) Each detected particle satisfies $PV = k_B T_{\text{cat}}$ for $N = 1$ — Theorem 6.2.
- (8) Adding the HL-LHC timing detector saves 8 crossings of latency budget — Theorem 8.1.
- (9) The current trigger uses $\sim 8\%$ of the available categorical resolution — Definition 9.1.

10.2 Open questions

Optimal trigger menu construction. Given $n_P = 60$ and $\eta_C \approx 8\%$, how should the trigger menu be extended to capture more of the available 10^{59} pattern space? The composition-inflation framework suggests that the most efficient extension uses temporal structure (varying the composition of crossings) rather than spatial thresholds (varying energy cuts within a fixed composition depth).

Trigger as molecular spectrometer. The connection between LHC triggers and mass spectrometers established in Sachikonye (c) suggests that spectroscopic techniques (harmonic resonator loops, spectral holography) may be applicable to trigger design. Specifically, the harmonic-scattering-loop framework (Sachikonye, b) provides a transfer matrix formulation in which the detector subsystem coupling graph determines the rank of the classification matrix, analogous to how the molecular harmonic graph determines the imaging capacity.

Non-integrable regime. The composition-inflation formula was derived assuming all d detector dimensions

are independent. In practice, the tracker-ECAL coupling (track-cluster matching), ECAL-HCAL coupling (hadronic leakage), and other correlations reduce the effective d . A rigorous analysis of how these couplings reduce the effective dimension and thus the Planck depth is needed.

Background rejection from partition structure. The selection rules $\Delta\ell = \pm 1$, $\Delta m \in \{0, \pm 1\}$ derived in Sachikonye (d) impose structural constraints on which event topologies are physically realizable. Trigger backgrounds that require selection-rule-violating intermediate states are not merely rare — they are categorically excluded. This provides a new framework for background rejection that does not depend on Monte Carlo simulation.

10.3 Relation to conventional trigger design

The present framework is not in tension with conventional trigger design methodology; it provides a theoretical underpinning for choices that are currently made empirically. The 25 ns window was chosen because it is experimentally found to minimise pileup contamination; the present framework derives it from first principles. The $\sim 10^3$ trigger path menu was chosen because it covers known physics processes; the present framework quantifies how much more it could, in principle, discriminate.

The Planck depth $n_P = 60$ provides a natural design benchmark: a trigger operating at $\text{PUI} \geq 1$ has accumulated sufficient categorical resolution to distinguish events at the Planck level of temporal precision. Operating significantly above n_P (as the current $\text{PUI} = 1.67$ does) provides safety margin but not additional categorical resolution, since the Planck threshold has already been crossed.

Acknowledgements

This work was supported by the AIME Registry for Artificial Intelligence. The LHC parameters are taken from publicly available CERN documentation.

References

- ATLAS Collaboration. The ATLAS experiment at the CERN large hadron collider. *JINST*, 3:S08003, 2008.
- ATLAS Collaboration. Technical design report: A high-granularity timing detector for the ATLAS phase-II upgrade. Technical Report CERN-LHCC-2020-007, CERN, 2020a.
- ATLAS Collaboration. Operation of the ATLAS trigger system in Run 2. *JINST*, 15:P10004, 2020b.
- CMS Collaboration. The CMS experiment at the CERN LHC. *JINST*, 3:S08004, 2008.

CMS Collaboration. The phase-2 upgrade of the CMS level-1 trigger. Technical Report CERN-LHCC-2020-004, CERN, 2020.

Lyndon Evans and Philip Bryant. LHC machine. *JINST*, 3:S08001, 2008.

Kundai Farai Sachikonye. Fundamental timing and structural incorruptibility in real-time event selection. Companion paper, this series, a.

Kundai Farai Sachikonye. Harmonic-scattering loop coupling: Multi-source resolution via path-multiplexed rays through polyatomic molecular resonators. Companion paper, this series, b.

Kundai Farai Sachikonye. Partition lagrangian and the ion as partition malformation. Companion paper, this series, c.

Kundai Farai Sachikonye. On the partition algebra of bounded phase space: Composition-inflation, angular resolution, and the categorical reformulation of fundamental constants. Companion paper, this series, d.

Part II

The LHC as a Partition Spectrometer

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1, Paper 3. The LHC as a High-Energy Partition Spectrometer: Lagrangian Derivation of All Four Analyzer Equations, Ions as Partition Malformations, and Three Routes to Mass, paper:spectrometer

Multi-Subsystem Quantum Non-Demolition Classification: Complete Commutation, Single-Particle Thermodynamics, and Simultaneous Detection at the LHC

Kundai Farai Sachikonye
 Technical University of Munich
 Chair of Proteomics and Bioanalytics
 kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

We prove that the four primary LHC detector subsystems — inner tracker, electromagnetic calorimeter, hadronic calorimeter, and muon spectrometer — measure four mutually commuting observables on the partition coordinate space (n, ℓ, m, s) . The complete commutation relation $[\hat{O}_n, \hat{O}_\ell] = [\hat{O}_\ell, \hat{O}_m] = [\hat{O}_m, \hat{O}_s] = \dots = 0$ (Theorem 2.3) establishes these as a Complete Set of Commuting Observables (CSCO), and the Quantum Non-Demolition (QND) property (Theorem 3.2) shows that measuring any one coordinate does not disturb the others. The fractional momentum backaction is bounded by $\Delta p/p \leq \lambda_{\text{dB}}/(2\pi L) \approx 4 \times 10^{-16}$ for typical LHC conditions (Theorem 3.4) — fourteen orders of magnitude below the detector’s intrinsic momentum resolution.

We resolve a foundational paradox of single-particle thermodynamics: classical thermodynamics requires $N \rightarrow \infty$ for temperature and pressure to be defined, yet every detected LHC event involves a finite, often small, number of particles. The resolution is the *categorical temperature* $T_{\text{cat}} = \hbar\omega_c/(2\pi k_B)$ (Definition 4.1), which enables the *single-particle ideal gas law* $PV = k_B T_{\text{cat}}$ for $N = 1$ (Theorem 4.4). For the LHC trigger reference oscillator at 40 MHz, $T_{\text{cat}} \approx 0.31$ nK; for a pion with $p_T = 1$ GeV/c in $B = 2$ T, $T_{\text{cat}} \approx 5.3$ μ K.

We prove the *Fundamental Identity* $dM/dt = \omega/(2\pi) = 1/\langle\tau_p\rangle$ (Theorem 5.1) unifying temporal evolution, oscillatory dynamics, and categorical state counting as three descriptions of one process. We prove *heat-entropy decoupling* $\text{Cov}(\delta Q, dS_{\text{cat}}) = 0$ (Theorem 5.4): detector thermal noise and physics signal are statistically independent because they occupy orthogonal degrees of freedom. This is the formal reason timing-only triggers can discriminate signal from background using $\Delta P(k)$ alone.

We introduce the *Kuramoto order parameter* for the LHC detector $R_c = \exp(-2\pi^2 \cdot \text{CV}^2)$ where $\text{CV} = \text{RMSSD}/\bar{T}_{BX}$, and prove that the five operational detector regimes (turbulent $R_c < 0.3$, aperture 0.3–0.5, cascade 0.5–0.8, coherent 0.8–0.95, phase-locked $R_c > 0.95$) correspond to the five thermodynamic regimes of de-

tected ions (Theorem 6.3), establishing a precise mapping from detector synchronisation quality to event classification quality.

Keywords: quantum non-demolition; complete commuting observables; single-particle thermodynamics; categorical temperature; fundamental identity; heat-entropy decoupling; Kuramoto order parameter; detector regimes; LHC trigger; partition coordinates

Contents

1	Introduction	2
1.1	The multi-subsystem problem	2
1.2	Overview of results	2
2	Partition Coordinates as Quantum Observables	2
2.1	Coordinate definitions and detector realizations	2
2.2	Complete commutation	2
3	Quantum Non-Demolition Measurement	3
3.1	The QND criterion	3
3.2	Measurement backaction bound	3
3.3	Resolution-time tradeoff	4
4	Single-Particle Thermodynamics	4
4.1	The single-particle paradox and its resolution	4
4.2	Categorical temperature	4
4.3	Single-particle ideal gas law	4
4.4	Categorical temperatures at the LHC	5
5	The Fundamental Identity and Its Consequences	5
5.1	Time, oscillation, and counting as one process	5
5.2	Heat-entropy decoupling	5
5.3	Counting irreversibility	6

6 Kuramoto Order Parameter for Detector Synchronisation	6
6.1 Phase coherence in the detector	6
6.2 Five operational regimes	6
7 The LHC Detector as a Partition Spectrometer	7
7.1 Analogy with mass spectrometry	7
7.2 Five thermodynamic regimes at the LHC	7
8 Discussion	7
8.1 What has been established	7
8.2 The four-subsystem trigger window . . .	8
8.3 Open questions	8

1 Introduction

1.1 The multi-subsystem problem

The ATLAS detector at the LHC comprises four primary detection subsystems, each exploiting a different physical mechanism: the inner tracker measures charged-particle momenta via curvature in a 2 T solenoid field; the electromagnetic calorimeter measures electron and photon energies via shower development in liquid argon; the hadronic calorimeter measures hadron energies via shower development in steel/scintillator; and the muon spectrometer identifies muons via track reconstruction in air-core toroids (ATLAS Collaboration, 2008).

These four subsystems produce four independent measurements of each traversing particle. The trigger must combine all four simultaneously in the L1 decision — within a fixed latency of 100 bunch crossings.

The question this paper addresses: *is simultaneous combination of all four subsystems fundamentally consistent with quantum mechanics?* The answer, which is not obvious a priori, is yes — but only because the four measured observables belong to the partition coordinate system (n, ℓ, m, s) of bounded phase space, which has the special property of complete mutual commutation. For conventional quantum observables (position, momentum), simultaneous measurement is forbidden by the Heisenberg uncertainty principle. For partition coordinates, simultaneous measurement is guaranteed by the CSCO structure.

1.2 Overview of results

The paper derives and proves:

- (1) Complete commutation of all four detector observables, enabling simultaneous measurement without ordering constraints (Theorem 2.3).
- (2) QND property: measuring any partition coordinate preserves all others (Theorem 3.2).
- (3) Backaction bound: $\Delta p/p \approx 4 \times 10^{-16}$, negligible at any LHC energy (Theorem 3.4).
- (4) Resolution-time tradeoff: $R = m/\Delta m = \omega\tau/(2\pi)$ — resolution scales with integration time (Theorem 3.7).

- (5) Single-particle ideal gas: $PV = k_B T_{\text{cat}}$ for $N = 1$ (Theorem 4.4).
- (6) Categorical temperature values for LHC particles (Section 4.4).
- (7) Fundamental identity $dM/dt = 1/\langle\tau_p\rangle$ (Theorem 5.1).
- (8) Heat-entropy decoupling: thermal noise and physics signal are statistically independent (Theorem 5.4).
- (9) Counting irreversibility: trigger decisions cannot be undone (Theorem 5.7).
- (10) Five detector operational regimes from the Kuramoto order parameter (Theorem 6.3).

2 Partition Coordinates as Quantum Observables

2.1 Coordinate definitions and detector realizations

From the partition algebra of bounded phase space (Sachikonye, b), every particle in a bounded three-dimensional oscillatory system carries partition coordinates (n, ℓ, m, s) :

- $n \geq 1$: principal level (radial closure number).
- $0 \leq \ell \leq n - 1$: angular sublevel (polar closure).
- $-\ell \leq m \leq +\ell$: azimuthal projection.
- $s \in \{-\frac{1}{2}, +\frac{1}{2}\}$: rotational parity.

At the LHC, each of these is measured by a dedicated detector subsystem:

Definition 2.1 (LHC Partition Coordinate Operators). The four partition coordinate operators for a particle traversing the ATLAS detector are:

- $\hat{O}_n = f(\hat{H})$: principal number, measured by the inner tracker as the momentum scale $n = \lfloor \sqrt{p_T/p_{\text{ref}}} \rfloor + 1$ where $p_{\text{ref}} = 1 \text{ GeV}/c$.
- $\hat{O}_\ell = g(\hat{L}^2)$: angular complexity, measured by the EM calorimeter as the shower shape parameter — $\ell = 0$ for electromagnetic (narrow, symmetric), $\ell \geq 1$ for hadronic (wide, structured).
- $\hat{O}_m = h(\hat{L}_z)$: orientation, measured by the hadronic calorimeter as the energy partition between EM and hadronic components, encoding the azimuthal angular momentum projection.
- $\hat{O}_s = k(\hat{S}_z)$: chirality, measured by the muon spectrometer as the track curvature sign — $s = +\frac{1}{2}$ for positive charge, $s = -\frac{1}{2}$ for negative.

Remark 2.2 (HL-LHC fifth coordinate). The HL-LHC precision timing detector (HGTD, CMS ETL) will measure a fifth coordinate — the phase of particle arrival relative to the bunch clock at 20–30 ps precision. This provides an independent handle on the parity/temporal coordinate, extending the system to $d = 4$ independent subsystems and reducing the Planck depth from $n_P = 60$ to 52 (Sachikonye, c).

2.2 Complete commutation

Theorem 2.3 (Complete Commutation of Detector Observables). *The four detector observables of Defini-*

Chapter 3. Paper 4. Multi-Subsystem Quantum Non-Demolition Classification: Complete

Commutation, Single-Particle Thermodynamics, and Simultaneous Detection at the LHC

tion 2.1 satisfy complete mutual commutation:

$$[\hat{O}_n, \hat{O}_\ell] = [\hat{O}_\ell, \hat{O}_m] = [\hat{O}_m, \hat{O}_s] = [\hat{O}_n, \hat{O}_m] = [\hat{O}_n, \hat{O}_s] = [\hat{O}_\ell, \hat{O}_s] = 0 \quad (1)$$

They form a Complete Set of Commuting Observables (CSCO) on the partition coordinate space.

Proof. Each operator is a function of a conserved quantity of the particle dynamics in the detector:

- $\hat{O}_n = f(\hat{H})$ (energy): $[\hat{H}, \hat{H}] = 0$, so $[\hat{O}_n, \hat{H}] = 0$. Since $\hat{O}_n$ depends only on $\hat{H}$, it commutes with all other conserved quantities.
- $\hat{O}_\ell = g(\hat{L}^2)$: for the approximately spherically symmetric EM shower, $[\hat{L}^2, \hat{H}] = 0$ and $[\hat{L}^2, \hat{L}_z] = 0$ (standard angular momentum algebra).
- $\hat{O}_m = h(\hat{L}_z)$: for z -axis-aligned tracking, $[\hat{L}_z, \hat{H}] = 0$ and $[\hat{L}_z, \hat{L}^2] = 0$.
- $\hat{O}_s = k(\hat{S}_z)$: electric charge is an internal quantum number, $[\hat{S}_z, \hat{H}] = [\hat{S}_z, \hat{L}^2] = [\hat{S}_z, \hat{L}_z] = 0$.

All four are functions of the mutually commuting set $\{\hat{H}, \hat{L}^2, \hat{L}_z, \hat{S}_z\}$, so any two functions of elements from this set commute.

The CSCO property follows: the four operators simultaneously diagonalize on a common eigenbasis (the partition states), and their eigenvalues (n, ℓ, m, s) uniquely specify each state. $\square$

Corollary 2.4 (No Uncertainty Principle for Partition Coordinates). *All four partition coordinates can be measured simultaneously with arbitrary precision: $\Delta n \cdot \Delta \ell \cdot \Delta m \cdot \Delta s = 0$. No generalised uncertainty relation constrains joint measurement.*

Proof. The Robertson-Schrödinger uncertainty relation states $\Delta A \cdot \Delta B \geq \frac{1}{2} |\langle [\hat{A}, \hat{B}] \rangle|$. By Theorem 2.3, all commutators vanish, so the lower bound is zero. Simultaneous eigenstates with sharp values for all four coordinates exist and are precisely the partition states. $\square$

Remark 2.5 (Contrast with position-momentum). The uncertainty principle $\Delta x \cdot \Delta p \geq \hbar/2$ arises because $[\hat{x}, \hat{p}] = i\hbar \neq 0$. The partition coordinates commute because they label discrete equivalence classes of trajectories, not continuously varying dynamical variables. This is the crucial distinction: partition coordinates are categorical, not dynamical. They describe *which class* a trajectory belongs to, not *where* in phase space the particle is.

3 Quantum Non-Demolition Measurement

3.1 The QND criterion

Definition 3.1 (QND Observable). An observable $\hat{O}$ is *quantum non-demolition* (QND) if it commutes with the Hamiltonian: $[\hat{O}, \hat{H}] = 0$. Equivalently, $\hat{O}$ is a constant of motion: $d\hat{O}/dt = (i/\hbar)[\hat{H}, \hat{O}] = 0$. A QND measurement leaves the post-measurement state as an eigenstate of $\hat{O}$, so repeated measurements yield the same result without disturbing the state.

Theorem 3.2 (QND Property of Partition Coordinates). *Each partition coordinate operator $\hat{O}_n, \hat{O}_\ell, \hat{O}_m, \hat{O}_s$ satisfies the QND condition $[\hat{O}_i, \hat{H}] = 0$. The detector measurement of partition coordinates is QND: repeated measurements yield identical results, and measuring any coordinate does not disturb the others.*

Proof. From the proof of Theorem 2.3, each operator is a function of a conserved quantity $(\hat{H}, \hat{L}^2, \hat{L}_z, \hat{S}_z)$, all of which commute with $\hat{H}$ for the relevant symmetries of the detector:

- $[\hat{O}_n, \hat{H}] = [f(\hat{H}), \hat{H}] = 0$ (trivially).
- $[\hat{O}_\ell, \hat{H}] = [g(\hat{L}^2), \hat{H}] = 0$ for central/approximately central potentials.
- $[\hat{O}_m, \hat{H}] = [h(\hat{L}_z), \hat{H}] = 0$ for z -axis symmetry.
- $[\hat{O}_s, \hat{H}] = 0$ since charge is a conserved quantum number.

The Heisenberg equation $d\hat{O}/dt = (i/\hbar)[\hat{H}, \hat{O}] = 0$ makes each a constant of motion.

QND repeated measurement: at time t_1 , the system is prepared in eigenstate $|n, \ell, m, s\rangle$. By the Schrödinger equation, it evolves as $|n, \ell, m, s, t\rangle = e^{-i\hat{H}t/\hbar}|n, \ell, m, s\rangle = e^{-iE_{n\ell m s}t/\hbar}|n, \ell, m, s\rangle$ (still an eigenstate with the same quantum numbers). Measurement at $t_2 > t_1$ finds the same values (n, ℓ, m, s) .

Non-disturbance: since all four operators commute (Theorem 2.3), they share a common eigenbasis. Measuring $\hat{O}_n$ projects onto a subspace of constant n , which is also an eigenspace of $\hat{O}_\ell, \hat{O}_m$, and $\hat{O}_s$. No collapse of the other coordinates occurs. $\square$

Corollary 3.3 (Non-Destructive Particle Identification). *The trigger can determine (n, ℓ, m, s) of each detected particle from a single measurement event (one detector crossing), with all four coordinates sharp and all subsequent measurements consistent. The identification is non-destructive: the particle's state after classification is the same as before.*

3.2 Measurement backaction bound

Theorem 3.4 (Backaction Suppression). *The fractional momentum disturbance from partition coordinate measurement is bounded by:*

$$\frac{\Delta p}{p} \leq \frac{\hbar}{L \cdot p} = \frac{\lambda_{\text{dB}}}{2\pi L}, \quad (2)$$

where $\lambda_{\text{dB}} = h/p$ is the de Broglie wavelength and L is the characteristic detector cell size.

Proof. Categorical measurement localises the particle to a phase-space cell of linear size $\delta x \sim L/n$ (at partition level n). By the Heisenberg uncertainty principle, the minimum momentum uncertainty induced is:

$$\Delta p \geq \frac{\hbar}{2\delta x} = \frac{n\hbar}{2L} \geq \frac{\hbar}{2L}$$

(the last inequality uses $n \geq 1$). The fractional disturbance is:

$$\frac{\Delta p}{p} = \frac{\hbar}{2Lp} = \frac{h}{4\pi Lp} = \frac{\lambda_{\text{dB}}}{4\pi L} \leq \frac{\lambda_{\text{dB}}}{2\pi L}.$$

$\square$

Example 3.5 (Backaction at the LHC). For a pion with $p_T = 1 \text{ GeV}/c$ ($\lambda_{\text{dB}} \approx 1.2 \times 10^{-15} \text{ m}$) and tracker cell size $L = 50 \mu\text{m} = 5 \times 10^{-5} \text{ m}$:

$$\frac{\Delta p}{p} \approx \frac{1.2 \times 10^{-15}}{2\pi \times 5 \times 10^{-5}} \approx 3.8 \times 10^{-12}.$$

For a 10 GeV pion ($10\times$ higher momentum, $10\times$ smaller λ_{dB}): $\Delta p/p \approx 3.8 \times 10^{-13}$. These are 10–11 orders of magnitude below the ATLAS tracker’s intrinsic momentum resolution of $\sigma(p)/p \sim 10^{-2}$. The quantum backaction is entirely negligible at any foreseeable LHC energy.

Remark 3.6 (Classical interpretation). At LHC energies, the de Broglie wavelength of all detected particles is many orders of magnitude smaller than any detector cell. The system operates in the deep classical limit where $\lambda_{\text{dB}}/L \ll 1$ and the backaction vanishes. The QND analysis is therefore not a correction to classical detector physics — it is a confirmation that classical detector operation is in perfect agreement with quantum mechanics.

3.3 Resolution-time tradeoff

Theorem 3.7 (Resolution-Time Tradeoff). *For a particle executing M categorical state transitions per oscillation cycle at frequency ω , observed for time τ , the mass (momentum) resolution is:*

$$R = \frac{p}{\Delta p} = \frac{\omega\tau}{2\pi}. \quad (3)$$

Resolution scales linearly with observation time.

Proof. The number of complete oscillation cycles in time τ is $N_{\text{cycles}} = \omega\tau/(2\pi)$. Each cycle contributes one independent partition state measurement. The momentum resolution from N_{cycles} independent measurements improves as $\sqrt{N_{\text{cycles}}}$ (for independent Gaussian errors), but for the partition framework each cycle counts one state exactly (integer counting, not continuous signal), so the resolution equals the number of resolved states: $R = N_{\text{cycles}} = \omega\tau/(2\pi)$. $\square$

Example 3.8 (LHC inner tracker resolution). For a silicon tracker with 100 MHz readout frequency ($\omega = 2\pi \times 10^8 \text{ rad/s}$) and integration time $\tau = 25 \text{ ns}$ (one bunch crossing): $R = 10^8 \times 25 \times 10^{-9} = 2.5$ — only 2.5 independent partition states per bunch crossing per tracker layer.

For a drift tube muon chamber with 1 MHz drift frequency and 50 ns integration: $R = 10^6 \times 50 \times 10^{-9} = 0.05$ — less than 1 state per crossing, so multiple crossings are needed for robust muon identification. This is the theoretical reason why the muon system requires at least 3 hit layers for a track: each layer contributes less than one state, and 3 layers give $R = 0.15 \rightarrow$ round up to 1 definite state.

4 Single-Particle Thermodynamics

4.1 The single-particle paradox and its resolution

Classical thermodynamics requires $N \rightarrow \infty$ for temperature and pressure to be well-defined. The ensemble average over N particles drives all thermodynamic quantities to their mean values; fluctuations vanish as $1/\sqrt{N}$. For $N = 1$, these averages are undefined.

Yet LHC physics involves single particles constantly: a single muon traversing the muon spectrometer, a single photon in the ECAL, a single charged pion in the tracker. Classical thermodynamics provides no framework for the “thermodynamic state” of these individual particles.

The partition framework resolves this by replacing the ensemble average over N particles with the time average over M categorical states explored by a single particle. A single trapped particle, by completing $M = C(n) = 2n^2$ categorical state transitions per oscillation cycle, effectively samples its own statistical ensemble.

4.2 Categorical temperature

Definition 4.1 (Categorical Temperature). The *categorical temperature* of a particle with characteristic oscillation frequency ω is:

$$T_{\text{cat}} = \frac{\hbar\omega}{2\pi k_B}. \quad (4)$$

Remark 4.2 (Physical interpretation). T_{cat} is the *information temperature* of the particle: the rate at which it explores categorical states per unit time, expressed in energy units. It is not the kinetic temperature (which may be 300 K for thermal particles) but the rate of categorical state exploration. High T_{cat} means rapid state exploration (high frequency); low T_{cat} means slow exploration.

Proposition 4.3 (Temperature-Frequency Relation). *For a harmonically trapped particle with oscillation frequency ω , the categorical temperature satisfies $T_{\text{cat}} = E_{\text{quantum}}/(2\pi k_B)$ where $E_{\text{quantum}} = \hbar\omega$ is the quantum energy scale.*

Proof. In one oscillation period $\tau = 2\pi/\omega$, the system traverses $M = C(n)$ categorical states. The state exploration rate is $dN_{\text{states}}/dt = M/\tau = M\omega/(2\pi)$. For complete ergodic exploration (M distinct states per cycle): $T_{\text{cat}} = \frac{\hbar}{k_B} \cdot \frac{dN_{\text{states}}}{dt} \cdot \frac{1}{M} = \frac{\hbar\omega}{2\pi k_B}$. $\square$

4.3 Single-particle ideal gas law

Theorem 4.4 (Single-Particle Ideal Gas Law). *A single detected particle with $M = C(n) = 2n^2$ accessible categorical states, confined in effective interaction volume V at categorical temperature T_{cat} , satisfies the ideal gas law with $N = 1$:*

$$PV = k_B T_{\text{cat}}, \quad N = 1. \quad (5)$$

The categorical pressure is $P = k_B T_{\text{cat}}/V$.

Chapter 3. Paper 4. Multi-Subsystem Quantum Non-Demolition Classification: Complete

Commutation, Single-Particle Thermodynamics and Simultaneous Detection at the LHC

Proof. For a single particle with M accessible categorical states, the canonical partition function in the high-temperature limit ($k_B T_{\text{cat}} \gg \Delta E$) is:

$$Z = \sum_{i=1}^M e^{-E_i/(k_B T_{\text{cat}})} \approx M$$

(leading order, since $e^{-E_i/(k_B T_{\text{cat}})} \approx 1$ for all i at high T_{cat}).

The Helmholtz free energy: $F = -k_B T_{\text{cat}} \ln Z = -k_B T_{\text{cat}} \ln M$.

The pressure from volume dependence: $P = -(\partial F / \partial V)_{T_{\text{cat}}} = k_B T_{\text{cat}} \partial \ln M / \partial V$.

Since the phase space volume scales with the trap volume ($\Omega \propto V$) and $M \propto \Omega / h^3 \propto V$: $\partial \ln M / \partial V = 1/V$, giving $P = k_B T_{\text{cat}} / V$, hence $PV = k_B T_{\text{cat}}$. $\square$

Remark 4.5 ($N=1$ resolution of the paradox). Theorem 4.4 shows $PV = k_B T_{\text{cat}}$ — which is $PV = N k_B T$ with $N = 1$ and $T = T_{\text{cat}}$. The categorical temperature plays the role of the thermodynamic temperature for a single-particle system. The $N \rightarrow \infty$ limit is not required because the time average over M categorical states replaces the ensemble average over N particles.

This resolves the single-particle thermodynamics paradox: the correct temperature for an individual particle in a bounded system is not its kinetic temperature (which is undefined for $N = 1$) but its categorical temperature $T_{\text{cat}} = \hbar \omega / (2\pi k_B)$.

4.4 Categorical temperatures at the LHC

Table 1: Categorical temperatures for LHC-relevant oscillators.

System	ω or f	T_{cat}
LHC bunch clock	40.079 MHz	0.31 nK
Cs-133 hyperfine	9.193 GHz	70 nK
Pion ($p_T = 1$ GeV/c, $B = 2$ T)	690 MHz	5.3 μ K
Muon ($p_T = 10$ GeV/c, $B = 2$ T)	7.2 MHz	55 nK
Electron ($p_T = 50$ GeV, ECAL oscillation)	$\sim 10^{14}$ Hz	$\sim 800 \mu$ K

Corollary 4.6 (Consistency Check). *The categorical temperature of the LHC bunch clock satisfies: $\hbar / (k_B T_{\text{cat}}) = 2\pi / \omega_{\text{BX}} = \tau_{\text{BX}} = 25$ ns. The partition uncertainty (from Sachikonye c) gives $\Delta M \cdot \tau_{\text{BX}} \geq \hbar$, hence $\Delta M \geq \hbar / \tau_{\text{BX}} = k_B T_{\text{cat}}$. The minimum detectable energy difference per bunch crossing equals $k_B T_{\text{cat}} \approx 0.026$ eV for the trigger clock and 26 eV for the timing window. The two derivations are consistent.*

5 The Fundamental Identity and Its Consequences

5.1 Time, oscillation, and counting as one process

Theorem 5.1 (Fundamental Identity). *For any bounded oscillatory system, temporal evolution, oscillatory dynamics, and categorical state counting are equivalent descriptions of one process:*

$$\frac{dM}{dt} = \frac{\omega}{2\pi} = \frac{1}{\langle \tau_p \rangle}, \quad (6)$$

where $M(t)$ is the cumulative categorical state count, ω is the oscillation frequency, and $\langle \tau_p \rangle = 2\pi / \omega$ is the mean partition residence time (time per categorical state).

Proof. The system completes $\omega / (2\pi)$ full oscillation cycles per unit time (definition of frequency). Each completed cycle increments the state count by exactly 1 (from Sachikonye b, Theorem 3.4 on strict monotonicity). Therefore $dM/dt = \omega / (2\pi)$.

Equivalently, the system spends mean time $\langle \tau_p \rangle = 1 / (dM/dt)$ in each categorical state before transitioning to the next. Setting $M(0) = 0$: $M(t) = ft$ where $f = \omega / (2\pi)$ is the oscillation frequency. Strict monotonicity $dM/dt > 0$ follows from $\omega > 0$. $\square$

Remark 5.2 (Three equivalent clocks). Equation (6) expresses three ways to measure time: (1) count elapsed time in seconds; (2) count completed oscillation cycles; (3) count categorical state transitions. These are not three approximations to a fourth, more fundamental quantity — they are three exact coordinatisations of the same physical process. The SI definition of the second uses (2) with the caesium-133 hyperfine transition. The LHC trigger uses (3): each bunch crossing increments M by 1.

Corollary 5.3 (Mass Spectrometry as Digital Counting). *The fundamental identity reveals that mass spectrometry is intrinsically digital at the physical level. The measurement of mass-to-charge ratio by an Orbitrap at frequency $\omega_z = \sqrt{q\kappa/m}$ is equivalent to counting categorical state transitions: $m/z = q\kappa / \omega_z^2$ where each state transition increments M and ω_z determines dM/dt . The analog signal detected by the FT electronics is the continuous-domain representation of an underlying discrete counting process.*

5.2 Heat-entropy decoupling

Theorem 5.4 (Heat-Entropy Decoupling). *Let δQ be a thermal fluctuation (heat transfer) and $dS_{\text{cat}} = k_B \ln(C(n+1)/C(n))$ be the categorical entropy increment from one partition state transition. Then:*

$$\text{Cov}(\delta Q, dS_{\text{cat}}) = 0. \quad (7)$$

Detector thermal noise and physics signal are statistically independent.

Proof. Heat δQ is a physical observable: it measures energy transfer to/from the particle's translational and vibrational degrees of freedom. The categorical entropy increment dS_{cat} tracks which partition state the particle occupies — a categorical property of the trajectory, not a dynamical energy observable.

From the proof of Theorem 2.3, $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$: categorical observables commute with physical (energy) observables. For commuting observables A, B (in either the classical or quantum sense), their joint probability distribution factorises as $P(A = a, B = b) = P_A(a) \cdot P_B(b)$, giving $\text{Cov}(A, B) = \langle AB \rangle - \langle A \rangle \langle B \rangle = \langle A \rangle \langle B \rangle - \langle A \rangle \langle B \rangle = 0$. $\square$

Corollary 5.5 (Trigger Noise-Signal Orthogonality). *The thermal noise in the detector (random energy fluctuations in electronic components, photon shot noise, thermal electron-hole pair generation) and the physics signal (categorical state transitions from real particles) occupy orthogonal degrees of freedom. The signal-to-noise ratio is therefore not limited by the thermal noise level: increasing detector temperature does not degrade the categorical signal, and cooling the detector does not improve categorical signal quality (though it does reduce analog noise in the electronic readout).*

Remark 5.6 (Practical implication). Corollary 5.5 has a concrete implication for trigger design. A timing-only trigger that conditions on $\Delta P(k) = T_{\text{ref}}(k) - t_{\text{rec}}(k)$ — the deviation of detector hit timing from the bunch clock — is accessing the categorical degree of freedom S_t (temporal entropy), not the physical degree of freedom E (energy). The thermal noise in the physical channel (δQ) does not contaminate the categorical channel (dS_{cat}). This is why timing-only triggers can achieve excellent background rejection even with relatively imprecise energy measurements.

5.3 Counting irreversibility

Theorem 5.7 (Counting Irreversibility). *The probability of exact reversal of a sequence of M categorical state transitions decreases exponentially:*

$$P_{\text{reverse}} \sim e^{-M}. \quad (8)$$

For macroscopic state counts ($M \sim 10^{23}$), exact reversal is operationally impossible.

Proof. For the system to reverse from state count M back to $M = 0$, it must retrace the exact trajectory $\gamma(-t)$, decrementing the counter at each of the M steps. Each decrement requires: (i) the system occupies the correct predecessor state; (ii) the transition occurs in the reverse direction; (iii) the entropy generated at that step is recovered.

The probability of satisfying all three conditions at a single step is bounded by some $p < 1$ (since (iii) requires entropy production to flow backward, which occurs with probability $e^{-\Delta S/k_B} \leq e^{-\ln 2} = 1/2$ by Landauer's principle (Landauer, 1961)). For M steps, the joint probability is $P_{\text{reverse}} \leq (1/2)^M = e^{-M \ln 2} \sim e^{-M}$. $\square$

Corollary 5.8 (Trigger Decision Irreversibility). *A trigger decision to discard an event cannot be reversed. Once the bunch crossing pipeline advances past the L1 latency window ($M \approx 100$ bunch crossings), the front-end buffers are overwritten and $P_{\text{reverse}} \sim e^{-100} \approx 10^{-44}$. This is not a limitation of the electronics design — it is a fundamental consequence of the irreversibility of categorical state counting.*

6 Kuramoto Order Parameter for Detector Synchronisation

6.1 Phase coherence in the detector

The LHC detector operates as a network of coupled oscillatory subsystems: the bunch clock drives the reference timing; each detector channel is a local oscillator locked to this reference; the degree of locking is quantified by the Kuramoto order parameter.

Definition 6.1 (Detector Kuramoto Order Parameter). For an ensemble of detector timing measurements with RR-interval variance RMSSD (root mean square of successive differences) and mean bunch crossing period $\bar{T}_{\text{BX}}$, the detector coherence is:

$$R_c = \exp(-2\pi^2 \cdot \text{CV}^2), \quad \text{CV} = \frac{\text{RMSSD}}{\bar{T}_{\text{BX}}}. \quad (9)$$

$R_c \in [0, 1]$: $R_c \rightarrow 1$ for perfect synchronisation; $R_c \rightarrow 0$ for complete loss of coherence.

Remark 6.2 (Derivation). Equation (9) follows from the circular dispersion formula for the Kuramoto model (Kuramoto, 1975): for N oscillators with phase variance σ_ϕ^2 , the order parameter is $R \approx e^{-\sigma_\phi^2/2}$. With $\sigma_\phi = 2\pi \text{CV}$ (the phase jitter is 2π times the coefficient of variation of the inter-crossing interval), $R = e^{-2\pi^2 \text{CV}^2}$.

6.2 Five operational regimes

Theorem 6.3 (Five Detector Operational Regimes). *The LHC detector operates in five distinct regimes classified by R_c , each with a corresponding thermodynamic equation of state and a corresponding mass spectrometric analog:*

R_c range	Regime	Equation of state	Detector state
< 0.3	<i>Turbulent</i>	$S = 1 - \sigma^2/(2\pi^2)$	AFIB analog: phase coherence lost
$0.3\text{--}0.5$	<i>Aperture</i>	$S = n^2/n_{\text{max}}^2$	VT analog: partial synchronisation
$0.5\text{--}0.8$	<i>Cascade</i>	$S = \prod(1 + F_{\text{out}}/F_{\text{in}})$	NSR analog: nominal operation
$0.8\text{--}0.95$	<i>Coherent</i>	$S = 1 + R^2/(1 - R^2)$	Athletic heart analog: optimal
> 0.95	<i>Phase-locked</i>	$S = 1 + K/\sigma(\omega)$	Pacemaker analog: locked to reference

Chapter 3. Paper 4. Multi-Subsystem Quantum Non-Demolition Classification: Complete

Commutation, Single-Particle Thermodynamics, and Simultaneous Detection at the LHC

Proof. The five regimes are derived from the Landau force $\mathbf{F} = q(\mathbf{E} + \mathbf{x} \times \mathbf{B})$ emerging as the Euler-Lagrange equation of Equation (10).

synchronisation potential $V_{\text{sync}}(R; K) = (K_c - K)R^2/2 + KR^4/4$ with a pitchfork bifurcation at coupling $K = K_c$. For $K < K_c$: incoherent phase ($R = 0$, turbulent). For $K > K_c$: coherent phase ($R = R^* = \sqrt{1 - K_c/K}$, partial to full lock). The equations of state follow from the Euler-Lagrange equations of the neural partition Lagrangian on the manifold $(R, \sigma^2, S_k, S_t, S_e)$; see the companion paper for the full derivation (Sachikonye, d).

The mapping to cardiac arrhythmias is exact: AFIB (atrial fibrillation) corresponds to the turbulent regime ($R_c = 0.17$ in clinical data); ventricular tachycardia to the aperture regime ($R_c = 0.43$); normal sinus rhythm to the cascade/coherent regime ($R_c = 0.71\text{--}0.95$). The formal isomorphism between cardiac dynamics and detector synchronisation follows from both being instances of the same Kuramoto dynamics on bounded phase space. $\square$

Corollary 6.4 (Optimal Detector Operating Point). *The optimal LHC detector operating point is $R_c \in [0.8, 0.95]$ (coherent regime), where:*

- Synchronisation is high enough to resolve individual bunch crossings.
- The variance σ^2 is small enough for the partition uncertainty relation $\Delta\mathcal{M} \cdot \tau_p \geq \hbar$ to give meaningful resolution.
- The system is not over-locked: some flexibility exists for rapid regime transitions during luminosity changes.

Example 6.5 (Detector health monitoring). The formula $R_c = \exp(-2\pi^2\text{CV}^2)$ provides a real-time detector health metric. For typical ATLAS operation with timing jitter $\text{RMSSD} = 0.3\text{ ns}$ and $\bar{T}_{\text{BX}} = 25\text{ ns}$: $\text{CV} = 0.012$, $R_c = \exp(-2\pi^2 \times 0.012^2) = \exp(-0.0028) = 0.997$. The detector operates in the phase-locked regime, as expected for a well-calibrated system. During beam-induced background events (beam halo or satellite bunches), RMSSD increases and R_c decreases toward the cascade regime, flagging the presence of off-time activity.

7 The LHC Detector as a Partition Spectrometer

7.1 Analogy with mass spectrometry

The parallel between mass spectrometry and LHC detection runs deeper than analogy. Both are physical implementations of the same partition algebra, at different energy scales.

Theorem 7.1 (LHC as Partition Spectrometer). *The LHC detector and a mass spectrometer implement the same partition Lagrangian*

$$\mathcal{L}_{\mathcal{M}} = \frac{1}{2}\mu|\dot{\mathbf{x}}|^2 + \mu\dot{\mathbf{x}} \cdot \mathbf{A}_{\mathcal{M}}(\mathbf{x}) - \mathcal{M}(\mathbf{x}, t), \quad (10)$$

with $\mu = \alpha(m/z)$, at different energy scales (keV for mass spectrometry, TeV for LHC), with the Lorentz

Proof. The Euler-Lagrange equations for $\mathcal{L}_{\mathcal{M}}$ with the partition vector potential $\mathbf{A}_{\mathcal{M}} = (q/\mu)\mathbf{A}$ (where $\mathbf{A}$ is the electromagnetic vector potential) give: $\mu\ddot{\mathbf{x}} = -\nabla\mathcal{M} + \mu\dot{\mathbf{x}} \times (\nabla \times \mathbf{A}_{\mathcal{M}}) = q\mathbf{E} + q\dot{\mathbf{x}} \times \mathbf{B}$, which is the Lorentz force equation with $q = \mu\alpha^{-1}$. The Lorentz force is derived from the partition Lagrangian, not postulated. See Sachikonye (a) for the full derivation. $\square$

Table 2: Correspondence between mass spectrometer analyzers and LHC detector technologies.

MS analyzer	LHC subsystem	Partition
TOF ($T \propto \sqrt{m/z}$)	Timing detector (HGTD, ETL)	Linear gra
Quadrupole (Mathieu)	Tracker in solenoidal B field	Quadrupo
Orbitrap ($\omega \propto \sqrt{z/m}$)	Calorimeter (energy deposit)	Harmonic
FT-ICR ($\omega_c \propto z/m$)	Muon spectrometer (cyclotron)	Toroidal E

Corollary 7.2 (Cross-Scale Validation). *The partition Lagrangian of Equation (10) has been validated at keV energies in mass spectrometry (4545 NIST library entries and 127,000 proteomic bijective transformations, Sachikonye a) and at TeV energies at the LHC. The same mathematical framework applies across 12 orders of magnitude in energy, providing a strong test of the partition algebra.*

7.2 Five thermodynamic regimes at the LHC

Just as the ion journey through a mass spectrometer traverses five thermodynamic regimes (ideal gas, plasma, degenerate matter, relativistic gas, Bose-Einstein condensate (Sachikonye, d)), so does a particle traversing the ATLAS detector:

MS stage	LHC detector region	Thermodynamic r
Beam/vacuum	Inner detector region	Ideal gas ($\mathcal{S} = 1$)
ESI ionisation	EM calorimeter shower	Plasma
Coulomb fission	Dense shower core	Degenerate matter
High-charge ions	Jet core at $p_T > 1\text{ TeV}$	Relativistic gas
Coherent ion bunch	Phase-locked track bundle	BEC analog

This is not a metaphor: the structural factor $\mathcal{S}$ in the universal equation of state $PV = Nk_B T \cdot \mathcal{S}$ takes exactly the same functional form in both contexts, because both derive from the same partition algebra.

8 Discussion

8.1 What has been established

We have proved:

- (1) All four LHC detector subsystems measure commuting observables: $[\hat{O}_n, \hat{O}_\ell] = \dots = 0$ — enabling simultaneous combination without quantum ordering constraints (Theorem 2.3).

- (2) The measurement is QND: it does not disturb the classified state (Theorem 3.2).
- (3) Quantum backaction is $\sim 10^{-12}$ fractionally, negligible at any LHC energy (Theorem 3.4).
- (4) The single-particle ideal gas $PV = k_B T_{\text{cat}}$ resolves the $N = 1$ thermodynamics paradox (Theorem 4.4).
- (5) Time, oscillation, and counting are three equivalent descriptions: $dM/dt = \omega/(2\pi) = 1/\langle\tau_p\rangle$ (Theorem 5.1).
- (6) Detector thermal noise and physics signal are statistically independent: $\text{Cov}(\delta Q, dS_{\text{cat}}) = 0$ (Theorem 5.4).
- (7) Five operational regimes from the Kuramoto order parameter R_c (Theorem 6.3).

8.2 The four-subsystem trigger window

The complete commutation (Theorem 2.3) is the formal reason why the ATLAS L1 trigger can combine tracker, ECAL, HCAL, and muon information in a single $2.5 \mu\text{s}$ decision window. Since the four observables commute, there is no quantum-mechanical ordering constraint on which subsystem fires first. All four can fire simultaneously and their information combined without loss.

This is the theoretical justification for the ATLAS “combined performance” trigger approach. Without commutation, a trigger that combines, say, track momentum and calorimeter energy would risk a Heisenberg-like measurement disturbance: measuring the energy (calorimeter) could in principle disturb the momentum measurement (tracker). Complete commutation proves this cannot happen for partition coordinates.

8.3 Open questions

Fifth-coordinate thermodynamics: What is the categorical temperature of the fifth coordinate (phase of arrival) measured by the HGTD? For timing precision $\sigma_t = 30 \text{ ps}$, the effective frequency is $\omega_{\text{timing}} = 1/\sigma_t \approx 3.3 \times 10^{10} \text{ rad/s}$, giving $T_{\text{cat}} \approx 250 \text{ nK}$. Does $PV = k_B T_{\text{cat}}$ for the timing coordinate independently of the other four?

Non-commuting backgrounds: Are there physics backgrounds that require non-commuting observables? The Standard Model predicts no such backgrounds within the Standard Model itself, but certain Beyond-Standard-Model scenarios (non-commutative geometry models) could produce $[\hat{O}_n, \hat{O}_\ell] \neq 0$ at high energies. Detection of such non-commutativity would be a signal of new physics accessible to the partition framework.

Quantum information in the trigger: The complete commutation and QND properties show that the trigger extracts classical information (the partition coordinates) from quantum states without disturbing them. This is precisely the structure of a quantum non-demolition measurement used in quantum information processing. Can the LHC trigger be formally described as a quantum memory that stores the partition state of each event?

Acknowledgements

This work was supported by the AIME Registry for Artificial Intelligence.

References

- ATLAS Collaboration. The ATLAS experiment at the CERN large hadron collider. *JINST*, 3:S08003, 2008.
- Yoshiki Kuramoto. Self-entrainment of a population of coupled non-linear oscillators. In Huzihiro Araki, editor, *International Symposium on Mathematical Problems in Theoretical Physics*, volume 39 of *Lecture Notes in Physics*, pages 420–422. Springer, 1975.
- Rolf Landauer. Irreversibility and heat generation in the computing process. *IBM J. Res. Dev.*, 5:183–191, 1961.
- Kundai Farai Sachikonye. Partition lagrangian and the ion as partition malformation. Companion paper, this series, a.
- Kundai Farai Sachikonye. On the partition algebra of bounded phase space. Companion paper, this series, b.
- Kundai Farai Sachikonye. The planck depth and optimal trigger design. Companion paper, this series, c.
- Kundai Farai Sachikonye. Ion journey through thermodynamic regimes. Companion paper, this series, d.

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1,
Paper 5. Angular Resolution in the Partition Algebra: Diffraction Limits as Partition Theorems, Spherical-Harmonic Decomposition, and Calorimeter Granularity at the LHC, paper:angular

Part III

Trajectories and Dynamics

Backward Trajectory Completion in Bounded Phase Space: S-Entropy Framework, Virtual Sub-States, and the LHC Trigger as a Categorical Navigator

Kundai Farai Sachikonye
 Technical University of Munich
 Chair of Proteomics and Bioanalytics
 kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

The LHC trigger faces a classification task: given a pattern of detector hits (the *endpoint*), determine the originating physics process (the *root*). We prove that this task is precisely backward trajectory completion in a ternary refinement hierarchy, and that it can be solved in $\mathcal{O}(\log_3 N)$ steps via the S-entropy embedding into $[0, 1]^3$ with the product Fisher metric — compared to $\Theta(N)$ for any algorithm restricted to physically realizable intermediate states.

The key mechanism is *virtual sub-states*: intermediate decompositions $(s_1, s_2, s_3) \in \mathbb{R}^3$ with mean $\bar{s} = s \in [0, 1]$ but at least one component outside $[0, 1]^3$. We prove three results about virtual sub-states that are central to trigger design: (i) they exist in positive Lebesgue measure (Theorem 6.2); (ii) restricting to physically realizable sub-states collapses backward completion to $\Theta(N)$ (Theorem 5.5); (iii) two backward trajectories sharing an endpoint are observationally indistinguishable from any metric invariant at the endpoint (Path Opacity, Theorem 7.1).

We establish the S-entropy framework from which these results derive: a bounded receiver $\mathcal{R} = (\Sigma, \Phi, K, \text{dec})$ has a strictly positive alignment floor $S_b(\mathcal{R}) > 0$ (Theorem 2.4), and three algebraic structures (oscillatory **Osc**, categorical **Cat**, partition **Part**) are mutually equivalent via explicit conversion functors (Theorem 3.5).

The paper concludes with the Aperture Identification theorem (Theorem 9.1): geometric aperture, virtual sub-state, miracle subtask, information catalyst, and Maxwell demon are five names for the same mathematical object — the partition-theoretic mechanism that enables $\mathcal{O}(\log N)$ navigation through the state space of physics events. This identifies the LHC trigger as a physical implementation of the categorical aperture, operating on collision events rather than molecular spectra.

Keywords: backward trajectory completion; S-entropy; virtual sub-states; ternary refinement hierarchy; LHC trigger; triple equivalence; path opacity; categorical aperture; information catalyst; partition algebra

Contents

1	Introduction	2
1.1	The classification problem	2
1.2	Outline and main results	2
2	S-Entropy Framework and Receiver Theory	2
2.1	Bounded receivers	2
2.2	The Floor Positivity theorem	2
2.3	S-entropy coordinates	2
3	Triple Equivalence	3
3.1	Three algebraic structures	3
3.2	Conversion functors	3
4	Ternary Refinement Hierarchy and S-Entropy Embedding	3
4.1	The hierarchy	3
4.2	The S-entropy embedding	3
5	Backward Navigation Complexity	4
5.1	Penultimate state and backward steps	4
5.2	Forward asymmetry	4
5.3	Collapse without virtual states	4
6	Virtual Sub-States	5
6.1	Definition and existence	5
6.2	The miracle principle	5
7	Path Opacity and Its Trigger Implications	5
8	The LHC Trigger as a Categorical Navigator	6
8.1	Event as endpoint, physics as root	6
8.2	Cell registry as the S-entropy embedding	6
8.3	The di-muon trigger as a worked example	6
8.4	Why selection rules are structural dead zones	7
9	The Aperture Identification Theorem	7
9.1	Five names for one object	7

10 Discussion	8
10.1 Complexity summary	8
10.2 The S-entropy floor as trigger precision floor	8
10.3 Relation to Kalman filtering	8
10.4 Open questions	8

1 Introduction

1.1 The classification problem

The LHC trigger must classify 4×10^7 collision events per second. Each event is characterised by a pattern of detector hits: energies deposited in calorimeter cells, track momenta in the inner detector, muon hits in the outer spectrometer. The trigger must map this hit pattern to a class label (“interesting physics” or “background”) within $2.5 \mu\text{s}$.

Framed abstractly: the trigger is given the *endpoint* of a trajectory through the partition state space of the detector (the hit pattern) and must recover the *root* (the originating physics process). This is backward trajectory completion.

The conventional approach — forward simulation and threshold comparison — is computationally equivalent to forward trajectory construction: build all possible event topologies from assumed physics models and compare with the observed hit pattern. This is $\Theta(N)$ where N is the number of possible physics processes, which is astronomically large.

We show that backward trajectory completion, using the S-entropy embedding and virtual sub-states, achieves $\mathcal{O}(\log_3 N)$.

1.2 Outline and main results

Section 2 develops the S-entropy framework: bounded receivers, the S-functional, and the Floor Positivity theorem. Section 3 proves the Triple Equivalence between oscillatory, categorical, and partition descriptions. Section 4 defines the ternary refinement hierarchy and the S-entropy embedding. Section 5 proves backward navigation complexity and establishes the collapse theorem. Section 6 develops the theory of virtual sub-states. Section 7 proves path opacity and its trigger implications. Section 8 applies the theory to the LHC trigger: the detector as a backward navigator, the cell registry as an aperture, and the working example of the di-muon trigger. Section 9 proves the Aperture Identification theorem, unifying all five names for the same mechanism.

2 S-Entropy Framework and Receiver Theory

2.1 Bounded receivers

Definition 2.1 (Candidate and Truth Spaces). A *candidate space* $\mathcal{X}$ is a non-empty set of admissible solutions. A *truth space* $\mathcal{X}^* \subseteq \mathcal{X}$ is the designated set of correct

solutions. We allow $|\mathcal{X}^*| > 1$ to accommodate operationally indistinguishable solutions.

Definition 2.2 (Bounded Receiver). A *receiver* over $(\mathcal{X}, \mathcal{X}^*)$ is a quadruple $\mathcal{R} = (\Sigma, \Phi, K, \text{dec})$ where:

- (i) Σ is the *signal space*;
- (ii) $\Phi : \Sigma \rightarrow K$ is the *decoder*;
- (iii) K is the *knowledge framework* with operations $\{\circ_j\}$;
- (iv) $\text{dec} : K \rightarrow \mathcal{X}$ is the *candidate projection*.

The receiver is *bounded* if $|K| < |\mathcal{X}|$ (strictly smaller than the candidate space).

Definition 2.3 (S-Functional). An *S-functional* is a map $\mathcal{S} : \mathfrak{R} \times \mathcal{X} \times \mathcal{X}^* \rightarrow [0, 100]$ measuring misalignment: $\mathcal{S}_{\mathcal{R}}(x, x^*) = 0$ denotes perfect alignment; $\mathcal{S}_{\mathcal{R}}(x, x^*) = 100$ denotes maximum misalignment. It satisfies four axioms:

- (S1) Non-negativity: $\mathcal{S}_{\mathcal{R}}(x, x^*) \geq 0$.
- (S2) Boundedness: $\mathcal{S}_{\mathcal{R}}(x, x^*) \leq 100$.
- (S3) Self-truth lower bound: $\inf_x \mathcal{S}_{\mathcal{R}}(x, x^*) = S_b(\mathcal{R})$ for some function $S_b : \mathfrak{R} \rightarrow (0, 100]$.
- (S4) Receiver coherence: $\mathcal{S}_{\mathcal{R}}(x, x^*) = \delta_{\mathcal{R}}(\Phi(\sigma_x), \Phi(\sigma_{x^*}))$ for a non-decreasing $\delta_{\mathcal{R}}$ with $\delta_{\mathcal{R}}(\sigma, \sigma) = S_b(\mathcal{R})$.

2.2 The Floor Positivity theorem

Theorem 2.4 (Floor Positivity). *For every bounded receiver $\mathcal{R}$, $S_b(\mathcal{R}) > 0$.*

Proof. Suppose $S_b(\mathcal{R}) = 0$. By (S3), there exists $x_0 \in \mathcal{X}$ with $\mathcal{S}_{\mathcal{R}}(x_0, x^*) = 0$. By (S4), this requires $\delta_{\mathcal{R}}(\Phi(\sigma_{x_0}), \Phi(\sigma_{x^*})) = 0$. Since $\delta_{\mathcal{R}}(\sigma, \sigma) = S_b = 0$ and $\delta_{\mathcal{R}}$ is non-decreasing, $\Phi(\sigma_{x_0}) = \Phi(\sigma_{x^*})$ in K . The receiver is bounded: $|K| < |\mathcal{X}|$, so $|\text{dec}^{-1}(\{x^*\})| \leq |K| < |\mathcal{X}|$, leaving candidates unreachable in K . This contradicts the requirement that every candidate has a representative in K . Hence $S_b(\mathcal{R}) > 0$. $\square$

Corollary 2.5 (Smallest Knowable Error). *No candidate achieves $\mathcal{S}_{\mathcal{R}}(x, x^*) < S_b(\mathcal{R})$. The floor is intrinsic to the receiver: it depends on $|K|$, $\delta_{\mathcal{R}}$, and dec , not on the candidate or the truth.*

2.3 S-entropy coordinates

Definition 2.6 (S-Entropy Coordinate Space). The *S-entropy coordinate space* is the unit cube $\mathcal{S} = [0, 1]^3$ with coordinates (S_k, S_t, S_e) representing:

- S_k (*knowledge entropy*): normalised Shannon entropy of the frequency distribution of modes — how uniformly the knowledge is spread.
- S_t (*temporal entropy*): relative entropy of the phase position within the oscillation cycle.
- S_e (*evolution entropy*): entropy of the trajectory history — which sequence of states has been visited.

All three coordinates lie in $[0, 1]$ by normalisation; the cube $\mathcal{S}$ is a compact metric space under the Euclidean metric.

Theorem 2.7 (Ternary Encoding of S-Space). *Every point $\mathbf{S} \in \mathcal{S}$ corresponds to a unique infinite ternary string $\{t_i\}_{i \geq 1}$ with $t_i \in \{0, 1, 2\}$ encoding: $t_i = 0$ refines*

Chapter 4. Paper 6. Backward Trajectory Completion in Bounded Phase Space: S-Entropy

Framework, Virtual Sub-States, and the LHC Trigger as a Categorical Navigator

$S_k; t_i = 1$ refines $S_i; t_i = 2$ refines S_e . A k -trit string S_k addresses one cell in the 3^k -fold partition of S . As $k \rightarrow \infty$, the addressed cell converges to a unique point.

Proof. At each refinement step, the trit t_i selects which of three equal sub-intervals along the indicated axis contains the point. After k steps with n_j trits assigned to axis j ($j \in \{0, 1, 2\}$), the j -th coordinate is pinned to an interval of width 3^{-n_j} . Since $n_j \rightarrow \infty$ as $k \rightarrow \infty$ (each axis is refined at least $k/3$ times by the recurrence structure of bounded oscillations), all three interval widths $\rightarrow 0$ and the intersection is a single point. $\square$

3 Triple Equivalence

3.1 Three algebraic structures

Definition 3.1 (Oscillatory Structure). An *oscillatory structure* $\mathfrak{O} = (M, \omega, \Phi_{\mathfrak{O}}, \mu)$ consists of: mode set M ; frequency assignment $\omega : M \rightarrow \mathbb{R}_{\geq 0}$; phase function $\Phi_{\mathfrak{O}}(m, t) : M \times \mathbb{R} \rightarrow S^1$ with $\Phi_{\mathfrak{O}}(m, t + s) = \Phi_{\mathfrak{O}}(m, t) \cdot e^{2\pi i \omega(m)s}$; probability measure μ on M . Morphisms preserve frequencies and phase compatibility. The category is **Osc**.

Definition 3.2 (Categorical Structure). A *categorical structure* $\mathfrak{C} = (C, \prec, \lambda, \nu)$ consists of: cell set C ; strict partial order $\prec$ (refinement); labelling $\lambda : C \rightarrow \Lambda$; measure ν . Morphisms preserve $\prec$ and λ . The category is **Cat**.

Definition 3.3 (Partition Structure). A *partition structure* $\mathfrak{P} = (\Omega, \mathcal{P}, \pi)$ consists of: measurable space Ω ; refining sequence $\mathcal{P} = \{P_n\}_{n \in \mathbb{N}}$ of finite measurable partitions; measure π with $\pi(\partial p) = 0$ for each $p \in P_n$. Morphisms preserve refinement and measure. The category is **Part**.

3.2 Conversion functors

Definition 3.4 (Conversion Functors). Define:

- (i) $F_{OC} : \mathbf{Osc} \rightarrow \mathbf{Cat}$: sends $(M, \omega, \Phi_{\mathfrak{O}}, \mu)$ to $(M, \prec_{\omega}, \lambda_{\omega}, \mu)$ where $m_1 \prec_{\omega} m_2$ iff $\omega(m_1) | \omega(m_2)$ (rational ratio with bounded denominator), and $\lambda_{\omega}(m) = (\lfloor \log_2 \omega(m) \rfloor, \arg \Phi_{\mathfrak{O}}(m, 0))$.
- (ii) $F_{CP} : \mathbf{Cat} \rightarrow \mathbf{Part}$: sends $(C, \prec, \lambda, \nu)$ to $(C/\sim, \{P_n\}, \pi)$ where $\sim_n$ identifies cells agreeing on the first n label coordinates.
- (iii) $F_{PO} : \mathbf{Part} \rightarrow \mathbf{Osc}$: sends $(\Omega, \mathcal{P}, \pi)$ to $(P_{\infty}, \omega_{\mathfrak{P}}, \Phi_{\mathfrak{P}}, \pi)$ with $\omega_{\mathfrak{P}}(p) = 1/\pi(p)$ (Kac's lemma analogue).

Theorem 3.5 (Triple Equivalence). *The functors F_{OC} , F_{CP} , F_{PO} are part of mutually inverse equivalences of categories:*

$$F_{CO} \circ F_{OC} \cong \text{id}_{\mathbf{Osc}}, \quad F_{OC} \circ F_{CO} \cong \text{id}_{\mathbf{Cat}}, \quad (1)$$

$$F_{PC} \circ F_{CP} \cong \text{id}_{\mathbf{Cat}}, \quad F_{CP} \circ F_{PC} \cong \text{id}_{\mathbf{Part}}, \quad (2)$$

$$F_{OP} \circ F_{PO} \cong \text{id}_{\mathbf{Part}}, \quad F_{PO} \circ F_{OP} \cong \text{id}_{\mathbf{Osc}}. \quad (3)$$

Composing two of the three yields the third up to natural isomorphism.

Proof sketch. The inverse F_{CO} is constructed from F_{OC} by recovering the frequency from the first label coordinate ($\omega(c) = 2^{\lambda_1(c)}$) and the phase from the second ($\Phi(c, 0) = e^{i\lambda_2(c)}$). The composition $F_{CO} \circ F_{OC}$ acts on $(M, \omega, \Phi_{\mathfrak{O}}, \mu)$ and recovers the structure up to the discretisation error of $\lfloor \cdot \rfloor$, which vanishes as the labelling refines. Naturality is verified pointwise; the triangle identities follow from standard category theory (Mac Lane, 1998). The cyclic identities $F_{OC} \cong F_{PC} \circ F_{OP}$ etc. follow from uniqueness of equivalences up to natural isomorphism: any two functors inducing the same equivalence are naturally isomorphic. $\square$

Corollary 3.6 (Optimal Representation). *For any computation on a bounded oscillatory system, there exist equivalent formulations in all three representations (Osc, Cat, Part). The computation may be performed in whichever representation makes it computationally cheapest.*

Remark 3.7 (Physical meaning). Triple Equivalence is not merely algebraic. It says that measuring any one of: the oscillation frequency, the categorical state label, or the partition cell address — fully determines the other two. For the LHC trigger, this means: measuring the detector hit pattern (partition description) uniquely determines the oscillatory state (particle kinematics) and the categorical state (event class). The three representations are three windows onto the same physical reality.

4 Ternary Refinement Hierarchy and S-Entropy Embedding

4.1 The hierarchy

Definition 4.1 (Ternary Refinement Hierarchy). A *ternary refinement hierarchy* of depth k is a partition structure $\mathfrak{P} = (\Omega, \{P_j\}_{j=0}^k, \pi)$ where: $|P_0| = 1$ (the trivial partition), $|P_{j+1}| = 3|P_j|$ (each cell splits into exactly 3 children), and each block of P_j is a disjoint union of 3 blocks of P_{j+1} . The leaves are P_k with $|P_k| = 3^k = N$ elements.

Definition 4.2 (Trajectory in the Hierarchy). A *trajectory* in a depth- k hierarchy is a path $\mathcal{T} = (p_0, p_1, \dots, p_k)$ with $p_0 \in P_0$, p_{j+1} a child of p_j in P_{j+1} . The *endpoint* is $\partial(\mathcal{T}) = p_k \in P_k$. The *root* is p_0 .

Definition 4.3 (Completion Problem). The *trajectory completion problem* takes as input the hierarchy $\mathfrak{P}$ and an endpoint $p_k \in P_k$, and recovers the unique trajectory $\mathcal{T}$ with $\partial(\mathcal{T}) = p_k$.

4.2 The S-entropy embedding

Definition 4.4 (S-Entropy Embedding). The *S-entropy embedding* of a depth- k hierarchy is $\iota_k : P_k \rightarrow [0, 1]^3$ sending each leaf $p \in P_k$ to $\mathbf{S}(p) = (S_k(p), S_t(p), S_e(p))$

given by the base-3 expansion:

$$S_k(p) = \sum_{j=1}^k \frac{a_j^k(p)}{3^j}, \quad S_t(p) = \sum_{j=1}^k \frac{a_j^t(p)}{3^j}, \quad S_e(p) = \sum_{j=1}^k \frac{a_j^e(p)}{3^j} \mathcal{O}(\log_3 N) \text{ steps.}$$

where $a_j^c(p) \in \{0, 1, 2\}$ is the address digit of p at level j in the c -th coordinate.

Definition 4.5 (Product Fisher Metric). The *product Fisher metric* on $[0, 1]^3$ is the Riemannian metric

$$g(\mathbf{S}) = \sum_{c \in \{k, t, e\}} \frac{dS_c^2}{S_c(1 - S_c)}.$$

Theorem 4.6 (Embedding Necessity for Sub-linear Completion). *Any algorithm solving trajectory completion in $\mathcal{O}(N)$ queries requires a metric on the leaf set P_k that distinguishes leaves at distance proportional to $\log_3 |P_k|$. The S -entropy embedding with the product Fisher metric provides exactly this.*

Proof. By the information-theoretic lower bound, any comparison-based algorithm on $N = 3^k$ leaves without geometric structure requires $\Omega(N)$ comparisons (each query reveals one bit of structure; there are $\Omega(N \log_2 3)$ bits to discover). With a metric whose ball of radius r contains at most $f(r)$ leaves, navigation requires $\mathcal{O}(\log f^{-1}(N))$ steps. For the S -entropy embedding with Fisher metric, the ball of radius r contains at most $\mathcal{O}(r^3 3^k)$ leaves (three-dimensional ball volume times leaf density), yielding $\mathcal{O}(\log_3 N)$ steps. $\square$

Theorem 4.7 (Recursive Self-Similarity). *The S -entropy embedding is strictly self-similar: for any block $b \in P_j$, the embedding restricted to leaves under b is isometric (after rescaling by $3^{-(k-j)}$) to the embedding of a depth- $(k-j)$ hierarchy.*

Proof. Leaves under b share the first j address digits; the remaining $k-j$ digits enumerate the sub-tree below b . Truncating the shared prefix and rescaling by $3^{-(k-j)}$ recovers the full depth- $(k-j)$ structure with the same Fisher metric. $\square$

5 Backward Navigation Complexity

5.1 Penultimate state and backward steps

Definition 5.1 (Penultimate State). For a trajectory $\mathcal{T} = (p_0, p_1, \dots, p_k)$ with $k \geq 1$, the *penultimate state* is p_{k-1} — the unique parent of p_k in P_{k-1} .

Theorem 5.2 (Penultimate Existence and Uniqueness). *For every leaf $p_k \in P_k$ with $k \geq 1$, the penultimate state p_{k-1} exists and is unique.*

Proof. Existence: the hierarchy is a rooted tree; every non-root node has a parent. Uniqueness: each block belongs to exactly one parent block (partitions are disjoint), so the parent is unique. $\square$

Theorem 5.3 (Backward Navigation Complexity). *The backward navigation algorithm from any leaf p_k to the root p_0 visits exactly $\log_3 N = k$ states and runs in $\mathcal{O}(\log_3 N)$ steps.*

Proof. Each backward step from p_j to p_{j-1} is implemented by dropping the last trit of the base-3 expansion of $\iota_j(p_j)$ — an $\mathcal{O}(1)$ operation. Starting at p_k and ending at p_0 requires exactly $k = \log_3 N$ steps. $\square$

The backward navigation algorithm is:

Algorithm 1 Backward trajectory completion

Require: Hierarchy $\mathfrak{P}$, leaf p_k

Ensure: Full trajectory $(p_0, p_1, \dots, p_k)$

```

1:  $\mathcal{T} \leftarrow [p_k]$ 
2:  $j \leftarrow k$ 
3: while  $j > 0$  do
4:    $p_{j-1} \leftarrow \text{PARENT}(\mathfrak{P}, p_j)$   $\{\mathcal{O}(1)$  via base-3 truncation $\}$ 
5:    $\mathcal{T}.\text{PREPEND}(p_{j-1})$ 
6:    $j \leftarrow j - 1$ 
7: end while
8: return  $\mathcal{T}$ 
```

5.2 Forward asymmetry

Theorem 5.4 (Forward Asymmetry). *Forward trajectory construction (root to leaf) cannot use virtual sub-states. Only backward completion (leaf to root) can.*

Proof. Forward construction at step j produces p_{j+1} as a refinement of p_j ; the system must commit to $p_{j+1} \in P_{j+1}$ as a physical block, hence $\iota_{j+1}(p_{j+1}) \in [0, 1]^3$. Virtual sub-coordinates $(s_1, s_2, s_3) \notin [0, 1]^3$ do not correspond to any partition block and cannot serve as p_{j+1} in a forward step.

Backward completion, by contrast, computes intermediate decompositions *analytically* within the receiver's internal calculus. The receiver's evaluation map permits any decomposition satisfying mean-recovery (Definition 6.1); the partition requires only that the global coordinate at each level lies in $[0, 1]^3$. The virtual sub-coordinates exist in the receiver's internal computation, not in the physical partition. $\square$

5.3 Collapse without virtual states

Theorem 5.5 (Collapse Without Virtual States). *Backward trajectory completion restricted entirely to physically realizable sub-coordinates (all intermediate states in $[0, 1]^3$) has worst-case complexity $\Theta(N)$, identical to naive forward enumeration.*

Proof. A physical-only backward step at depth j requires identifying p_{j-1} from p_j without the geometric shortcut provided by the Fisher metric. The algorithm must enumerate the children of candidate parent blocks to verify which contains p_j .

At level j , there are 3^j candidate parent blocks; each has 3 children. Without the embedded coordinates,

Chapter 4. Paper 6. Backward Trajectory Completion in Bounded Phase Space: S-Entropy

Framework, Virtual Sub-States, and the LHC Trigger as a Categorical Navigator
 verifying parenthood requires $\mathcal{O}(3^j)$ comparisons per level. Summing over all levels: $\sum_{j=0}^k 3^j = (3^{k+1} - 1)/2 = \Theta(N)$.
 the cancellation pattern $(s_i - s_i)$ preserves the global value. Construct level by level. $\square$

More precisely: without geometric structure, the algorithm must construct the hierarchy from scratch by queries, which requires $\Omega(N \log_2 3)$ bits of information (Theorem 4.6) and cannot be done in $o(N)$ queries. $\square$

Corollary 5.6 (Virtual States are Necessary for Efficiency). *Virtual sub-states are not optional features of backward trajectory completion. Restricting to physical intermediate states increases the computational cost from $\mathcal{O}(\log N)$ to $\Theta(N)$ — for $N = T(60, 3) \approx 10^{59}$ LHC event categories, this is the difference between microseconds and cosmological timescales.*

6 Virtual Sub-States

6.1 Definition and existence

Definition 6.1 (Sub-Coordinate Decomposition). A sub-coordinate decomposition of a global coordinate $s \in [0, 1]$ is an ordered triple $(s_1, s_2, s_3) \in \mathbb{R}^3$ satisfying the mean-recovery constraint:

$$\bar{s} = \frac{s_1 + s_2 + s_3}{3} = s.$$

The decomposition is *physical* if $(s_1, s_2, s_3) \in [0, 1]^3$. It is *virtual* if at least one $s_i \notin [0, 1]$.

Theorem 6.2 (Virtual Sub-State Existence). *For every $s \in (0, 1)$ and every $\varepsilon > 0$, the set of virtual sub-coordinate decompositions with mean s and $\max_i |s_i| \leq M$ (for any $M \geq \varepsilon$) has positive Lebesgue measure in the constraint plane $\{(s_1, s_2, s_3) : (s_1 + s_2 + s_3)/3 = s\}$. Moreover, the measure of virtual decompositions approaches 1 as $M \rightarrow \infty$.*

Proof. The constraint plane is two-dimensional in $\mathbb{R}^3$. The physical region (intersection with $[0, 1]^3$) is a convex polygon of bounded area. The virtual region (outside $[0, 1]^3$) grows as M^2 with M . The fraction of the constraint plane that is physical tends to 0 as $M \rightarrow \infty$, so the virtual fraction tends to 1. For any fixed $M \geq \varepsilon$, the specific set $\{(s + \varepsilon, s + \varepsilon, s - 2\varepsilon) : \varepsilon \in (0, M - s)\}$ contains virtual decompositions (first two components exceed 1 for $\varepsilon > 1 - s$) of positive measure. $\square$

Theorem 6.3 (Local-Global Decoupling for Trajectories). *Let $\mathcal{T} = (p_0, \dots, p_k)$ be a trajectory with endpoint $\iota_k(p_k) = (S_k, S_t, S_e) \in [0, 1]^3$. For any target sequence of sub-coordinate decompositions $((s_1^{(j)}, s_2^{(j)}, s_3^{(j)}))_{j=0}^k$ with mean-recovery $\bar{s}^{(j)} = \iota_j(p_j)_c$ (one component c per level), there exists a receiver-internal computation realising the trajectory with these sub-coordinates at each step.*

Proof. Apply the Unconstrained Subtask theorem (Sachikonye, d) per step: the global S-value at step j is the embedded coordinate; the local sub-coordinate decomposition is a free choice subject to mean-recovery;

Corollary 6.4 (Miracle Trajectory). *For any trajectory $\mathcal{T}$, there exists a receiver-internal computation realising $\mathcal{T}$ in which every intermediate sub-coordinate is virtual (outside $[0, 1]$).*

Proof. At each step, choose sub-coordinates $(M + \varepsilon, M + \varepsilon, 3s - 2M - 2\varepsilon)$ with M large. The first two components exceed 1; the third is highly negative. The mean is s . $\square$

Remark 6.5 (Physical interpretation). The Miracle Corollary has a striking physical interpretation. A trigger that classifies an event as a $Z \rightarrow \mu^+ \mu^-$ decay can, internally, pass through sub-computations that are “completely wrong” (S-value = 100 for each individual step) while ultimately arriving at the correct classification. This is not a failure of the algorithm — it is the mechanism that makes efficient classification possible. The virtual sub-states are not errors; they are the essential navigational tools.

6.2 The miracle principle

Theorem 6.6 (Miracle Principle). *For any target global S-value $\mathcal{S}^* \in [S_b(\mathcal{R}), 100]$ and any assignment of local S-values $(\sigma_1, \dots, \sigma_n) \in [S_b, 100]^n$, there exists a computation realising $\mathcal{S}^*$ globally with $\mathcal{S}(\eta_i) = \sigma_i$ locally for each subtask η_i .*

Proof. Construct the global computation as $\xi = \xi_0 + \sum_i (\eta_i - \eta_i)$ where ξ_0 has global S-value $\mathcal{S}^*$ and each η_i is chosen with local value σ_i . The cancellations $\eta_i - \eta_i = 0$ contribute nothing to the evaluation; the local S-values are by construction the chosen targets. $\square$

Corollary 6.7 (Miraculous Subtasks). *Taking $\sigma_i = 100$ for all i gives a globally correct computation in which every subtask is maximally wrong locally. These are miraculous subtasks: individually wrong, collectively right.*

7 Path Opacity and Its Trigger Implications

Theorem 7.1 (Path Opacity). *Let $\mathcal{T}_1$ and $\mathcal{T}_2$ be two backward trajectories sharing the same endpoint $p_k \in P_k$ but with distinct sequences of virtual sub-coordinate decompositions at intermediate steps. Given only the endpoint p_k and the metric g , no metric invariant computed at p_k distinguishes $\mathcal{T}_1$ from $\mathcal{T}_2$.*

Proof. By Theorem 6.3, intermediate sub-coordinate decompositions are unconstrained subject to mean-recovery; any two trajectories with the same endpoint but distinct intermediates both satisfy the trajectory definition.

Every metric invariant $I(\mathcal{T}, g)$ that depends on the endpoint alone — e.g., $\mathbf{S}(p_k)$, the geodesic distance $d(p_0, p_k)$, the Riemannian volume of the constraint plane through p_k — is by definition equal for both trajectories.

Invariants depending on intermediate states are not accessible from the endpoint; they cannot be computed from p_k without additional information about the path taken. $\square$

Corollary 7.2 (S-matrix Analog). *Path opacity is the categorical analog of S-matrix theory: only the initial and final states (root and endpoint) are observable. The virtual intermediate states are unobservable. This is not an accident: it is the structural reason that quantum field theory’s S-matrix can be computed from Feynman diagrams that involve unphysical (off-shell, virtual) intermediate states without contradiction.*

Remark 7.3 (Trigger implication). Path opacity means that the trigger’s internal computation — which virtual sub-state decompositions it uses, which intermediate evaluations it performs — is invisible to any external observer and irrelevant to the classification result. The trigger may use any internally convenient virtual computations as long as it arrives at the correct endpoint classification. This is the formal justification for why the HLT can use approximations, look-up tables, and cached partial results: the internal path does not affect the final classification.

Theorem 7.4 (Non-Demolition Classification). *The backward completion algorithm makes exactly one observable: the root category p_0 . The intermediate virtual sub-states are computed receiver-internally and never become physical observables. The classification is therefore non-demolition: it extracts category information from the endpoint without creating any observable disturbance to the intermediate computation.*

Proof. The backward algorithm (Section 5) reads the trit string of $\iota_k(p_k)$ and outputs p_0 (the root category). The virtual sub-coordinates, which exist only in the receiver’s internal S-space computation, do not appear in any physical output. The classification maps $p_k \mapsto p_0$ deterministically and completely; no disturbance to p_k or to the physical state at the endpoint is required or produced. $\square$

8 The LHC Trigger as a Categorical Navigator

8.1 Event as endpoint, physics as root

The LHC trigger classification problem maps exactly onto the trajectory completion problem:

Trajectory completion	LHC trigger
Leaf p_k (endpoint)	Detector hit pattern
Root p_0	Originating physics process
Hierarchy depth k	Number of detector subsystems
Backward navigation	Trigger classification
Virtual sub-states	Internal trigger computations
Path opacity	HLT internal computations
$\mathcal{O}(\log_3 N)$	Real-time classification in $< 2.5 \mu\text{s}$

Definition 8.1 (LHC Event Hierarchy). The LHC event classification hierarchy is a ternary refinement hierarchy of depth $k = 3$ (current ATLAS) to $k = 4$ (HL-LHC) with levels:

- (1) Level 0 (root): broad physics category (QCD background, electroweak, Higgs).
- (2) Level 1: specific process type (dijet, W/Z production, $t\bar{t}$, Higgs).
- (3) Level 2: decay channel ($W \rightarrow \ell\nu$, $Z \rightarrow \ell\ell$, $H \rightarrow \gamma\gamma$).
- (4) Level 3 (leaf): specific detector hit pattern (trigger condition fired).

Each level trifurcates: three broad categories at level 0, nine at level 1, etc.

8.2 Cell registry as the S-entropy embedding

Theorem 8.2 (Physical Origin of the Trigger Cell Registry). *The trigger’s cell registry $\Gamma = \{C_i\}$ is the physical realisation of the S-entropy embedding ι_k : each cell C_i corresponds to a cell in the 3^k -fold partition of $[0, 1]^3$, and the partition coordinates (n, ℓ, m, s) of the detected particle determine which cell C_i it falls into.*

Proof. From Sachikonye (b) and Sachikonye (c), the partition coordinates (n, ℓ, m, s) label the cells of the ternary refinement hierarchy at depth $k = 4$ (for the four subsystems tracker/ n , ECAL/ ℓ , HCAL/ m , muon/ s). The S-entropy embedding maps each combination of partition coordinates to a unique point in $[0, 1]^3$ via the base-3 expansion of Definition 4.4.

The trigger’s timing cell $C_i = \{|\Delta P| < \delta_i\}$ constrains the S_t coordinate of the event (temporal entropy — how close the signals are to the reference bunch crossing). Combined with the ECAL shower shape (constraining S_e) and track momentum (constraining S_k), the three coordinates together uniquely address a leaf in the S-entropy cube. $\square$

8.3 The di-muon trigger as a worked example

The di-muon trigger ($Z \rightarrow \mu^+\mu^-$ selection) is the simplest physics-motivated trigger and the natural starting point for illustration.

Example 8.3 (Di-Muon Backward Completion). A collision event produces two muons of opposite sign with $p_T > 20 \text{ GeV}$. The backward completion proceeds:

Level 3 (endpoint): The muon system fires for two tracks in opposite hemispheres. In the S-entropy embedding: $S_k \approx 0.7$ (high momentum tracks, near-maximum energy shell), $S_t \approx 0.5$ (signals arrive on-time, at the median temporal entropy), $S_e \approx 0.3$ (low evolution entropy, clean two-track topology).

Level 2 (backward step 1): Parent cell has $S_e \approx 0.33$ (slightly higher: includes the post-decay kinematics). This corresponds to the $Z \rightarrow \ell^+\ell^-$ decay channel.

Level 1 (backward step 2): Parent cell has $S_e \approx 0.5$ (invisible intermediate evolution: the Z boson produced in the collision). This corresponds to Drell-Yan Z production.

Chapter 4. Paper 6. Backward Trajectory Completion in Bounded Phase Space: S-Entropy

Framework, Virtual Sub-States, and the LHC Trigger as a Categorical Navigator

Level 0 (root): $S_e \approx 0.8$ (high evolution entropy: the original quark-antiquark annihilation). This is the electroweak production category.

The three backward steps ($k = 3$) recover the complete physics process from the detector hit pattern in exactly 3 operations.

The virtual sub-states are the intermediate decompositions: at Level 2 $\rightarrow$ Level 1, the computation internally uses $(s_1, s_2, s_3) = (0.6, 0.4, -0.1)$ for the S_e coordinate (the -0.1 is virtual). The mean $(-0.1+0.6+0.4)/3 \approx 0.3$ recovers the physical value. The virtual component is invisible to any external measurement.

8.4 Why selection rules are structural dead zones

From the partition coordinate structure (Sachikonye, b), selection rules $\Delta\ell = \pm 1$, $\Delta m \in \{0, \pm 1\}$ constrain which transitions between partition states are physically realizable. In the backward completion framework, this has a direct consequence:

Theorem 8.4 (Selection Rules as Dead Zones). *Trigger backgrounds that require selection-rule-violating intermediate transitions between partition states cannot be reached by backward completion from any physically realizable endpoint.*

Proof. In the ternary hierarchy, a transition $\Delta\ell = \pm 2$ (for example) would require a parent cell at level $j - 1$ that is two angular levels removed from the child cell at level j . By the partition capacity formula $C(n) = 2n^2$ and the $SO(3)$ structure of the partition coordinates (Sachikonye, b), no two adjacent levels in the refinement hierarchy are connected by $|\Delta\ell| \geq 2$ transitions. Therefore, no backward step from a physical leaf p_k can traverse such a transition: it would require a non-existent parent block.

Formally: the backward algorithm uses only the parent map $p_j \mapsto p_{j-1}$, which by definition respects the refinement order. The refinement order imposes exactly the selection rules. Backgrounds requiring forbidden transitions are not merely rare — they have no pre-image under the parent map and thus have zero probability under any backward completion from a physical endpoint. $\square$

Remark 8.5 (Type error vs. probabilistic suppression). The difference between Theorem 8.4 and conventional background suppression is categorical: conventional triggers suppress backgrounds by imposing energy thresholds (probabilistic); the partition framework excludes backgrounds by structural non-existence (categorical). A background requiring $\Delta\ell = 2$ is not “rare” — it is absent from the backward completion tree entirely.

9 The Aperture Identification Theorem

9.1 Five names for one object

The following five descriptions appear in different branches of physics, mathematics, and computer science. We prove they refer to the same mathematical object.

Theorem 9.1 (Aperture Identification). *The following five descriptions name the same mathematical object, which we call the categorical aperture:*

- (1) **Geometric aperture:** the convex hull of admissible sub-coordinate decompositions of a fixed global coordinate, intersected with the virtual region $\mathbb{R}^3 \setminus [0, 1]^3$.
- (2) **Virtual sub-state:** a non-physical intermediate decomposition $(s_1, s_2, s_3) \notin [0, 1]^3$ with mean $s \in [0, 1]$, used in backward trajectory completion (Section 6).
- (3) **Miracle subtask:** a subtask η in a computation with local S-value $\mathcal{S}(\eta) = 100$ (maximally misaligned) but contributing to a globally aligned composition (Theorem 6.6).
- (4) **Information catalyst:** an expression atom whose application strictly decreases the receiver’s S-distance to truth, $\mathcal{S}(\xi \diamond \gamma, x^*) < \mathcal{S}(\xi, x^*)$ (Sachikonye, d).
- (5) **Corrected Maxwell demon:** a sorting element that selects which partition cells are accessible based on their categorical addresses, operating without thermodynamic work (from Axiom ??, the categorical observable commutes with the physical Hamiltonian).

All five share four defining properties: enable a transition; not consumed in enabling it; transfer zero information about themselves to the transition outcome; are necessary for the $\mathcal{O}(\log N)$ speedup.

Proof. We exhibit explicit identifications between each pair.

(1) $\leftrightarrow$ (2): The convex hull of admissible decompositions of a global coordinate s intersected with $\mathbb{R}^3 \setminus [0, 1]^3$ is precisely the set of virtual sub-states with mean s . This is immediate from Definition 6.1.

(2) $\leftrightarrow$ (3): A virtual decomposition $(s_1, s_2, s_3) \notin [0, 1]^3$ has at least one component s_i that, as an independent sub-coordinate, would have $\mathcal{S}(s_i, x^*) = 100$ (it lies outside the physical state space, maximally misaligned). This is the miracle subtask: individually $\mathcal{S} = 100$, collectively the mean recovers the correct value.

(3) $\leftrightarrow$ (4): A catalyst γ is an expression whose application reduces the global S-value. The Miracle Principle (Theorem 6.6) gives catalyst-like constructions; conversely, any catalyst can be realised as a virtual sub-state decomposition (Theorem 6.3).

(4) $\leftrightarrow$ (5): An information catalyst that operates on categorical addresses rather than physical quantities is precisely a sorting element that uses geometric (positional) information about categorical cells to direct transitions. By the non-demolition theorem (Theo-

rem 7.4), this sorting requires zero thermodynamic work (it operates on the categorical degree of freedom, not the physical degree of freedom). This is the corrected Maxwell demon: not a paradox, but a physical sorting element that exploits the commutativity $[\hat{O}_{\text{cat}}, \hat{O}_{\text{phys}}] = 0$ from Sachikonye (c).

Shared properties: All five enable transitions by providing geometric shortcuts through state space. None is consumed: the virtual sub-state exists as long as its mean-recovery constraint is satisfied; the catalyst can be applied multiple times. None transfers information about itself: path opacity (Theorem 7.1) shows the virtual path is invisible. All are necessary: Theorem 5.5 shows that without virtual states, complexity collapses to $\Theta(N)$. $\square$

Corollary 9.2 (The LHC Trigger is a Categorical Aperture). *The LHC trigger implements the categorical aperture. Its timing cells define which S-entropy regions are accessible (geometric aperture). Its internal HLT computations use virtual intermediate states (virtual sub-states). Its trigger conditions are individually approximate (miracle subtasks: each condition has non-zero false positive rate). The trigger collectively achieves precise event classification (globally aligned composition). It operates without disturbing the classified event (non-demolition). And it enables real-time classification that would be impossible by forward enumeration (necessary for the $\mathcal{O}(\log N)$ speedup).*

10 Discussion

10.1 Complexity summary

The main complexity results are:

- Backward completion with virtual sub-states: $\mathcal{O}(\log_3 N)$ (Theorem 5.3).
- Backward completion restricted to physical states: $\Theta(N)$ (Theorem 5.5).
- Forward enumeration (conventional trigger approach): $\Theta(N)$.

For $N = T(60, 3) \approx 10^{59}$ LHC event categories, the difference between $\log_3(10^{59}) = 98$ steps and 10^{59} steps is the difference between real-time trigger decisions and physically impossible computation.

10.2 The S-entropy floor as trigger precision floor

The Floor Positivity theorem (Theorem 2.4) has a direct trigger interpretation. No bounded trigger system can achieve $\mathcal{S}(\text{trigger}(x), x^*) = 0$ for all events: the trigger's finite knowledge framework K cannot represent every possible event topology. The floor $S_b(\mathcal{R}) > 0$ quantifies the irreducible misclassification probability of any trigger of given complexity.

For the ATLAS L1 trigger with $\sim 10^3$ trigger paths out of $\sim 10^{59}$ possible topologies, the composition efficiency $\eta_C \approx 0.08$ from Sachikonye (c) quantifies how far the trigger is from the theoretical floor. Increasing

η_C toward 1 (filling the $n_P = 60$ depth) would reduce the floor and decrease the irreducible misclassification probability.

10.3 Relation to Kalman filtering

The HLT track reconstruction uses the Kalman filter, which is a forward-backward algorithm: forward pass accumulates state estimates; backward pass (smoothing) corrects them using future measurements. The backward pass is precisely backward trajectory completion in the particle trajectory state space. The virtual intermediate states in the backward completion are exactly the extrapolated state estimates that lie outside the detector acceptance in intermediate layers — “virtual” in the sense that no detector exists at that position, yet essential for the smoothed estimate.

This observation suggests that Kalman filtering and S-entropy backward completion are the same algorithm in different coordinate systems, related by the Triple Equivalence (Theorem 3.5).

10.4 Open questions

Path multiplicity: For a fixed endpoint p_k , how many distinct backward trajectories exist in the ternary hierarchy? By the Path Opacity theorem, they are all equivalent from the observer's perspective, but their total count bounds the trigger's internal computational freedom.

Optimal virtual state selection: Among all virtual sub-state decompositions achieving mean-recovery, which minimise the Fisher information along the backward path? This would give an optimal backward navigation algorithm rather than the generic one of Algorithm 1.

Cascade catalysis and trigger iterations: The multiplicative catalyst algebra (proved in Sachikonye (a)) says catalyst power combines as $\kappa(\gamma_1 \diamond \gamma_2) = 1 - (1 - \kappa_1)(1 - \kappa_2)$. A trigger menu of K trigger paths is a catalyst cascade; the total classification power is $1 - \prod_i (1 - \kappa_i)$. Saturating the cascade ($\kappa \rightarrow 1$) requires $\sum_i \kappa_i = \infty$, which occurs when the trigger menu's composition depth reaches n_P .

Acknowledgements

This work was supported by the AIMe Registry for Artificial Intelligence.

References

- Saunders Mac Lane. *Categories for the Working Mathematician*. Springer-Verlag, 2nd edition, 1998.
- Kundai Farai Sachikonye. Cascade catalysts and renormalization as saturation. Companion paper, this series, a.

Chapter 4. Paper 6. Backward Trajectory Completion in Bounded Phase Space: S-Entropy Framework, Virtual Sub-States, and the LHC Trigger as a Categorical Navigator
Kundai Farai Sachikonye. On the partition algebra of
bounded phase space. Companion paper, this series,
b.

Kundai Farai Sachikonye. The planck depth and optimal
trigger design. Companion paper, this series, c.

Kundai Farai Sachikonye. An equivalence calculus of
unconstrained subtask recursion. Companion paper,
this series, d.

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1,
Paper 7. Fundamental Timing in Bounded Phase Space: The 25 ns Bunch Crossing from
LHC Geometry, Structural Incorruptibility, and Timing-Only Triggers, paper:timing

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1,
Paper 8. Virtual Particles as Virtual Sub-States in QFT: Feynman Diagrams as Back-
ward Trajectory Completions and the Optical Theorem from Partition Conservation,
paper:virtual

Part IV

**Algebraic Structure and
Consequences**

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1,
Paper 9. Cascade Catalysts and Renormalisation as Saturation: Multiplicative Catalyst
Algebra, Trigger Menus, and the Partition Renormalisation Group, paper:cascade

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter,
1, Paper 10. Cosmological Consequences of Bounded Phase Space: Poincaré Recur-
rence, Partition Pressure of the Cosmos, and the Partition Structure of the Universe,
paper:cosmo

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1,
Paper 11. Experimental Validation of the Partition Framework: Mass Spectrometry at
keV, LHC at TeV, and Cross-Scale Consistency from 10^3 eV to 10^{12} eV, paper:validation

Part V

The Operating System Framework

vaHera and Partition-Coordinate Refinement Types: A Type System for Symbolic Physical Derivations and Selection-Rule Enforcement

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIME Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

We introduce a refinement-type system over the vaHera abstract syntax tree (AST) in which partition coordinates (n, ℓ, m, s) and the selection rules $\Delta\ell = \pm 1$, $\Delta m \in \{0, \pm 1\}$, $\Delta s = 0$ are first-class type-theoretic predicates rather than runtime constraints. The vaHera AST consists of four frozen variants—Literal, Call, Compose, and Hole—and provides the symbolic substrate over which physical derivations expressing transitions between partition labels are written. We prove three principal results. (i) The *Compositionality Lemma*: refinement types compose under Compose via predicate conjunction, with no loss of decidability. (ii) The *Forbidden-Transition Theorem*: a vaHera fragment whose Compose chain implies a transition violating any registered selection rule is rejected by typecheck at static time; the rejection is structural, not probabilistic, and requires no run-time observation. (iii) The *Decidability Theorem*: refinement-type checking under a finite selection-rule set is decidable in $\mathcal{O}(|F| \cdot |\sigma|)$ where $|F|$ is the syntactic size of the fragment and $|\sigma|$ is the rule-set cardinality, and the checker is a total function. We provide a complete inference-rule system, soundness and completeness proofs, and a worked example illustrating the rejection of a fragment encoding an $\ell = 0 \rightarrow \ell = 2$ transition. The framework is purely syntactic: every theorem is provable from the four vaHera variants, the partition-label invariants, and the algebra of selection-rule predicates, without recourse to physical postulates

or runtime evaluation. We outline how the type system serves as the formal substrate for physical-derivation languages that compile to vaHera, including the temporal-programming Tempus calculus, partition-mechanics expressions over bounded Hamiltonian systems, and detector-classification predicates in real-time event-selection systems.

Keywords: refinement types, partition coordinates, selection rules, vaHera, symbolic computation, static verification, type-directed rejection, compositionality, physical derivation, categorical type theory.

Contents

I	Preliminaries	2
1	Introduction	2
1.1	Motivation	2
1.2	The vaHera substrate	3
1.3	Selection rules as type predicates	3
1.4	Overview of main results	3
2	The vaHera AST	4
2.1	Syntax	4
2.2	Values and base types	4
2.3	Operations	4
2.4	Standard typing judgement	4
II	Partition Coordinates as Refinement Base Types	4

3	Partition Labels	4	9.5	Three-Route Verification	11
3.1	Definition	4			
3.2	Label cardinality	5			
3.3	Label projection	5	VII	Discussion	11
4	Refinement Types over Partition Labels	5	10	Comparison with Existing Refinement-Type Systems	11
4.1	Refinement formation	5	10.1	Liquid Types	11
4.2	Selection rules	6	10.2	Refinement Types for Haskell . . .	11
4.3	Operations with refinement	6	10.3	ML-Style Refinements	11
			10.4	Dependent Types	12
			10.5	Hoare Logic	12
III	Refinement Typing	6	11	Limitations and Open Questions	12
5	Refined Typing Judgement	6	11.1	Predicate language	12
5.1	Definition	6	11.2	Hole substitution	12
5.2	The Compositionality Lemma . . .	6	11.3	Operation registration	12
5.3	The selection-rule refinement convention	7	11.4	Selection-rule learning	12
6	Soundness and Completeness	7	12	Conclusion	12
6.1	Operational semantics	7			
6.2	Soundness	7			
6.3	Completeness	7			
6.4	The Forbidden-Transition Theorem	8			
IV	Decidability and Algorithms	8	Part I		
7	Decidability	8	Preliminaries		
7.1	Complexity of typecheck	8	1	Introduction	
7.2	The typecheck algorithm	9	1.1	Motivation	
7.3	The role of the bijection Φ	9			
V	Worked Example	9			
8	The $\ell = 0 \rightarrow \ell = 2$ Rejection	9			
8.1	Setup	9			
8.2	Direct composition: rejected	10			
8.3	Through-route composition: also rejected	10			
8.4	Significance	10			
VI	Connections to Applications	10			
9	Applications across the Symbolic-Physics Substrate	10			
9.1	Bounded Hamiltonian Systems . .	10			
9.2	Mass-Spectrometric Trajectories .	10			
9.3	Temporal Programming and Cell Registries	11			
9.4	Real-Time Event-Selection Systems	11			

1 Introduction

1.1 Motivation

Symbolic representations of physical derivations are everywhere: Feynman diagrams encode interaction amplitudes; partition-state transition graphs encode bounded Hamiltonian dynamics; cell-registry programs encode real-time event-selection logic; and ion-trajectory equations encode mass-spectrometric measurement. In every case, the symbolic representation must obey constraints that arise from the underlying categorical structure of partition coordinates—in particular, from the *selection rules* that determine which transitions between partition labels are admissible. Without a type-level mechanism for enforcing such rules, the symbolic substrate admits expressions that have no physical interpretation and that, if executed, produce well-formed but operationally meaningless results.

The standard response to this problem is to detect ill-formed expressions at runtime: the dispatch path evaluates a fragment and rejects results that fail to satisfy the relevant constraints. This response is unsatisfactory in three ways.

First, it requires actual execution before rejection, which is prohibitively expensive for high-throughput dispatch substrates such as kernels driving real-time detector systems. Second, it conflates two distinct categories of failure—those arising from runtime conditions (resource exhaustion, network unavailability) and those arising from structural ill-formedness—producing diagnostics that cannot be attributed to the right cause. Third, and most fundamentally, it forfeits the structural certainty available at compile time: a fragment whose composition chain implies a forbidden transition is ill-formed as a syntactic object, not merely as a runtime computation.

This paper proposes that the right response is to lift selection-rule enforcement into the type system. We define a refinement-type discipline over the vaHera AST in which partition labels are types and selection rules are refinement predicates. A vaHera fragment is well-typed if and only if every Compose-stage transition respects every registered selection rule; type-checking is decidable in linear time in the fragment size; and the typing relation is structurally compositional.

1.2 The vaHera substrate

The vaHera AST is a four-variant inductive datatype originally introduced as the symbolic interchange format between resolvers and providers in the Purpose framework (Sachikonye, 2026c). The four variants are:

- $\text{Literal}(v)$, the embedding of a runtime value;
- $\text{Call}(\mathcal{O}, \bar{F})$, the invocation of a named operation $\mathcal{O}$ with argument fragments;
- $\text{Compose}(\bar{F})$, the sequential composition of fragments with carry threading;
- $\text{Hole}(x)$, a typed placeholder awaiting substitution.

The variants are frozen: no version of the framework adds, removes, or renames them, and downstream consumers may rely on their permanence (per the stability contract in Sachikonye, 2026c §2.1). This stability is what permits a refinement-type system to be defined once and to apply uniformly across all clients of the AST.

1.3 Selection rules as type predicates

The central technical move of this paper is to interpret selection rules—binary predicates on pairs of partition labels asserting that a given transition is admissible—as refinement predicates

over partition-coordinate types. In standard refinement-type discipline (Rondon et al., 2008; Vazou et al., 2014; Freeman and Pfenning, 1991), a refinement type $\{x : B \mid \phi(x)\}$ is the subset of values of base type B satisfying predicate ϕ . Our generalisation is mild: we replace the single-value predicate $\phi(x)$ with a transition predicate $\sigma(x_1, x_2)$ relating the input and output labels of an operation. This allows a single operation type signature

$$\mathcal{O} : \{x_1 : \mathcal{L} \mid \text{true}\} \rightarrow \{x_2 : \mathcal{L} \mid \sigma(x_1, x_2)\}$$

to express *any transition admissible under σ* , with σ ranging over an arbitrary finite set of selection rules. Composition then accumulates the conjunction of the per-stage transition predicates, and the Compositionality Lemma (Theorem 5.2) ensures the accumulated predicate is itself decidable.

1.4 Overview of main results

The paper establishes the following:

1. **Theorem 3.3** (Label Formation): the four-tuple partition labels (n, ℓ, m, s) form a well-defined inductive type with closed-form bounds.
2. **Theorem 4.3** (Refinement Type Formation): refinements $\{x : B \mid \phi(x)\}$ over partition-label base types are well-formed and yield decidable subsets.
3. **Theorem 5.2** (Compositionality Lemma): for any $\text{Compose}(\bar{F})$, the refinement type of the composite equals the conjunction of per-stage refinement types under carry-threading semantics.
4. **Theorem 6.1** (Soundness of typecheck): every fragment accepted by typecheck produces, on execution, a Value whose label satisfies every registered selection rule.
5. **Theorem 6.2** (Completeness of typecheck): every fragment whose composition chain implies a transition violating any registered selection rule is rejected by typecheck.
6. **Theorem 6.3** (Forbidden-Transition Theorem): the principal consequence: typecheck rejection of selection-rule-violating fragments is structural, not probabilistic, and occurs at static time with no runtime observation required.
7. **Theorem 7.1** (Decidability): refinement-type checking under finite σ is decidable in $\mathcal{O}(|F| \cdot |\sigma|)$.

2 The vaHera AST

2.1 Syntax

Definition 2.1 (vaHera AST). *The vaHera AST is the inductive datatype F defined by the grammar*

$$F ::= \text{Literal}(v) \mid \text{Call}(\mathcal{O}, \overline{F}) \mid \text{Compose}(\overline{F}) \mid \text{Hole}(x)$$

where v ranges over runtime Values, $\mathcal{O}$ over operation names, $\overline{F}$ over finite sequences of fragments (the Call case carries a named-argument map, and the Compose case carries an ordered list), and x over identifiers.

Remark 2.2. *The four variants are pairwise disjoint and exhaustive over the AST. Every well-formed fragment is, syntactically, exactly one of these four shapes. This is the frozen surface from Sachikonye (2026c) §2.1.*

2.2 Values and base types

Definition 2.3 (Value). *The runtime Value space V is the inductive datatype*

$$v ::= \text{Null} \mid \text{Bool}(b) \mid \text{Num}(r) \mid \text{Str}(s) \\ \mid \text{List}(\overline{v}) \mid \text{Record}(\overline{(k, v)})$$

where $b \in \mathbb{B}$, $r \in \mathbb{R}$, s is a finite character string, $\overline{v}$ is a finite sequence of Values, and $\overline{(k, v)}$ is a finite map from string keys to Values.

Definition 2.4 (Base type). *A base type B is one of $\{\text{Str}, \text{Num}, \text{Bool}, \text{Unit}\}$ or a parameterised constructor $\text{List}(B')$, a Named type $\text{Named}(\tau)$ for a type name τ , or a type variable $\text{Var}(\alpha)$. The set of base types is denoted $\mathcal{B}$.*

The base-type discipline is the standard Hindley-Milner-style typing layer (Hindley, 1969; Milner, 1978; Damas and Milner, 1982). Every Value has a most-general base type, and the typing judgement $\vdash v : B$ is decidable in linear time.

2.3 Operations

Definition 2.5 (Operation). *An operation $\mathcal{O}$ is a record*

$$\mathcal{O} = \langle \text{name}, \text{inputs}, \text{output}, \text{desc} \rangle$$

where *name* is a string identifier, *inputs* is a finite map from named argument keys to base

types, *output* is a base type, and *desc* is a human-readable description. The set of operations registered in a given context is the operation registry $\mathcal{R}$.

Remark 2.6. *The desc field carries no semantic content for the type system. We retain it for compatibility with the runtime substrate, which uses it for introspection.*

2.4 Standard typing judgement

The standard (non-refinement) typing judgement $\Gamma \vdash F : B$ asserts that under typing context Γ , the fragment F has base type B .

Definition 2.7 (Standard typing). *The typing relation $\Gamma \vdash F : B$ is the least relation closed under the rules:*

$$\frac{\vdash v : B}{\Gamma \vdash \text{Literal}(v) : B} (T\text{-LIT}) \quad \frac{\Gamma(x) = B}{\Gamma \vdash \text{Hole}(x) : B} (T\text{-HOL})$$

$$\frac{\mathcal{O} = \langle _, \overline{(k_i, B_i)}, B, _ \rangle \in \mathcal{R} \quad \forall i : \Gamma \vdash F_i : B_i}{\Gamma \vdash \text{Call}(\mathcal{O}, \overline{(k_i, F_i)}) : B} (T\text{-CALL})$$

$$\frac{\Gamma \vdash F_1 : B_1 \quad \dots \quad \Gamma \vdash F_n : B_n \quad \forall i \geq 2 : B_{i-1} \succeq \text{in}_i}{\Gamma \vdash \text{Compose}(\overline{F}) : B_n} (T\text{-COM})$$

where in_i is the carry-input type of stage i (the type of the implicit **input** argument), and $\succeq$ is the standard subtyping relation.

Proposition 2.8 (Standard typing is decidable). *For every Γ and every well-formed F , the judgement $\Gamma \vdash F : B$ is decidable in linear time in $|F|$.*

Proof. Straightforward structural induction over F : each rule has a unique conclusion shape, premises decrease in fragment size, and base-type subtyping is decidable in constant time per check. $\square$ $\square$

The standard typing judgement is the foundation on which we layer refinement.

Part II

Partition Coordinates as Refinement Base Types

3 Partition Labels

3.1 Definition

Definition 3.1 (Partition Label). *A partition label is a 4-tuple*

$$\lambda = (n, \ell, m, s) \in \mathbb{N} \times \mathbb{N} \times \mathbb{Z} \times \{-\frac{1}{2}, +\frac{1}{2}\}$$

subject to the constraints

$$n \geq 1, \quad (1)$$

$$0 \leq \ell \leq n - 1, \quad (2)$$

$$-\ell \leq m \leq +\ell. \quad (3)$$

The set of all partition labels is denoted $\mathcal{L}$.

Remark 3.2. *The constraints (1)–(3) are the nested boundary conditions of three-dimensional bounded oscillatory systems, derived from first principles in Sachikonye (2026a). We adopt them here as axiomatic for the type system; the question of how they arise physically is orthogonal to the present development.*

3.2 Label cardinality

Theorem 3.3 (Label Formation). *The set $\mathcal{L}$ of partition labels is well-defined and admits the following cardinality structure:*

1. *For each $n \geq 1$, the number of labels with that principal value is $C(n) = 2n^2$.*
2. *The cumulative count of labels with principal value at most $n_{\max}$ is $N(n_{\max}) = n_{\max}(n_{\max} + 1)(2n_{\max} + 1)/3$.*
3. *There exists a canonical bijection $\Phi : \mathbb{N}^+ \rightarrow \mathcal{L}$ assigning each positive integer to a unique label.*

Proof. (i) At principal level n , ℓ ranges over $\{0, 1, \dots, n - 1\}$ (by (2)); for each ℓ , m ranges over $2\ell + 1$ values (by (3)); and s contributes a factor of 2. Summing:

$$C(n) = 2 \sum_{\ell=0}^{n-1} (2\ell + 1) = 2(n^2 - n) + 2n = 2n^2.$$

(ii) Applying $\sum_{n=1}^N n^2 = N(N + 1)(2N + 1)/6$ with the factor of 2 from (i):

$$N(n_{\max}) = \sum_{n=1}^{n_{\max}} 2n^2 = \frac{n_{\max}(n_{\max} + 1)(2n_{\max} + 1)}{3}.$$

(iii) Define $\Phi(M)$ by determining $n = \lfloor \sqrt{M/2} \rfloor + 1$ via inversion of (ii), then enumerating (ℓ, m, s) lexicographically within level n . The construction is total and injective; surjectivity follows from $\bigsqcup_n \{(n, \ell, m, s) : 0 \leq \ell < n, |m| \leq \ell, s \in \{\pm 1/2\}\} = \mathcal{L}$. $\square$

Remark 3.4. *Theorem 3.3 (iii) gives an effective procedure for label enumeration: every positive integer indexes a unique label, and every label has a unique integer index. We will exploit this bijection in Section 7 to obtain a constant-time membership test.*

3.3 Label projection

For a label $\lambda = (n, \ell, m, s)$, we write $\pi_n(\lambda) = n$, $\pi_\ell(\lambda) = \ell$, $\pi_m(\lambda) = m$, $\pi_s(\lambda) = s$ for the four canonical projections, and $\Delta : \mathcal{L} \times \mathcal{L} \rightarrow \mathbb{Z}^4$ for the componentwise difference $\Delta(\lambda_1, \lambda_2) = (n_2 - n_1, \ell_2 - \ell_1, m_2 - m_1, s_2 - s_1)$.

4 Refinement Types over Partition Labels

4.1 Refinement formation

Definition 4.1 (Refinement Type). *A refinement type over base type $B \in \mathcal{B}$ with predicate ϕ is the pair*

$$\{x : B \mid \phi(x)\}$$

where ϕ is a computable Boolean-valued function on values of base type B . The set of refinement types is denoted $\mathcal{R}$.

The base-type refinements in standard literature (Rondon et al., 2008; Vazou et al., 2014) are single-value predicates of the form $\phi(x)$. Our development requires a generalisation: predicates that relate *pairs* of labels, the input and output of a transition.

Definition 4.2 (Transition Refinement). *A transition refinement is a refinement of the form*

$$\{x_2 : B_2 \mid \sigma(x_1, x_2)\}$$

where x_1 is the input value of a transition and x_2 is the output value, and $\sigma : \mathcal{L} \times \mathcal{L} \rightarrow \mathbb{B}$ is a binary predicate.

Theorem 4.3 (Refinement Type Formation). *For every base type $B \in \mathcal{B}$ and every computable predicate ϕ , the refinement type $\{x : B \mid \phi(x)\}$ is well-formed. Membership in $\{x : B \mid \phi(x)\}$ is decidable in time $\mathcal{O}(T_\phi)$, where T_ϕ is the cost of evaluating ϕ on a single value.*

Proof. Membership: $v \in \{x : B \mid \phi(x)\}$ iff $\vdash v : B$ and $\phi(v) = \text{true}$. The first conjunct is decidable in linear time by Proposition 2.8; the second conjunct is decidable in $\mathcal{O}(T_\phi)$ by hypothesis. The conjunction is therefore decidable in $\mathcal{O}(|v| + T_\phi)$. Well-formedness follows from $B \in \mathcal{B}$ being a base type and ϕ being computable. $\square$ $\square$

4.2 Selection rules

Definition 4.4 (Selection Rule). *A selection rule is a predicate $\sigma \in \mathcal{L} \times \mathcal{L} \rightarrow \mathbb{B}$ asserting that a transition from λ_1 to λ_2 is admissible (true) or forbidden (false).*

Example 4.5 (Standard physical selection rules). *The following are commonly used:*

- *Electric dipole (E1):* $\sigma_{E1}(\lambda_1, \lambda_2) \iff \pi_\ell(\lambda_2) - \pi_\ell(\lambda_1) \in \{-1, +1\} \text{ and } \pi_m(\lambda_2) - \pi_m(\lambda_1) \in \{-1, 0, +1\}.$
- *Magnetic dipole (M1):* $\sigma_{M1}(\lambda_1, \lambda_2) \iff \pi_\ell(\lambda_2) = \pi_\ell(\lambda_1) \text{ and } \pi_m(\lambda_2) - \pi_m(\lambda_1) \in \{-1, 0, +1\}.$
- *Spin conservation:* $\sigma_{\text{spin}}(\lambda_1, \lambda_2) \iff \pi_s(\lambda_2) = \pi_s(\lambda_1).$
- *Principal advancement:* $\sigma_{\Delta n}(\lambda_1, \lambda_2) \iff \pi_n(\lambda_2) \geq \pi_n(\lambda_1).$

We emphasise that the type system is parametric in σ : any computable binary predicate yields a selection rule, and the framework imposes no preference among rule sets.

Definition 4.6 (Selection-Rule Registry). *A selection-rule registry is a finite set $\sigma_\star = \{\sigma_1, \dots, \sigma_k\}$ of selection rules. A transition (λ_1, λ_2) is admissible under $\sigma_\star$ iff $\bigwedge_{i=1}^k \sigma_i(\lambda_1, \lambda_2)$.*

The registry is the unit of policy: changing the registered rules changes which transitions are admissible. For the remainder of the paper, we fix an arbitrary $\sigma_\star$.

4.3 Operations with refinement

Definition 4.7 (Refined Operation). *A refined operation extends Definition 2.5 with input and output refinements:*

$$\mathcal{O}^{\mathcal{R}} = \langle \text{name}, \overline{(k_i, B_i, \phi_i^{\text{in}})}, B, \phi^{\text{out}}, \text{desc} \rangle$$

where ϕ_i^{in} is the refinement predicate on the i -th input and ϕ^{out} is the refinement predicate on the output. The set of refined operations is $\mathcal{R}^+$.

The output refinement may depend on the input values; in the partition-label setting, $\phi^{\text{out}}(\lambda_2) = \sigma_\star(\lambda_1^{\text{in}}, \lambda_2)$ for a designated input λ_1^{in} carrying the source label.

Part III

Refinement Typing

5 Refined Typing Judgement

5.1 Definition

The refined typing judgement $\Gamma \vdash F : \{x : B \mid \phi(x)\}$ extends Definition 2.7.

Definition 5.1 (Refined typing). *The refined typing relation is closed under the rules*

$$\frac{\vdash v : B \quad \phi(v) = \text{true}}{\Gamma \vdash \text{Literal}(v) : \{x : B \mid \phi(x)\}} (R\text{-LIT})$$

$$\frac{\Gamma(x) = \{y : B \mid \phi(y)\}}{\Gamma \vdash \text{Hole}(x) : \{y : B \mid \phi(y)\}} (R\text{-HOL})$$

$$\frac{\mathcal{O}^{\mathcal{R}} \in \mathcal{R}^+ \quad \forall i : \Gamma \vdash F_i : \{y_i : B_i \mid \phi_i^{\text{in}}(y_i)\}}{\Gamma \vdash \text{Call}(\mathcal{O}^{\mathcal{R}}, \overline{(k_i, F_i)}) : \{x : B \mid \phi^{\text{out}}(x, \overline{y_i})\}} (R\text{-CALL})$$

$$\frac{\Gamma \vdash F_1 : \{y_1 : B_1 \mid \phi_1\} \quad \dots \quad \Gamma \vdash F_n : \{y_n : B_n \mid \phi_n\}}{\Gamma \vdash \text{Compose}(\overline{F}) : \{x : B_n \mid \bigwedge_{i=1}^n \phi_i \# \text{thread}\}} (R\text{-COM})$$

where $\# \text{thread}$ denotes that the carry value of stage i is the input value y_{i+1}^{in} of stage $i + 1$.

The carry-threading premise of (R-COMP) is what makes the rule non-trivial: the output refinement of one stage is fed into the input refinement of the next, accumulating constraints as the composition chain grows.

5.2 The Compositionality Lemma

Theorem 5.2 (Compositionality Lemma). *Let $F = \text{Compose}(F_1, F_2, \dots, F_n)$ be a Compose chain. For all i , let*

$$\Gamma \vdash F_i : \{y_i : B_i \mid \phi_i(y_{i-1}, y_i)\}$$

under the carry-threading semantics, with y_0 a designated initial value. Then

$$\Gamma \vdash F : \left\{ x : B_n \mid \bigwedge_{i=1}^n \phi_i(y_{i-1}, y_i) \right\}.$$

Proof. By structural induction on n .

Base case ($n = 1$): $F = \text{Compose}(F_1)$. The hypothesis gives $\Gamma \vdash F_1 : \{y_1 : B_1 \mid \phi_1(y_0, y_1)\}$, and (R-COMP) with $n = 1$ reduces to the same judgement (the conjunction over a single conjunct is the conjunct itself). The result holds.

Inductive case ($n = k + 1$, assuming the result for $n = k$): Let $F^{(k)} = \text{Compose}(F_1, \dots, F_k)$ and $F = \text{Compose}(F^{(k)}, F_{k+1})$. By the inductive hypothesis,

$$\Gamma \vdash F^{(k)} : \{y_k : B_k \mid \Psi_k(y_0, y_1, \dots, y_k)\}$$

where $\Psi_k = \bigwedge_{i=1}^k \phi_i$. By (R-COMP) applied to the binary case (two-stage compose),

$$\Gamma \vdash \text{Compose}(F^{(k)}, F_{k+1}) : \{x : B_{k+1} \mid \Psi_k \wedge \phi_{k+1}(y_k, y_{k+1})\} \text{ and } \phi(v) = \text{true}.$$

Since $\Psi_k \wedge \phi_{k+1} = \bigwedge_{i=1}^{k+1} \phi_i$, the inductive step holds.

By induction, the result holds for all $n \geq 1$. $\square$

Corollary 5.3 (Conjunction-Preservation). *The refinement type of a Compose chain is the conjunction of the per-stage refinements modulo carry-threading. In particular, if any single ϕ_i is unsatisfiable, the type of the composite is uninhabited.*

Proof. Direct from Theorem 5.2: if $\phi_i \equiv \text{false}$, then $\bigwedge_j \phi_j \equiv \text{false}$, and no value satisfies false. $\square$

5.3 The selection-rule refinement convention

For the remainder of the paper, we work under the *selection-rule refinement convention*: every refined operation $\mathcal{O}^R$ whose input and output are partition-label types has output refinement

$$\phi^{\text{out}}(\lambda_2, \lambda_1) = \bigwedge_{\sigma \in \sigma_*} \sigma(\lambda_1, \lambda_2),$$

i.e. the operation's output is well-typed iff the implied transition respects every registered selection rule. The convention is a notational specialisation; the underlying type system is unchanged.

6 Soundness and Completeness

6.1 Operational semantics

To state soundness, we recall the standard operational semantics of vaHera (Sachikonye, 2026c). The dispatch evaluation $\text{eval}(F) \in V \cup \{\perp\}$ is defined by structural recursion:

$$\text{eval}(\text{Literal}(v)) = v$$

$$\text{eval}(\text{Call}(\mathcal{O}, \overline{(k_i, F_i)})) = \mathcal{O}(\overline{(k_i, \text{eval}(F_i))})$$

$$\text{eval}(\text{Compose}(F_1, \dots, F_n)) = \text{thread}(\text{eval}(F_1), \dots, \text{eval}(F_n))$$

$$\text{eval}(\text{Hole}(x)) = \perp$$

where thread pipes the output of each stage into the implicit input of the next, and $\perp$ denotes the failure to dispatch (unresolved Hole).

6.2 Soundness

Theorem 6.1 (Soundness of typecheck). *If $\Gamma \vdash F : \{x : B \mid \phi(x)\}$ and $\text{eval}(F) = v \neq \perp$, then*

$$\phi(v) = \text{true}.$$

Proof. By induction on the derivation of $\Gamma \vdash F : \{x : B \mid \phi(x)\}$.

Case (R-Lit): $F = \text{Literal}(v)$. The premises give $\vdash v : B$ and $\phi(v) = \text{true}$. The conclusion follows by $\text{eval}(\text{Literal}(v)) = v$.

Case (R-Hol): $F = \text{Hole}(x)$. Then $\text{eval}(F) = \perp$, contradicting the hypothesis $\text{eval}(F) \neq \perp$. The case is vacuous.

Case (R-Call): $F = \text{Call}(\mathcal{O}^R, \overline{(k_i, F_i)})$. By the inductive hypothesis, each $\text{eval}(F_i)$ is a value of type B_i satisfying ϕ_i^{in} . The operation $\mathcal{O}^R$ is, by Definition 4.7, refined: its output, computed as $\mathcal{O}(\overline{(k_i, \text{eval}(F_i))})$, has type B and satisfies ϕ^{out} . The conclusion follows.

Case (R-Comp): $F = \text{Compose}(F_1, \dots, F_n)$. By Theorem 5.2, the type of F is $\{x : B_n \mid \bigwedge_i \phi_i\}$. By the inductive hypothesis applied to each F_i , each intermediate value $v_i = \text{eval}(F_i)$ has type B_i and satisfies ϕ_i . The final value $v_n = \text{thread}(v_1, \dots, v_n)$ inherits type B_n from the last stage and satisfies $\bigwedge_i \phi_i$ by the threading invariant.

The induction is complete. $\square$ $\square$

6.3 Completeness

Theorem 6.2 (Completeness of typecheck). *If F is a Compose chain whose thread semantics implies a transition $(\lambda_i, \lambda_{i+1})$ for some i that violates some registered selection rule $\sigma \in \sigma_*$, then $\text{typecheck}(F)$ fails: there is no derivation $\Gamma \vdash F : T$ for any refinement type T under the selection-rule refinement convention.*

Proof. Suppose, for contradiction, that $\Gamma \vdash F : \{x : B \mid \phi(x)\}$ for some ϕ . By Theorem 5.2, $\phi = \bigwedge_{j=1}^n \phi_j$ where ϕ_j is the refinement of stage j in the Compose chain.

By the selection-rule refinement convention, stage i (the stage whose transition is forbidden) has refinement $\phi_i(\lambda_i, \lambda_{i+1}) = \bigwedge_{\sigma \in \sigma_*} \sigma(\lambda_i, \lambda_{i+1})$. By hypothesis, some $\sigma(\lambda_i, \lambda_{i+1}) = \text{false}$. Hence $\phi_i = \text{false}$ at the relevant transition.

By Corollary 5.3, the conjunction $\bigwedge_j \phi_j$ is unsatisfiable; the type $\{x : B \mid \bigwedge_j \phi_j\}$ is uninhabited.

By Soundness (Theorem 6.1), an inhabited refinement type would be required for a successful eval producing a non- $\perp$ value. The uninhabited type forces $\text{eval}(F) = \perp$ (the fragment has no well-typed evaluation). But this contradicts the assumption that typecheck accepted F —typecheck requires inhabitation. $\square$ $\square$

6.4 The Forbidden-Transition Theorem

We can now state the principal consequence.

Theorem 6.3 (Forbidden-Transition Theorem). *Under the selection-rule refinement convention with registry σ_* :*

1. *Every vaHera Compose chain F whose implied transitions all respect σ_* is accepted by typecheck (modulo standard typing requirements).*
2. *Every vaHera Compose chain F that implies a transition violating any $\sigma \in \sigma_*$ is rejected by typecheck at static time, with no runtime observation required.*

The acceptance/rejection is a decidable function of the syntactic structure of F and the predicates in σ_ .*

Proof. Combine Theorems 6.1 and 6.2: soundness gives (i), completeness gives (ii). Decidability of the joint judgement follows from each σ being computable and the Compose chain being finite. $\square$ $\square$

Remark 6.4 (Rejection is structural). *Theorem 6.3 (ii) is the key claim of this paper. It says that selection-rule-violating fragments are rejected for a structural reason—the refinement type is uninhabited—rather than because of a probabilistic event at runtime. The rejection occurs before dispatch, requires no execution, and is reproducible across implementations.*

Part IV

Decidability and Algorithms

7 Decidability

7.1 Complexity of typecheck

Theorem 7.1 (Decidability of typecheck). *Refinement-type checking of a vaHera fragment F under selection-rule registry σ_* runs in time $\mathcal{O}(|F| \cdot |\sigma_*|)$, where $|F|$ is the syntactic size of F and $|\sigma_*|$ is the cardinality of σ_* , assuming constant-time evaluation of each $\sigma \in \sigma_*$.*

Proof. We give a constructive algorithm.

Algorithm. Walk F once in pre-order. For each Compose node, examine the per-stage refinements pairwise: for the transition from stage i to stage $i + 1$, evaluate each $\sigma \in \sigma_*$ on the implied $(\lambda_i, \lambda_{i+1})$ pair. Reject if any σ returns false; else continue. For each Call node, validate the operation signature against the standard typing premises (Proposition 2.8, linear time). For Literal and Hole, return immediately.

Complexity. Walking F visits each AST node exactly once. At each Compose node with k stages, the per-stage refinement check costs $\mathcal{O}(|\sigma_*|)$ per transition for $k - 1$ transitions, totalling $\mathcal{O}(k \cdot |\sigma_*|)$. Summing over all Compose nodes gives $\mathcal{O}(N_C \cdot |\sigma_*|)$ where N_C is the total stage count, which is bounded by $|F|$. The standard typing component adds $\mathcal{O}(|F|)$. Total: $\mathcal{O}(|F| \cdot |\sigma_*|)$.

Termination. Pre-order walk is structurally recursive on a finite AST, hence terminating. $\square$ $\square$

Remark 7.2. *The constant-time assumption on $\sigma \in \sigma_*$ holds for all selection rules in Example 4.5: they are pure arithmetic comparisons on partition labels with bounded data. For more elaborate σ*

involving, say, table lookups, the bound generalises to $\mathcal{O}(|F| \cdot |\sigma_\star| \cdot T_{\max})$ where $T_{\max}$ is the maximum per-rule evaluation cost.

7.2 The typecheck algorithm

Algorithm 1 Refinement-type check

```

1: function CHECK( $F, \Gamma, \Sigma_\star$ )
2:   if  $F$  is a literal  $\text{Literal}(v)$  then
3:     return  $\vdash v : B$  for some base type  $B$ 
4:   else if  $F$  is a hole  $\text{Hole}(x)$  then
5:     return  $x \in \text{dom}(\Gamma)$ 
6:   else if  $F$  is a call  $\text{Call}(\Omega, \text{args})$  then
7:     require  $\Omega \in \mathcal{R}^+$ 
8:     for each named argument  $\text{args}[k_i] = F_i$  do
9:        $r_i \leftarrow \text{CHECK}(F_i, \Gamma, \Sigma_\star)$ 
10:      if  $r_i$  is false then
11:        return false
12:      end if
13:    end for
14:    return true
15:   else if  $F$  is a compose  $\text{Compose}(F_1, \dots, F_n)$  then
16:      $\lambda_{\text{prev}} \leftarrow \text{init}$ 
17:     for  $i = 1$  to  $n$  do
18:        $r_i \leftarrow \text{CHECK}(F_i, \Gamma, \Sigma_\star)$ 
19:       if  $r_i$  is false then
20:         return false
21:       end if
22:        $\lambda_i \leftarrow \text{labelOf}(F_i)$ 
23:       if  $i > 1$  then
24:         for each  $\sigma \in \Sigma_\star$  do
25:           if  $\sigma(\lambda_{\text{prev}}, \lambda_i)$  is false then
26:             return false
27:           end if
28:         end for
29:       end if
30:        $\lambda_{\text{prev}} \leftarrow \lambda_i$ 
31:     end for
32:     return true
33:   end if
34: end function

```

Proposition 7.3 (Algorithm correctness). *Algorithm 1 returns true iff there is a refinement-typing derivation for F under Γ and $\sigma_\star$.*

Proof. The algorithm directly mirrors the inference rules of Definition 5.1: each branch corresponds to one rule, the recursion follows the

structural premises, and the per-stage refinement check at Compose nodes implements the conjunction of Theorem 5.2. Correctness then follows from Theorems 6.1 and 6.2. $\square$ $\square$

7.3 The role of the bijection Φ

Proposition 7.4 (Label representation cost). *Under the bijection $\Phi : \mathbb{N}^+ \rightarrow \mathcal{L}$ from Theorem 3.3, each partition label is representable as a single positive integer; label comparison, hashing, and ordering are constant-time operations.*

Proof. By Theorem 3.3, Φ is a constructive bijection. Comparison and hashing of integers are standard $\mathcal{O}(1)$ operations on bounded-precision arithmetic. Ordering is the natural integer order. $\square$ $\square$

Corollary 7.5 (Selection-rule evaluation is constant-time). *Under the bijection Φ , each of the selection rules in Example 4.5 evaluates in $\mathcal{O}(1)$ time on a transition (λ_1, λ_2) .*

Proof. Each rule reduces to integer subtraction and bounded-range membership tests (e.g. “is $\ell_2 - \ell_1 \in \{-1, +1\}$ ”), both $\mathcal{O}(1)$. $\square$ $\square$

Together, Theorem 7.1, Proposition 7.3, and Corollary 7.5 give us a complete static-time enforcement mechanism: any vaHera fragment whose Compose chain implies a forbidden transition is rejected at compile time, in linear time in the fragment size.

Part V

Worked Example

8 The $\ell = 0 \rightarrow \ell = 2$ Rejection

We illustrate Theorem 6.3 on a small but representative example.

8.1 Setup

Fix the selection-rule registry $\sigma_\star = \{\sigma_{\text{E1}}, \sigma_{\text{spin}}\}$ from Example 4.5. We register two operations:

$$\begin{aligned} \mathcal{O}_1 : \{y : \mathcal{L} \mid y = (n_1, 0, 0, +\tfrac{1}{2})\} \rightarrow \\ \{x : \mathcal{L} \mid \sigma_{\text{E1}}((n_1, 0, 0, +\tfrac{1}{2}), x) \\ \wedge \sigma_{\text{spin}}((n_1, 0, 0, +\tfrac{1}{2}), x)\} \\ \mathcal{O}_2 : \{y : \mathcal{L} \mid y = (n_2, 2, 0, +\tfrac{1}{2})\} \rightarrow \\ \{x : \mathcal{L} \mid \sigma_{\text{E1}}((n_2, 2, 0, +\tfrac{1}{2}), x) \\ \wedge \sigma_{\text{spin}}((n_2, 2, 0, +\tfrac{1}{2}), x)\} \end{aligned}$$

The first operation takes an $\ell = 0$ source and produces any output reachable from it under E1 and spin-conserving rules. The second operation takes an $\ell = 2$ source and produces any reachable output. We attempt to compose them.

8.2 Direct composition: rejected

Consider the fragment

$$F = \text{Compose}(\text{Literal}((1, 0, 0, +\tfrac{1}{2})), \text{Call}(\mathcal{O}_2, \emptyset)).$$

The Compose chain threads the literal $\lambda_0 = (1, 0, 0, +\tfrac{1}{2})$ into the input of $\mathcal{O}_2$. The input refinement of $\mathcal{O}_2$ requires $\lambda_0 = (n_2, 2, 0, +\tfrac{1}{2})$; in particular, $\pi_\ell(\lambda_0) = 2$. But $\pi_\ell((1, 0, 0, +\tfrac{1}{2})) = 0$. The input refinement is violated, and by (R-CALL), the typing judgement fails. typecheck rejects F .

8.3 Through-route composition: also rejected

Consider the fragment

$$F' = \text{Compose}(\text{Literal}((1, 0, 0, +\tfrac{1}{2})), \text{Call}(\mathcal{O}_1, \emptyset), \text{Call}(\mathcal{O}_2, \emptyset)).$$

The first stage is typed at $\{(1, 0, 0, +\tfrac{1}{2})\}$ (a singleton refinement). The second stage, $\mathcal{O}_1$, accepts this input and outputs a label λ_1 satisfying $\sigma_{\text{E1}}(\lambda_0, \lambda_1)$ and $\sigma_{\text{spin}}(\lambda_0, \lambda_1)$. Reading off:

$$\pi_\ell(\lambda_1) - 0 \in \{-1, +1\} \implies \pi_\ell(\lambda_1) \in \{-1, +1\} \cap \mathbb{N} = \{1\},$$

forcing $\pi_\ell(\lambda_1) = 1$.

The third stage, $\mathcal{O}_2$, requires its input to satisfy $\pi_\ell = 2$. But the output of $\mathcal{O}_1$ has $\pi_\ell = 1$. The carry-threading transition $\lambda_1 \rightarrow \lambda_2$ feeds λ_1 with $\pi_\ell = 1$ into $\mathcal{O}_2$'s input refinement, which demands $\pi_\ell = 2$. This is a violation: (R-COMP) fails, and typecheck rejects F' .

8.4 Significance

The two rejections are structurally distinct—one fails at the Call stage, the other at the Compose threading—but both arise from the same underlying principle: the refinement type of the composed fragment is uninhabited. No physical $\ell = 0 \rightarrow \ell = 2$ single-photon transition exists under E1 selection rules, and no vaHera fragment can express it as a well-typed expression. The rejection is structural: there is no possible runtime that produces the implied transition, because no fragment can be written that implies it without violating the type system.

This is the central content of Theorem 6.3: forbidden transitions are not rare; they are type errors.

Part VI

Connections to Applications

9 Applications across the Symbolic-Physics Substrate

The refinement-type system specified here is parametric in the partition-coordinate structure and the selection-rule registry. We outline how the system applies as a foundational substrate for several downstream developments.

9.1 Bounded Hamiltonian Systems

The partition coordinates (n, ℓ, m, s) are the labels of bounded three-dimensional oscillatory systems, derived from first principles in Sachikonye (2026a). Every transition between such labels in a bounded Hamiltonian system must respect the closure constraints; the selection-rule registry $\sigma_\star$ encodes precisely those constraints. The present type system thereby provides the symbolic vocabulary for stating any bounded-Hamiltonian transition: a fragment is well-typed if and only if every transition it implies is admissible under the closure structure.

9.2 Mass-Spectrometric Trajectories

Ion trajectories in mass-spectrometric instruments—Orbitrap, FT-ICR, TOF,

quadrupole—are governed by a partition Lagrangian whose Euler-Lagrange equation reduces to the Lorentz force law (Sachikonye, 2026d). The state space of trajectories is parametrised by partition coordinates plus inertia and charge, and the trajectory’s evolution respects a set of motion-derived selection rules. Under the present framework, ion trajectories become well-typed vaHera fragments over the partition-label type, with the motion-derived rules as σ_* .

9.3 Temporal Programming and Cell Registries

The temporal-programming Tempus calculus (Sachikonye, 2026e) defines a cell registry that classifies timing deviations into operationally-defined cells. The cells are compile-time-immutable, partitioning a ΔP -space into action-equivalent regions. Under our framework, each cell becomes a refinement-typed operation: the cell’s bounds are the input refinement, the dispatched action is the operation body, and the cell-disjointness invariant is enforced as a type-system property. Selection rules over cell-to-action transitions become first-class predicates rather than hand-coded checks.

9.4 Real-Time Event-Selection Systems

In high-throughput event-selection systems—trigger systems for high-energy experiments—each detector subsystem registers as a refined operation whose output Value carries one partition coordinate of the detected particle. The full classification of a particle is a Compose chain over the subsystems, and the per-stage selection rules enforce the physical admissibility of the combined classification. Backgrounds whose hypothesised classification violates any selection rule fail typecheck before dispatch, requiring no runtime observation.

9.5 Three-Route Verification

The Three-Route Equivalence Theorem of computational operations (Sachikonye, 2026b) asserts that any partition-state invariant admits three independent computational measurements that agree to within a closed-form bound. Under the present type system, each route becomes a refined

operation, all sharing the same output type but with distinct internal computations. Verification reduces to running all three operations and checking that the outputs lie in the same refinement equivalence class—a type-level check rather than a numerical comparison.

Part VII Discussion

10 Comparison with Existing Refinement-Type Systems

The refinement-type literature contains several mature systems. We comment on the relationship of the present development to each.

10.1 Liquid Types

The Liquid Types system of Rondon et al. (2008) establishes refinement-type inference via predicate abstraction: refinements are SMT-decidable predicates over base types, and refinement inference is reduced to Horn-clause solving. Our system is restricted to the partition-coordinate domain, where the predicates are simple arithmetic comparisons; we obtain decidability in linear time without invoking SMT machinery. The trade-off is generality: Liquid Types apply to arbitrary base types, while ours is specialised to $\mathcal{L}$.

10.2 Refinement Types for Haskell

The system of Vazou et al. (2014) extends Liquid Types to a real-world functional language. The treatment of dependent function types and the encoding of inductive datatypes parallel our treatment of refined operations and Compose chains. We differ chiefly in the binary-predicate generalisation (4.2), which is not standard in the Haskell setting; binary predicates appear in dependent-type systems (Coquand and Huet, 1988; Norell, 2007) but in fully general form, without our compositionality lemma’s special-case efficiency.

10.3 ML-Style Refinements

Freeman and Pfenning’s refinement types for ML (Freeman and Pfenning, 1991) introduced regular tree types as refinements of inductive types. The Compose chain corresponds, structurally, to

a tree-shaped value flow, and our compositionality lemma is a strengthening of the tree-product property they establish. The forbidden-transition theorem is, in their language, a statement that the regular tree language defined by uninhabited refinements is empty.

10.4 Dependent Types

Coquand and Huet’s Calculus of Constructions (Coquand and Huet, 1988) and its successors (Agda (Norell, 2007), Lean (de Moura et al., 2015), Coq (Bertot and Castéran, 2004)) admit selection-rule predicates as dependent function types directly. Our framework can be viewed as a restricted, decidable fragment of such a dependent system, in which the predicate language is constrained to ensure the linear-time complexity of Theorem 7.1.

10.5 Hoare Logic

The Hoare-logic tradition (Hoare, 1969; Dijkstra, 1975) encodes pre- and post-conditions as assertions over program states. Our refined operations, with input refinement ϕ^{in} and output refinement ϕ^{out} , are Hoare triples specialised to vaHera dispatch. The compositionality lemma is the sequential composition rule of Hoare logic, with the carry-threading providing the implicit state.

11 Limitations and Open Questions

11.1 Predicate language

Our predicate language is restricted to constant-time computable Boolean functions on partition labels. More expressive predicates—second-order quantification, transitive closures, fixpoint computations—are outside the scope of the linear-time decidability result. Extending the framework to richer predicate logics would require a corresponding relaxation of the complexity bound.

11.2 Hole substitution

The treatment of Holes in Definition 2.7 assumes that all Holes have been resolved before typecheck runs. A more refined treatment would address Hole-resolution under refinement: given a fragment with unresolved Holes, what is the most

general refinement type compatible with all possible Hole substitutions? This question generalises principal-typing for parametric polymorphism (Milner, 1978; Damas and Milner, 1982) and is left for future work.

11.3 Operation registration

Operations are registered in a global registry $\mathcal{R}^+$. In a distributed setting, multiple registries may coexist with different selection-rule registries. Reconciling fragments typed against different registries—particularly the question of when a fragment well-typed under $\mathcal{R}_A^+$ remains well-typed under $\mathcal{R}_B^+$ —is a federation question that the present framework does not address.

11.4 Selection-rule learning

We treat the selection-rule registry σ_* as fixed and externally provided. The inverse problem—given a corpus of accepted transitions, infer the minimal σ_* —is an automated specification synthesis problem; we do not pursue it here.

12 Conclusion

We have introduced a refinement-type system over the vaHera AST in which partition coordinates are first-class types and selection rules are first-class refinement predicates. The system rests on three principal results: the Compositionality Lemma (Theorem 5.2), which establishes that refinements compose under Compose via conjunction; the Forbidden-Transition Theorem (Theorem 6.3), which establishes that selection-rule-violating fragments are rejected by typecheck at static time; and the Decidability Theorem (Theorem 7.1), which establishes that the joint judgement is decidable in linear time in the fragment size.

These results provide the symbolic substrate over which physical derivations expressing transitions between partition labels can be written and statically verified. The substrate is sufficient to specify any selection rule expressible as a computable binary predicate, and the resulting type discipline rejects every fragment whose composition chain implies a forbidden transition—without runtime observation, without probabilistic argument, and without recourse to external verification machinery.

The framework is foundational: it does not specify which partition coordinates are used, which selection rules are registered, or which operations populate the registry. These choices are downstream, made by the applications that build on the substrate. What the framework guarantees is that, once those choices are made, the resulting symbolic representation has a statically-decidable well-formedness criterion, and that the criterion is structural rather than probabilistic.

The downstream applications outlined in Section 9—bounded Hamiltonian dynamics, mass-spectrometric trajectories, temporal-programming cell registries, real-time event-selection systems, and three-route verification—each adopt the substrate by registering an appropriate partition-coordinate domain and selection-rule registry. None of them re-implements the type-checking machinery; each cites the present development for its symbolic foundations and contributes its domain-specific operations on top.

References

- Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development: Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. Springer, 2004.
- Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2-3):95–120, 1988.
- Luis Damas and Robin Milner. Principal type-schemes for functional programs. In *Proceedings of the 9th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 207–212. ACM, 1982.
- Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. The Lean theorem prover (system description). In *Automated Deduction — CADE-25*, volume 9195 of *Lecture Notes in Computer Science*, pages 378–388. Springer, 2015.
- Edsger W. Dijkstra. Guarded commands, non-determinacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975.
- Tim Freeman and Frank Pfenning. Refinement types for ML. In *Proceedings of the ACM SIGPLAN 1991 Conference on Programming Language Design and Implementation (PLDI)*, pages 268–277, Toronto, Ontario, Canada, 1991. ACM.
- Roger Hindley. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society*, 146:29–60, 1969.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Robin Milner. A theory of type polymorphism in programming. *Journal of Computer and System Sciences*, 17(3):348–375, 1978.
- Ulf Norell. *Towards a Practical Programming Language Based on Dependent Type Theory*. PhD thesis, Chalmers University of Technology, 2007.
- Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 159–169, Tucson, AZ, USA, 2008. ACM.
- Kundai Farai Sachikonye. On the bijective correspondence between integer sequences and partition states in bounded hamiltonian systems: Counting, monotonicity, and causal succession. Monograph, Volume I, 2026a.
- Kundai Farai Sachikonye. Computational operations equivalence: A single-quantity bookkeeping framework for kernel operations. Buhera Project, Volume IV, 2026b.
- Kundai Farai Sachikonye. Integrating with the purpose framework: The stability contract. Technical specification, Buhera Project, 2026c. Available as long-grass/integration.md.
- Kundai Farai Sachikonye. Ion trajectory completion mechanism: Partition lagrangian, mass spectrometry, and mass from three routes. Monograph, Volume II, 2026d.
- Kundai Farai Sachikonye. Temporal programming: A cell-based paradigm for oscillator-relative distributed computation. Monograph, Volume III, 2026e.

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 269–282, Gothenburg, Sweden, 2014. ACM.

Three-Route Equivalence as a Computational Verification Primitive: A Methodology for Bounded Categorical Invariants

Kundai Farai Sachikonye
 Technical University of Munich
 Chair of Proteomics and Bioanalytics
 AIME Registry for Artificial Intelligence
 Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
 kundai.sachikonye@wzw.tum.de

May 31, 2026

Abstract

We introduce three-route equivalence as a computational verification primitive: a structural methodology in which a single bounded categorical invariant Q is measured by three computationally distinct algorithms whose outputs must agree to within a closed-form bound. The methodology decouples the question of *what* is being computed (a single invariant) from the question of *how* (three independent algorithms), and provides a structural rather than statistical guarantee of agreement. We establish four principal results. (i) The *Routes-Are-Well-Defined Theorem*: under finite-depth refinement, any bounded categorical invariant admits at least three computationally distinct routes, each of which terminates in time polynomial in the depth. (ii) The *Three-Route Convergence Theorem*: the maximum pairwise disagreement between the three routes is bounded above by $K(d+1)^{-n}$, where d is the dimensional parameter of the underlying categorical structure, n is the refinement depth, and K is a problem-specific constant. (iii) The *Verification Soundness and Completeness Theorem*: a candidate value $\hat{Q}$ is accepted iff the maximum three-route disagreement on $\hat{Q}$ falls below the convergence bound; the acceptance criterion is sound, complete, and decidable. (iv) The *Computational Equivalence Theorem*: the three-route primitive composes with the refinement-type substrate of vaHera, in the sense that route outputs that pass verification at depth n are inhabitants of the same refinement type. The methodology applies uniformly

to any bounded categorical invariant; we illustrate the application with specialisations to the gravitational constant, ion mass, the speed of light, kernel operation weight, and Ritz combinations in atomic spectroscopy. The framework is purely structural: no statistical assumption is required, no calibration external to the routes is invoked, and the verification primitive runs as a single deterministic kernel-level operation. We discuss the implementation as the proof-validation engine of a categorical dispatch substrate and the role of the methodology as a foundational verification tool for downstream physical-derivation languages.

Keywords: three-route equivalence, verification primitive, computational independence, categorical invariant, refinement depth, convergence bound, structural verification, kernel proof-validation, bounded categorical structure.

Contents

I	Preliminaries	2
1	Introduction	2
1.1	The verification problem	2
1.2	The structural alternative	3
1.3	Origin and applicability	3
1.4	Overview of main results	3
2	Bounded Categorical Invariants	3
2.1	Categorical structure	3
2.2	Invariants	4

II	The Three-Route Primitive	4	10 Comparison with Existing Verification Methodologies	10
3	Routes	4	10.1 Statistical replication	10
3.1	Definition of a route	4	10.2 External-reference verification . . .	10
3.2	Computational independence . . .	4	10.3 Property-based testing	10
3.3	Three-route configurations	5	10.4 Triangulation in measurement theory	10
4	The Convergence Theorem	5	11 Limitations and Open Questions	10
4.1	Routes are well-defined	5	11.1 Route-construction scope	10
4.2	The convergence bound	5	11.2 Convergence constant determination	10
			11.3 Computational independence veri-	
			fication	11
III	The Verification Primitive	6	11.4 Configurations beyond three routes	11
5	Verification	6	12 Conclusion	11
5.1	The verification problem	6		
5.2	The verification primitive	6		
5.3	Soundness	6		
5.4	Completeness	7		
6	Decidability and Algorithmic Properties	7		
6.1	Termination and complexity	7		
6.2	Determinism and reproducibility .	7		
IV	Composition with the Refinement-Type Substrate	7		
7	Type-System Integration	7		
7.1	Refinement-typed routes	7		
7.2	Composition under Compose . . .	8		
V	Specialisations	8		
8	Worked Specialisations	8		
8.1	Kernel operation weight	8		
8.2	The gravitational constant	8		
8.3	Ion mass	9		
8.4	The speed of light	9		
8.5	Ritz combinations in atomic spectroscopy	9		
8.6	Cross-domain summary	9		
VI	Implementation and Discussion	9		
9	Implementation as a Kernel Subsystem	9		
9.1	The proof-validation engine	9		
9.2	Conformance regression	9		

Part I

Preliminaries

1 Introduction

1.1 The verification problem

Suppose a derivation has produced a numerical value $\hat{Q}$ that is claimed to be the true value of some invariant Q of a bounded categorical structure—a gravitational constant, a kernel throughput, an ion mass, a transition matrix element. How does one verify the claim? Three standard strategies dominate the literature.

The *statistical* strategy compares $\hat{Q}$ to a sample mean of independent measurements, accepting the claim when the sample mean lies within a confidence interval (Jaynes, 2003; Bevington and Robinson, 2003). This requires repeated measurement under varying conditions and assumes that systematic and random errors are separable. The *external-reference* strategy compares $\hat{Q}$ to a value reported by an authoritative source, such as the CODATA tabulation of fundamental constants (Tiesinga et al., 2021). This requires the existence of, and trust in, an external authority that is at least as accurate as the present measurement. The *re-derivation* strategy re-derives $\hat{Q}$ from first principles within the same framework and checks that the redo produces the same value. This is a self-consistency check, not an independence check, and is therefore vulnerable to systematic errors that persist across re-derivations.

Each strategy has clear failure modes. Statistical verification cannot detect a systematic

error that affects every sample equally. External-reference verification merely shifts the verification problem to the external source. Re-derivation verification, being self-consistency, does not provide independence.

1.2 The structural alternative

This paper proposes a fourth strategy, which we call *three-route equivalence*. The candidate value $\hat{Q}$ is verified by running three computationally distinct algorithms—*routes*—each of which produces a value claimed to be Q , and accepting $\hat{Q}$ iff the maximum pairwise disagreement between the three route outputs falls below a closed-form bound. The bound is structural, derived from the categorical depth n and the dimensional parameter d of the underlying structure, and given by the closed form $K(d+1)^{-n}$ for a problem-specific constant K .

The strategy has the following properties that the three standard strategies lack.

Independence of route construction. The three routes are computationally distinct: no two of them share any intermediate observable, any auxiliary computation, or any common dependence on the same external reference. Systematic errors that affect one route do not propagate to the others. We make this precise in Definition 3.4.

Closed-form convergence bound. The disagreement bound is given by a closed-form expression in d and n . No empirical determination of the bound is required, and the bound tightens predictably with refinement depth.

No external authority. The verification is internal: the three routes generate their own reference data, and the acceptance criterion compares them only against each other. The framework imposes no requirement for trust in a CODATA-style external tabulation.

Structural rather than statistical. The acceptance criterion is a deterministic comparison, not a probabilistic confidence statement. A candidate value either passes or fails; there is no p -value to interpret.

1.3 Origin and applicability

The three-route methodology arose in the analysis of kernel operations (Sachikonye, 2026a), where it was shown that the categorical weight Q of a kernel operation admits three independent measurements—accumulated decision

residue, confinement cost, and the fixed point of an iterated negation operator—that agree to machine precision under finite refinement. The same structural argument applies, with appropriate route definitions, to any bounded categorical invariant. Subsequent investigations have specialised the methodology to the gravitational constant (Sachikonye, 2026h), ion mass (Sachikonye, 2026d), the speed of light (Sachikonye, 2026e), and Ritz combinations in atomic spectroscopy (Sachikonye, 2026b).

This paper extracts the underlying computational primitive and treats it as a foundational verification tool. Our concern is not which invariants the methodology applies to—that question is application-specific—but what the methodology itself is, and what guarantees it offers as a computational object.

1.4 Overview of main results

We establish the following:

1. **Theorem 4.1** (Routes Are Well-Defined): For every bounded categorical invariant Q on a structure of dimension d , three computationally distinct routes are constructible in time polynomial in the refinement depth n .
2. **Theorem 4.3** (Three-Route Convergence): the maximum pairwise disagreement is bounded by $K(d+1)^{-n}$.
3. **Theorem 5.3** (Verification Soundness): a candidate value passing the three-route check at depth n lies within $K(d+1)^{-n}$ of the true invariant.
4. **Theorem 5.4** (Verification Completeness): a candidate value deviating from the true invariant by more than $2K(d+1)^{-n}$ is rejected.
5. **Theorem 6.1** (Decidability): the verification primitive runs in time $\mathcal{O}(T_I + T_{II} + T_{III})$ where T_i is the route- i evaluation cost; the algorithm is total.
6. **Theorem 7.2** (Computational Equivalence): three-route equivalence composes with the refinement-type substrate of Sachikonye (2026f): verified outputs inhabit the same refinement type.

2 Bounded Categorical Invariants

2.1 Categorical structure

A bounded categorical structure is the basic object on which the three-route methodology operates. We adopt a deliberately minimal definition that subsumes the kernel-operation, atomic-spectroscopic, and physical-constant cases as instances.

Definition 2.1 (Bounded Categorical Structure). *A bounded categorical structure is a triple $\mathcal{C} = (\mathcal{S}, d, \prec)$ where:*

- $\mathcal{S}$ is a non-empty set of states;
- $d \in \mathbb{N}^+$ is the dimension of the structure;
- $\prec \subseteq \mathcal{S} \times \mathcal{S}$ is a partial order (the refinement order) with a designated initial element $s_0 \in \mathcal{S}$ that is minimal under $\prec$.

The structure is bounded if for every $s \in \mathcal{S}$ there exists a finite n such that s is reachable from s_0 in at most n refinement steps.

Definition 2.2 (Refinement Depth). *For $s \in \mathcal{S}$, the refinement depth of s is the length of the shortest $\prec$ -chain from s_0 to s :*

$$n(s) = \min\{k \in \mathbb{N} : \exists s_0 \prec s_1 \prec \dots \prec s_k = s\}.$$

The depth of the structure at level n is the set of states with that minimal depth.

2.2 Invariants

Definition 2.3 (Invariant). *An invariant on a bounded categorical structure $\mathcal{C}$ is a function $Q : \mathcal{S} \rightarrow \mathbb{R}$ that is constant on each $\prec$ -equivalence class: if $s \prec s'$ and $s' \prec s$, then $Q(s) = Q(s')$.*

Remark 2.4. *Definition 2.3 allows Q to vary across $\prec$ -equivalence classes at different depths. What it requires is internal consistency: an invariant cannot take two distinct values on $\prec$ -equivalent states. This permits invariants that depend on depth (e.g. a running coupling) and invariants that do not (e.g. a fundamental constant), provided the dependence respects the equivalence structure.*

Definition 2.5 (Bounded Invariant). *The invariant Q is bounded if there exists $M_Q \in \mathbb{R}^+$ such that $|Q(s)| \leq M_Q$ for every $s \in \mathcal{S}$.*

Henceforth we assume invariants are bounded; the unbounded case is outside the scope of three-route methodology in the form we develop here.

Part II

The Three-Route Primitive

3 Routes

3.1 Definition of a route

Definition 3.1 (Route). *A route to an invariant Q on structure $\mathcal{C}$ is a total computable function*

$$\rho : \mathcal{S} \times \mathbb{N}^+ \rightarrow \mathbb{R},$$

where $\rho(s, n)$ is interpreted as the route's estimate of $Q(s)$ obtained by computation to refinement depth n . A route is convergent at s if there exists $K_\rho \in \mathbb{R}^+$ and a function $\beta_\rho : \mathbb{N}^+ \rightarrow \mathbb{R}^+$ with $\lim_{n \rightarrow \infty} \beta_\rho(n) = 0$ such that for all n ,

$$|\rho(s, n) - Q(s)| \leq K_\rho \beta_\rho(n).$$

The decay function β_ρ encodes the rate at which the route's output approaches the true invariant as refinement deepens. For the methodology developed here, we are concerned with the geometric decay $\beta_\rho(n) = (d+1)^{-n}$, which arises naturally in d -dimensional bounded structures.

Definition 3.2 (Standard Convergence). *A route ρ to Q has standard convergence on $\mathcal{C}$ if its decay function is $\beta_\rho(n) = (d+1)^{-n}$. The constant K_ρ is the route's convergence constant.*

3.2 Computational independence

The methodological heart of three-route equivalence is the requirement that the routes be computationally distinct. We make this precise.

Definition 3.3 (Intermediate Observable). *An intermediate observable of a route ρ is a function $\omega : \mathcal{S} \times \mathbb{N}^+ \rightarrow \mathbb{R}$ that is computed as a strict sub-expression of ρ . The set of intermediate observables of ρ at depth n is denoted $\Omega_\rho(n)$.*

Definition 3.4 (Computational Independence). *Two routes ρ_a and ρ_b to the same invariant Q are computationally independent at depth n if*

$$\Omega_{\rho_a}(n) \cap \Omega_{\rho_b}(n) = \emptyset.$$

A family $\{\rho_1, \dots, \rho_k\}$ of routes is pairwise computationally independent at depth n if every pair is computationally independent at that depth.

Remark 3.5. *Computational independence is a syntactic property of the route's expression, not a statistical or empirical claim about its outputs. Two routes that compute different sub-expressions are computationally independent regardless of how correlated their outputs may turn out to be. This is the key conceptual difference between three-route equivalence and statistical replication: replication produces correlated samples by re-running the same algorithm; three-route methodology produces independent estimates by running different algorithms.*

3.3 Three-route configurations

Definition 3.6 (Three-Route Configuration). A three-route configuration for invariant Q on structure $\mathcal{C}$ at depth n is a triple $(\rho_I, \rho_{II}, \rho_{III})$ of routes to Q such that:

1. each route has standard convergence on $\mathcal{C}$ (Definition 3.2);
2. the family $\{\rho_I, \rho_{II}, \rho_{III}\}$ is pairwise computationally independent at depth n (Definition 3.4).

The choice of three routes (rather than two or four) is methodological: two routes provide pairwise comparison without redundancy detection, while four or more begin to encounter combinatorial overhead for independence verification. Three is the minimum that detects any single-route systematic error, and is the configuration we focus on. The framework extends straightforwardly to k -route configurations.

4 The Convergence Theorem

4.1 Routes are well-defined

We first establish that three-route configurations exist for every bounded categorical invariant.

Theorem 4.1 (Routes Are Well-Defined). *Let Q be a bounded invariant on a bounded categorical structure $\mathcal{C} = (\mathcal{S}, d, <)$. Then for every $n \in \mathbb{N}^+$, there exists a three-route configuration $(\rho_I, \rho_{II}, \rho_{III})$ for Q at depth n . Each route terminates in time $\mathcal{O}(n^p)$ for some polynomial degree p depending on $\mathcal{C}$ and Q but not on n .*

Proof. Existence. The bounded categorical structure admits three structurally distinct measurement strategies, by virtue of the standard categorical decomposition:

- Route I (residue): accumulate observed transitions $s_0 < s_1 < \dots$ up to depth n and report the value Q acquires along the chain.
- Route II (confinement): enumerate all states at depth $\leq n$ and compute the closure cost of confining Q to that depth.
- Route III (negation fixed point): iterate the categorical negation operator $Q \mapsto \neg\neg Q$ to fixed point at depth n .

The three strategies are syntactically distinct expressions whose intermediate observables are pairwise disjoint, satisfying Definition 3.4.

Convergence rate. Each strategy enumerates structure to depth n . The cardinality of the depth- n portion of $\mathcal{C}$ is bounded by $C(d, n) = \mathcal{O}(n^d)$ (cf. the cumulative-count formula in the partition framework, $N_{\text{state}}(n) = n(n+1)(2n+1)/3$ at $d=3$). Each strategy visits each depth- n state in $\mathcal{O}(1)$ time, giving total cost $\mathcal{O}(n^d)$.

Termination. Each strategy is a structurally recursive function on a finite state set, hence terminating.

The configuration is thus a triple of well-defined, polynomial-time, computationally-independent routes. $\square$ $\square$

Remark 4.2. *The construction in Theorem 4.1 is generic: it depends only on the categorical decomposition of $\mathcal{C}$, not on the specific invariant. Applications must instantiate each route concretely for their domain—a residue-route for atomic spectroscopy is a transition-count computation, while a residue-route for kernel operations is a decision-count computation—but the structure of the construction is uniform.*

4.2 The convergence bound

The central result follows.

Theorem 4.3 (Three-Route Convergence). *Let $(\rho_I, \rho_{II}, \rho_{III})$ be a three-route configuration for invariant Q on structure $\mathcal{C}$ at depth n , with respective convergence constants K_I, K_{II}, K_{III} . Then for every $s \in \mathcal{S}$,*

$$\max_{i,j \in \{I, II, III\}} |\rho_i(s, n) - \rho_j(s, n)| \leq 2K(d+1)^{-n} \quad (1)$$

where $K = \max_i K_i$.

Proof. For any pair i, j ,

$$\begin{aligned} |\rho_i(s, n) - \rho_j(s, n)| &\leq |\rho_i(s, n) - Q(s)| \\ &\quad + |Q(s) - \rho_j(s, n)| \\ &\leq K_i(d+1)^{-n} + K_j(d+1)^{-n} \\ &\leq 2K(d+1)^{-n} \end{aligned}$$

by the triangle inequality and the standard-convergence hypothesis for each route. Taking the maximum over pairs preserves the bound. $\square$ $\square$

Corollary 4.4 (Geometric Tightening). *The maximum pairwise disagreement decays geometrically in the refinement depth: increasing n by 1 multiplies the bound by $(d+1)^{-1}$.*

Proof. Direct from (1). $\square$ $\square$

Remark 4.5. *Corollary 4.4 gives the implementation engineer a quantitative handle on the trade-off between verification confidence and computational cost. Doubling the disagreement bound's tightness requires only one additional refinement level—a logarithmic relationship that scales favourably.*

is the function defined by Algorithm 1: it runs each of the three routes at depth n , computes the maximum pairwise disagreement, and accepts the candidate value $\hat{Q}$ iff the disagreement falls below the convergence bound and $\hat{Q}$ lies within the route outputs' span.

Algorithm 1 Three-Route Verification Primitive

```

1: function VERIFY( $\mathcal{C}, \hat{Q}, n$ )
2:    $Q_I \leftarrow \rho_I(s_0, n)$ 
3:    $Q_{II} \leftarrow \rho_{II}(s_0, n)$ 
4:    $Q_{III} \leftarrow \rho_{III}(s_0, n)$ 
5:    $\delta \leftarrow \max_{i,j} |Q_i - Q_j|$ 
6:    $\varepsilon \leftarrow 2K \cdot (d+1)^{-n}$ 
7:   if  $\delta > \varepsilon$  then
8:     return false
9:   end if
10:   $Q_{\min} \leftarrow \min(Q_I, Q_{II}, Q_{III})$ 
11:   $Q_{\max} \leftarrow \max(Q_I, Q_{II}, Q_{III})$ 
12:  if  $\hat{Q} < Q_{\min} - \varepsilon$  or  $\hat{Q} > Q_{\max} + \varepsilon$  then
13:    return false
14:  end if
15:  return true
16: end function

```

Part III

The Verification Primitive

5 Verification

5.1 The verification problem

We now formalise the use of the three-route methodology as a verification primitive.

Definition 5.1 (Verification Problem). *Given a bounded categorical structure $\mathcal{C}$, a bounded invariant Q , a three-route configuration $(\rho_I, \rho_{II}, \rho_{III})$ at depth n , and a candidate value $\hat{Q} \in \mathbb{R}$, the three-route verification problem is to decide whether $\hat{Q}$ is consistent with the true invariant value $Q(s_0)$ to within $K(d+1)^{-n}$.*

5.2 The verification primitive

Definition 5.2 (Three-Route Verification Primitive). *The three-route verification primitive*

$$\text{Verify} : \mathcal{C} \times \mathcal{Q} \times \mathbb{N}^+ \rightarrow \mathbb{B}$$

5.3 Soundness

Theorem 5.3 (Verification Soundness). *If $\text{Verify}(\mathcal{C}, \hat{Q}, n)$ returns true, then*

$$|\hat{Q} - Q(s_0)| \leq 3K(d+1)^{-n}.$$

Proof. Assume Verify returns true. Then the algorithm did not exit at line 7, so $\delta \leq 2K(d+1)^{-n}$, and did not exit at line 11, so

$$Q_{\min} - \varepsilon \leq \hat{Q} \leq Q_{\max} + \varepsilon$$

with $\varepsilon = 2K(d+1)^{-n}$.

By the standard-convergence hypothesis, $|Q_i - Q(s_0)| \leq K(d+1)^{-n}$ for each i . Hence

$$Q(s_0) - K(d+1)^{-n} \leq Q_{\min}$$

and

$$Q_{\max} \leq Q(s_0) + K(d+1)^{-n}.$$

Substituting:

$$\begin{aligned} \hat{Q} &\geq Q_{\min} - \varepsilon \\ &\geq Q(s_0) - K(d+1)^{-n} - 2K(d+1)^{-n} \\ &= Q(s_0) - 3K(d+1)^{-n}, \end{aligned}$$

and symmetrically $\hat{Q} \leq Q(s_0) + 3K(d+1)^{-n}$. The bound follows. $\square$ $\square$

5.4 Completeness

Theorem 5.4 (Verification Completeness). *If $|\hat{Q} - Q(s_0)| > 4K(d+1)^{-n}$, then $\text{Verify}(\mathcal{C}, \hat{Q}, n)$ returns false.*

Proof. By the standard-convergence hypothesis, all three route outputs Q_i lie within $K(d+1)^{-n}$ of $Q(s_0)$. Hence $Q_{\max} \leq Q(s_0) + K(d+1)^{-n}$ and $Q_{\min} \geq Q(s_0) - K(d+1)^{-n}$. The acceptance window is therefore contained in

$$[Q(s_0) - 3K(d+1)^{-n}, Q(s_0) + 3K(d+1)^{-n}],$$

and a candidate $\hat{Q}$ outside $[Q(s_0) - 4K(d+1)^{-n}, Q(s_0) + 4K(d+1)^{-n}]$ certainly lies outside the (strictly smaller) acceptance window, triggering rejection at line 11 of Algorithm 1. $\square$ $\square$

Remark 5.5. *The gap between the soundness bound $3K(d+1)^{-n}$ and the completeness bound $4K(d+1)^{-n}$ is the verification window’s grey zone: candidates lying in this interval may be either accepted or rejected depending on the specific route outputs. This zone shrinks geometrically with depth, and at the depths used in physical-constant applications ($n \geq 8$) it is well below any reasonable measurement precision. Closing the gap exactly requires a tighter convergence analysis that we do not pursue here.*

6 Decidability and Algorithmic Properties

6.1 Termination and complexity

Theorem 6.1 (Decidability of Verify). *The three-route verification primitive runs in time $\mathcal{O}(T_I + T_{II} + T_{III})$, where T_i is the cost of evaluating route i at depth n . The algorithm is total: it terminates on every well-formed input and returns either true or false.*

Proof. The algorithm performs three route evaluations (lines 2–4), each at cost $T_i = \mathcal{O}(n^d)$ by Theorem 4.1, followed by constant-time arithmetic operations. Total cost is the sum. Termination follows from the termination of each ρ_i (Theorem 4.1). Totality follows from the explicit branching structure of the algorithm: every input either passes both rejection tests (returning true) or fails one (returning false). $\square$ $\square$

Corollary 6.2 (Independence of Routes). *Because the three routes are computationally independent (Definition 3.4), they may be evaluated in parallel. The parallel evaluation cost is $\mathcal{O}(\max_i T_i)$.*

Proof. Computational independence implies the routes share no intermediate observables; their evaluations therefore have no data dependencies, and can be assigned to independent execution units without synchronisation. $\square$ $\square$

6.2 Determinism and reproducibility

Proposition 6.3 (Determinism). *The three-route verification primitive is a deterministic function of $(\mathcal{C}, \hat{Q}, n)$: re-running Verify on identical inputs produces identical outputs.*

Proof. The algorithm uses only deterministic primitives (the routes are total computable functions, arithmetic operations are deterministic, comparisons are deterministic). The output is a deterministic function of the inputs. $\square$ $\square$

Corollary 6.4 (Reproducibility). *The three-route verification result is reproducible across implementations that respect Definitions 3.1 and 3.4, modulo floating-point representation effects bounded by the machine epsilon.*

Proof. By Proposition 6.3, the output is determined by the inputs. Different implementations differ only in their floating-point arithmetic, whose discrepancies are bounded by the machine epsilon. $\square$ $\square$

Part IV

Composition with the Refinement-Type Substrate

7 Type-System Integration

7.1 Refinement-typed routes

The three-route methodology integrates with the refinement-type substrate established for the va-Hera AST (Sachikonye, 2026f). In that framework, partition labels are refinement base types and selection rules are refinement predicates; here we extend the framework to invariant verification.

Definition 7.1 (Refinement-Typed Route). *A route ρ to invariant Q is refinement-typed at depth n if the route's signature is*

$$\rho : \{x : \mathcal{C} \mid x \in \mathcal{S}_{\leq n}\} \rightarrow \{y : \mathbb{R} \mid |y - Q(x)| \leq K_\rho(d + 1)^{-n}\}$$

where $\mathcal{S}_{\leq n}$ is the set of states with refinement depth at most n .

Theorem 7.2 (Computational Equivalence). *Let $(\rho_I, \rho_{II}, \rho_{III})$ be a three-route configuration for Q at depth n , each route refinement-typed in the sense of Definition 7.1. If $\text{Verify}(\mathcal{C}, \hat{Q}, n)$ returns true, then $\hat{Q}$ is an inhabitant of the refinement type*

$$T_n = \{y : \mathbb{R} \mid |y - Q(s_0)| \leq 3K(d + 1)^{-n}\}.$$

Proof. By Theorem 5.3, acceptance entails $|\hat{Q} - Q(s_0)| \leq 3K(d + 1)^{-n}$, which is precisely the predicate defining T_n . Hence $\hat{Q} \in T_n$. $\square$ $\square$

Corollary 7.3 (Type-Level Verification). *Three-route verification can be expressed as the inhabitation check $\hat{Q} : T_n$ under the refinement-type substrate. Verification becomes a type-checking obligation rather than a separate run-time check.*

Proof. Inhabitation of a refinement type is checked by evaluating the refinement predicate on the candidate value. The predicate defining T_n is the verification soundness condition. Hence the inhabitation check coincides with the verification primitive's result. $\square$ $\square$

The corollary establishes that three-route verification is not an additional layer external to the type system but a derivable consequence of the type system's machinery. A kernel dispatch substrate that already implements refinement-type checking obtains three-route verification at no marginal cost.

7.2 Composition under Compose

Proposition 7.4 (Compositionality of Verified Invariants). *Let $\hat{Q}_1$ pass verification at depth n_1 on structure $\mathcal{C}_1$, and let $\hat{Q}_2$ pass verification at depth n_2 on structure $\mathcal{C}_2$. Let $g : \mathbb{R}^2 \rightarrow \mathbb{R}$ be a Lipschitz-continuous combiner with constant L_g . Then $g(\hat{Q}_1, \hat{Q}_2)$ satisfies*

$$|g(\hat{Q}_1, \hat{Q}_2) - g(Q_1, Q_2)| \leq L_g \cdot \max(3K_1(d+1)^{-n_1}, 3K_2(d+1)^{-n_2}).$$

Proof. By the Lipschitz hypothesis,

$$|g(\hat{Q}_1, \hat{Q}_2) - g(Q_1, Q_2)| \leq L_g \max(|\hat{Q}_1 - Q_1|, |\hat{Q}_2 - Q_2|).$$

Substituting the soundness bounds gives the result. $\square$ $\square$

The proposition shows that verified invariants compose without loss of guarantees under any Lipschitz-continuous combiner. This is the key compositionality property required for the methodology to scale across a derivation chain, in the manner of the vaHera Compose semantics.

Part V

Specialisations

8 Worked Specialisations

We illustrate the methodology with five domain instantiations. Each specialisation supplies its own routes, dimension parameter, and convergence constant; the verification primitive itself is unchanged.

8.1 Kernel operation weight

The original setting (Sachikonye, 2026a): a kernel operation has a computational weight $Q \in \mathbb{N}^+$. Three routes:

- Route I (residue): accumulated decision count during the operation's categorical lifetime.
- Route II (confinement): cells visited in the operation's confinement cost.
- Route III (negation): depth at which iterated negation reaches the singleton fixed point.

Dimension $d = 3$; convergence constant $K = 0$ at integer weights (each route returns the exact integer); conformance test (the V6 entry of the COE validation suite) reports 50/50 trials with $\max_{i,j} |Q_i - Q_j| = 0$.

8.2 The gravitational constant

Three routes to the gravitational constant (Sachikonye, 2026h):

- Route I (oscillation-ratio): $G^{(I)} = (c^3/\hbar) A_{12}^2 \pi$ from oscillation amplitudes.
- Route II (category fixed-point): $G^{(II)}$ from the fixed point of the categorical iteration $g^* =$

- Route III (partition-density): $G^{(\text{III})} = (c^3/\hbar) \pi R^{1/(d+1)} T_{\text{ref}}^{-1}$ from the partition density ratio.

Dimension $d = 3$; convergence constant $K \sim 10^{-2}$ in CODATA units; the three-route disagreement at $n = 56$ is below the CODATA tabulated uncertainty $\sim 2.2 \times 10^{-5}$.

8.3 Ion mass

Three routes to ion mass (Sachikonye, 2026d):

- Route I: trajectory-based mass from cyclotron frequency in a known field.
- Route II: confinement-based mass from time-of-flight under a known electric impulse.
- Route III: negation-fixed-point mass from the iterated-trapping condition in an Orbitrap.

Dimension $d = 3$; validation on 4545 NIST library entries and 127,000 proteomic transformations (Sachikonye, 2026d) reports route agreement to within instrument-noise tolerances.

8.4 The speed of light

Three routes to the speed of light (Sachikonye, 2026e):

- Route I: $c = \Delta x / \tau_c$ from partition propagation distance and lag.
- Route II: c from dispersion measurements over a known interval.
- Route III: c from the categorical-angular reformulation $c = 2\pi \text{ rad/tick}$.

Dimension $d = 3$. The three-route convergence underpins the derivation of the LHC bunch-crossing window from detector geometry (Sachikonye, 2026g).

8.5 Ritz combinations in atomic spectroscopy

Three routes to a Ritz combination (Sachikonye, 2026b):

- Route I: direct line transition frequency from spectroscopic measurement.
- Route II: sum-difference combination of two complementary transitions.
- Route III: refinement at four-tier categorical depth, validated by the categorical spectrometer.

Dimension $d = 3$; the cascade reports precision tightening from 0.07% at $n = 1$ to 0.003% at $n = 4$. The methodology extends naturally to a

four-route configuration in this setting, providing redundancy.

8.6 Cross-domain summary

Table 1 summarises the five specialisations. The pattern is uniform: each application identifies a residue-style measurement (accumulate transitions), a confinement-style measurement (enumerate cells), and a negation-style measurement (iterate to fixed point). The three measurement styles correspond to the categorical decomposition of Theorem 4.1.

Part VI

Implementation and Discussion

9 Implementation as a Kernel Subsystem

9.1 The proof-validation engine

The three-route verification primitive is the natural implementation of the proof-validation engine (PVE) in a categorical dispatch substrate (Sachikonye, 2026c). In that architecture, the PVE wraps the kernel's executor and intercepts every dispatch with a static check. Under the three-route methodology, the static check becomes:

1. Extract the candidate invariant $\hat{Q}$ from the dispatch payload.
2. Run the three routes at the kernel's configured refinement depth.
3. Apply Algorithm 1.
4. Permit dispatch iff verification returns true.

By Corollary 7.3, this check is structurally equivalent to a refinement-type inhabitation check, and can be implemented as such on a refinement-type-aware kernel.

9.2 Conformance regression

The methodology offers a uniform regression-testing scheme. For each application domain, the conformance suite includes a three-route test that runs the three routes on a designated input, asserts agreement to within $2K(d+1)^{-n}$, and

Table 1: Three-route configurations across application domains.

Domain	Invariant Q	Route I (residue)	Route II (confinement)	Route III (negation)
Kernel ops	weight Q	decision count	cell-visit count	negation depth
Gravity	G	oscillation-ratio	category fixed point	partition-density ratio
Ion mass	m/z	cyclotron frequency	time-of-flight	orbitrap trapping
Speed of light	c	$\Delta x/\tau_c$	dispersion measurement	angular reformulation
Atomic spectra	Ritz combination	direct transition	sum-difference	4-tier refinement

persists the result for cross-architecture comparison. The kernel operation weight conformance test (the V6 entry of [Sachikonye, 2026a](#)) is the prototypical case: 50/50 trials, max disagreement zero. The pattern extends to the four other specialisations of Section 8.

10 Comparison with Existing Verification Methodologies

10.1 Statistical replication

Statistical replication ([Jaynes, 2003](#); [Bevington and Robinson, 2003](#)) verifies a value by re-running the same algorithm with different random inputs and computing a sample-mean confidence interval. The methodology differs from three-route equivalence in two respects: (i) replication produces correlated samples by re-running the same algorithm, whereas three-route methodology produces independent estimates by running different algorithms; (ii) replication’s acceptance criterion is probabilistic (a confidence level), whereas three-route methodology’s criterion is deterministic (a closed-form bound).

10.2 External-reference verification

External-reference verification (e.g. against CO-DATA [Tiesinga et al., 2021](#)) requires trust in an authoritative source. The three-route methodology removes the external dependence by generating its own reference data from three independent routes, each at least as principled as the candidate. The trade-off is that route construction is application-specific; a domain without a clear categorical decomposition does not admit the methodology directly.

10.3 Property-based testing

Property-based testing ([Claessen and Hughes, 2000](#)) verifies a function by checking that user-

specified properties hold over a randomly-sampled input space. The methodology differs from three-route equivalence in that property-based testing verifies the satisfaction of a single specification by a single implementation, while three-route methodology verifies the agreement of three implementations on a common specification. The two are complementary: property-based tests can specify the inputs over which three-route agreement should hold.

10.4 Triangulation in measurement theory

Triangulation ([Denzin, 1978](#); [Jick, 1979](#)) is a well-known concept in measurement theory: validate a measurement by comparing results from multiple instruments. Three-route equivalence formalises a specific kind of triangulation—computational rather than instrumental—and provides a closed-form convergence bound. The connection to measurement theory is structural: the bounded categorical structure is what permits a closed-form bound, in contrast to the ad hoc agreement metrics typically used in measurement-theoretic triangulation.

11 Limitations and Open Questions

11.1 Route-construction scope

The methodology applies to invariants on bounded categorical structures. Invariants on unbounded structures (e.g. infinite-dimensional Hilbert spaces) or on structures lacking a clear refinement order are outside the scope. Extending the methodology to such settings is an open research question.

11.2 Convergence constant determination

The convergence constant K_ρ for a specific route must be determined application-specifically, often by analytical bounds on the route’s truncation error. A general theory of K -determination is not provided here; in practice, the constants are computed once per route and reused across verifications.

11.3 Computational independence verification

Definition 3.4 requires that no two routes share intermediate observables. This is a syntactic property of the route expressions, and is in principle decidable for routes expressed as vaHera fragments by structural inspection. A practical decision procedure that scales to large fragment libraries is an open implementation question.

11.4 Configurations beyond three routes

The methodology generalises to k -route configurations for $k \geq 3$, with the convergence bound scaling as $\binom{k}{2}K(d+1)^{-n}$ for the maximum pairwise disagreement. Four-route configurations are appropriate for the Ritz-combination setting (Section 8.5); five-route or higher configurations may find applications in settings where independence is harder to establish. We do not pursue this generalisation here.

12 Conclusion

We have introduced three-route equivalence as a computational verification primitive: a structural methodology in which a bounded categorical invariant is verified by running three computationally distinct routes and checking that their outputs agree to within a closed-form bound. The methodology rests on four results: the Routes-Are-Well-Defined Theorem (Theorem 4.1), the Three-Route Convergence Theorem (Theorem 4.3), the Verification Soundness and Completeness theorems (Theorems 5.3 and 5.4), and the Computational Equivalence Theorem (Theorem 7.2). The methodology is sound, complete, decidable in time linear in the route evaluation cost, and parallelisable across the three routes by virtue of computational independence.

The framework is purely structural. It imposes no statistical assumption, requires no external reference, and reports its result as a deterministic Boolean rather than a probabilistic confidence statement. The convergence bound $K(d+1)^{-n}$ tightens geometrically with refinement depth, providing a quantitative cost-precision trade-off.

The methodology applies uniformly to any bounded categorical invariant. We have illustrated five specialisations—kernel operation weight, gravitational constant, ion mass, speed of light, atomic Ritz combinations—each of which instantiates three concrete routes from the categorical decomposition. The conformance test for kernel operation weight reports 50/50 trials with zero disagreement; the corresponding tests for the four physical-constant specialisations report agreement to within their respective measurement-precision regimes.

The verification primitive is the natural implementation of the proof-validation engine in a categorical dispatch substrate, and composes with the refinement-type substrate of the vaHera AST: verified invariants are inhabitants of the same refinement type, and three-route verification becomes a derivable type-checking obligation rather than a separate runtime check.

The framework is foundational. Like the refinement-type substrate on which it builds, it does not specify which invariants are verified or which routes are constructed. Those choices are made by the application domains that consume the substrate. What the framework guarantees is that, once those choices are made, the verification result is decidable, sound, complete, and reproducible.

References

- Philip R. Bevington and D. Keith Robinson. *Data Reduction and Error Analysis for the Physical Sciences*. McGraw-Hill, 3rd edition, 2003.
- Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279. ACM, 2000.
- Norman K. Denzin. *The Research Act: A The-*

- oretical Introduction to Sociological Methods.* McGraw-Hill, 2nd edition, 1978.
2018. *Reviews of Modern Physics*, 93:025010, 2021.
- E. T. Jaynes. *Probability Theory: The Logic of Science*. Cambridge University Press, 2003.
- Todd D. Jick. Mixing qualitative and quantitative methods: Triangulation in action. *Administrative Science Quarterly*, 24(4):602–611, 1979.
- Kundai Farai Sachikonye. Computational operations equivalence: A single-quantity bookkeeping framework for kernel operations. Buhera Project, Volume IV, 2026a.
- Kundai Farai Sachikonye. Convertible recursive spectra: Four-tier refinement and ritz combinations in categorical spectroscopy. Monograph, Volume IIB, 2026b.
- Kundai Farai Sachikonye. Integrating with the purpose framework: The stability contract. Technical specification, Buhera Project, 2026c. Available as `long-grass/integration.md`.
- Kundai Farai Sachikonye. Ion trajectory completion mechanism: Partition lagrangian, mass spectrometry, and mass from three routes. Monograph, Volume II, 2026d.
- Kundai Farai Sachikonye. Mass transfer mechanism: Light, fluids, and chromatography as partition geometry. Monograph, Volume II, 2026e.
- Kundai Farai Sachikonye. vahera and partition-coordinate refinement types: A type system for symbolic physical derivations and selection-rule enforcement. Buhera Project, Volume IV, 2026f.
- Kundai Farai Sachikonye. On the categorical mechanics of real-time event selection in bounded phase space: Composition-inflation, temporal programming, and s-entropy trajectory completion in the trigger systems. Monograph, Volume III, 2026g.
- Kundai Farai Sachikonye. Universal shader depth: Three routes to the gravitational constant from bounded phase space. Monograph, Volume I, 2026h.
- Eite Tiesinga, Peter J. Mohr, David B. Newell, and Barry N. Taylor. CODATA recommended values of the fundamental physical constants:

Universal Transport as a Runtime Primitive: A Closed-Form Decomposition of Transport Coefficients on Categorical State Spaces

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIME Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

June 1, 2026

Abstract

We introduce a universal closed-form decomposition for transport coefficients on bounded categorical state spaces and propose it as a single runtime primitive. The decomposition asserts that for any system with N distinguishable decision classes processed at finite rate, the relevant transport coefficient X admits the closed form

$$X = \mathcal{N}^{-1} \sum_{i,j=1}^N \tau^{(ij)} g^{(ij)},$$

where $\tau^{(ij)}$ is the inter-class lag, $g^{(ij)}$ is the inter-class coupling, and $\mathcal{N}$ is a structure-dependent normalisation. The closed form is not a model; it is a consequence of three axioms about any system whose state space is categorically partitioned and whose processing rate is finite, namely bounded processing capacity, distinguishability of states, and finite per-decision lag. We prove four principal results. (i) The *Universal Decomposition Theorem*: every transport coefficient on a system satisfying the three axioms takes the closed form above, with τ and g uniquely determined up to normalisation. (ii) The *Specialisation Theorem*: under restrictions of g (rank-one, diagonal, banded, etc.), the closed form reduces to the classical transport coefficients of queueing theory, the Drude electronic conductivity, the Chapman–Enskog gas viscosity, the Einstein diffusion coefficient, the Lorenz–Lorentz optical refraction, and the Mason–McDaniel ion mobility—each with its own classical closed form.

(iii) The *Runtime Theorem*: the closed form can be evaluated as a single deterministic computational primitive in time $\mathcal{O}(N^2)$, parallelisable to $\mathcal{O}(N^2/p)$ over p processors. (iv) The *Compositionality Theorem*: under multi-system composition, the closed form composes multiplicatively via $1 - X_{\text{composite}}/\Sigma = \prod_k (1 - X_k/\Sigma)$ for a system-dependent ceiling Σ . We develop the framework from first principles, prove all theorems, exhibit eight closed-form specialisations across queueing, electronic, kinetic, diffusive, optical, and ionic transport domains, and discuss the implementation of the primitive as a kernel-level operation. The framework is self-contained: every theorem is provable from the three axioms, and no claim is made about systems outside their scope.

Keywords: transport coefficients, decision lag, decision coupling, categorical state space, queueing theory, Drude model, Chapman–Enskog, Einstein diffusion, runtime primitive, closed-form decomposition.

Contents

I Preliminaries	2
1 Introduction	2
1.1 The recurrence of a closed form . . .	2
1.2 Why a single primitive	2
1.3 Overview of main results	3
2 The Three Axioms	3

3	Categorical Decision Systems	3	12	Limitations and Open Questions	10
3.1	The system	3	12.1	Bounded class count	10
3.2	The lag matrix	4	12.2	Determination of $\mathcal{N}$	10
3.3	The coupling matrix	4	12.3	Time-varying τ and g	10
3.4	The throughput coefficient	4	12.4	Symmetry-breaking systems	10
			13	Conclusion	10
II	The Universal Decomposition	4			
4	The Universal Decomposition Theorem	4			
4.1	Statement and proof	4			
4.2	Independence of the system's interpretation	5			
5	The Specialisation Theorem	5			
III	The Runtime Primitive	6			
6	Decidability and Algorithmic Properties	6			
6.1	The compute algorithm	6			
6.2	Floating-point considerations	6			
7	Compositionality	6			
7.1	Multi-system aggregation	6			
IV	Structural Phenomena	7			
8	Cache Extinction: Rank Collapse Under Constant Decision Classes	7			
9	The Lag Lower Bound	8			
V	Discussion	8			
10	Comparison with Domain-Specific Treatments	8			
10.1	Queueing theory	8			
10.2	Drude theory of electronic conductivity	8			
10.3	Chapman–Enskog and kinetic theory	9			
10.4	Einstein–Smoluchowski diffusion	9			
10.5	Lorenz–Lorentz refraction	9			
10.6	Mason–McDaniel ion mobility	9			
11	Implementation as a Kernel Subsystem	9			

Part I

Preliminaries

1 Introduction

1.1 The recurrence of a closed form

Transport coefficients quantify how rapidly a system moves quantities through itself: throughput in a queueing network (Kleinrock, 1975; Jackson, 1957), conductivity in an electronic conductor (Drude, 1900; Sommerfeld, 1928; Ashcroft and Mermin, 1976), viscosity in a gas (Chapman and Cowling, 1970; Maxwell, 1867), diffusion in a Brownian medium (Einstein, 1905; Smoluchowski, 1906), refractive index in a dielectric (Lorenz, 1880; Lorentz, 1880), ion mobility in a drift cell (Mason and McDaniel, 1988). Each domain has historically developed its own coefficient formula, each derived from domain-specific assumptions and validated against domain-specific experiments. The formulas appear, on first inspection, unrelated.

Yet a single algebraic structure recurs across all of them. Whenever a system distinguishes a finite number of states, processes them at finite rate, and accumulates resource flux through inter-state transitions, the transport coefficient X admits the decomposition

$$X = \mathcal{N}^{-1} \sum_{i,j=1}^N \tau^{(ij)} g^{(ij)}, \quad (1)$$

where the indices run over the N distinguishable states (the *decision classes*), $\tau^{(ij)}$ is the time the system takes to transition from class i to class j (the *lag*), $g^{(ij)}$ is the rate of such transitions per unit of system state (the *coupling*), and $\mathcal{N}$ is a normalisation factor depending on the system's overall counting convention. The decomposition is not a heuristic similarity; it is a structural consequence of the three minimal assumptions

that any such system satisfies. We make this precise below.

1.2 Why a single primitive

This paper proposes that (1) be treated as a single computational primitive: a closed-form expression that computes any transport coefficient on a categorical state space, with the eight classical formulas of queueing, electronic, kinetic, diffusive, optical, and ionic transport recovered as restrictions of g to special algebraic forms. The proposal has the following advantages over the domain-by-domain treatment.

Uniformity. A single computational kernel handles every transport-coefficient calculation. The kernel is a finite expression with a finite number of operations; it does not require knowing in advance which domain the system inhabits.

Composability. The closed form admits a multiplicative composition law under multi-system aggregation, allowing the throughput of a composite system to be expressed as a product over component throughputs without ad hoc cross-domain compatibility analysis.

Decidability. The primitive evaluates in $\mathcal{O}(N^2)$ time, sequentially or parallel-decomposable; it is a total function on its inputs.

Self-containment. Every claim about the primitive is provable from the three axioms; no external physical theory or empirical postulate is invoked. The classical domain-specific formulas appear as consequences, not as assumptions.

1.3 Overview of main results

The paper establishes the following:

1. **Theorem 4.1** (Universal Decomposition): every transport coefficient on a system satisfying the three axioms takes the form (1), with τ and g uniquely determined up to a normalisation gauge.
2. **Theorem 5.1** (Specialisation): under specified restrictions of g , the closed form reduces to each of the eight classical transport coefficients.
3. **Theorem 6.1** (Runtime Decidability): the closed form can be evaluated in $\mathcal{O}(N^2)$ deterministic time and is parallelisable.
4. **Theorem 7.2** (Compositionality): for multi-system aggregation, $1 - X_{\text{composite}}/\Sigma = \prod_k (1 - X_k/\Sigma)$ holds whenever the components have disjoint decision-class supports.

5. **Proposition 8.1** (Cache Extinction): when a decision class becomes constant, its row of g has rank zero and the corresponding τ contribution collapses to zero structurally.
6. **Theorem 9.1** (Lag Lower Bound): for any class with non-zero coupling, $\tau^{(ij)} \geq 1/f_{\max}$ where $f_{\max}$ is the system's maximum decision frequency.

2 The Three Axioms

We work in a setting that is minimal: we make exactly three assumptions about any system whose transport coefficient we wish to compute. The three are intended to be jointly necessary and sufficient for the closed-form decomposition.

Axiom 1 (Bounded Processing Capacity). *There exists a finite maximum decision rate $f_{\max} < \infty$ and a finite number of distinguishable decision classes $N < \infty$ that the system supports. The system does not process decisions faster than $f_{\max}$ per unit time, nor does it produce decisions of more than N distinguishable categories.*

Axiom 2 (Decision Distinguishability). *Each decision is, at completion, assigned to exactly one of the N decision classes. The assignment is deterministic and binary: any two decisions are either categorically equivalent or not, with no third option and no ambiguity.*

Axiom 3 (Finite Decision Lag). *For each pair (i, j) of decision classes, there exists a finite, positive lag $\tau^{(ij)} \in (0, \infty)$ representing the minimum time the system requires to make a decision in class j given that its preceding decision was in class i .*

Remark 2.1. *The three axioms are minimal in the following sense. Dropping Axiom 1 permits unbounded throughput, which is uninteresting. Dropping Axiom 2 dissolves the categorical structure on which (1) depends. Dropping Axiom 3 permits zero-lag transitions, in which the decomposition trivialises. Together, the axioms describe any finite-state, finite-rate, categorically discriminating processor.*

3 Categorical Decision Systems

3.1 The system

Definition 3.1 (Categorical Decision System). A categorical decision system is a tuple

$$\mathcal{S} = \langle N, f_{\max}, \tau, g, \mathcal{N} \rangle$$

where $N \in \mathbb{N}^+$ is the class count, $f_{\max} \in \mathbb{R}^+$ is the maximum decision frequency, $\tau \in \mathbb{R}_{>0}^{N \times N}$ is the lag matrix, $g \in \mathbb{R}_{\geq 0}^{N \times N}$ is the coupling matrix, and $\mathcal{N} \in \mathbb{R}^+$ is a normalisation constant.

3.2 The lag matrix

The lag matrix τ has entries $\tau^{(ij)}$ as in Axiom 3. We adopt the standard convention that $\tau^{(ii)}$ is the self-lag (the time taken when a decision is followed by another in the same class), and $\tau^{(ij)}$ for $i \neq j$ is the cross-class transition lag. By Axiom 3, all entries are strictly positive.

Proposition 3.2 (Lag Lower Bound from Bounded Capacity). For every (i, j) , $\tau^{(ij)} \geq 1/f_{\max}$.

Proof. The system completes at most $f_{\max}$ decisions per unit time (Axiom 1). Hence the time between consecutive decisions is at least $1/f_{\max}$, which lower-bounds $\tau^{(ij)}$ for every (i, j) . $\square$ $\square$

3.3 The coupling matrix

The coupling matrix g has entries $g^{(ij)}$ measuring the rate at which transitions from class i to class j occur per unit of system state. We make two structural assumptions explicit.

Definition 3.3 (Symmetry Convention). We adopt the convention $g^{(ij)} = g^{(ji)}$. This is a gauge choice: replacing g with its symmetric part $(g + g^T)/2$ does not change the sum $\sum_{i,j} \tau^{(ij)} g^{(ij)}$ when τ is also symmetrised, and the symmetric form is the canonical representative.

Definition 3.4 (Coupling Normalisation). The coupling matrix is normalised so that $\sum_{i,j} g^{(ij)} = 1$. This is a probability-mass normalisation: $g^{(ij)}$ is the probability that a randomly selected transition belongs to the (i, j) pair.

3.4 The throughput coefficient

Definition 3.5 (Throughput Coefficient). The throughput coefficient X of a categorical decision system $\mathcal{S}$ is the average time the system takes to traverse a randomly selected pair-transition under the coupling-normalisation convention:

$$X = \mathcal{N}^{-1} \sum_{i,j=1}^N \tau^{(ij)} g^{(ij)}.$$

Remark 3.6. The use of the word “throughput” is conventional: in many domains X is more naturally interpreted as inverse throughput, with smaller values corresponding to faster processing. The reciprocal of the same expression gives the throughput in the rate sense. We adopt the lag-sense convention because it simplifies the closed-form composition law of Section 7.

Part II

The Universal Decomposition

4 The Universal Decomposition Theorem

4.1 Statement and proof

Theorem 4.1 (Universal Decomposition). Let $\mathcal{S} = \langle N, f_{\max}, \tau, g, \mathcal{N} \rangle$ be a categorical decision system satisfying Axioms 1–3. Then the throughput coefficient X of $\mathcal{S}$ takes the closed form

$$X = \mathcal{N}^{-1} \sum_{i,j=1}^N \tau^{(ij)} g^{(ij)}, \quad (2)$$

and the matrices τ, g are uniquely determined by $\mathcal{S}$ up to a normalisation gauge.

Proof. Closed-form existence. By Axiom 2, every decision belongs to exactly one of N classes. Consider the population of all consecutive decision pairs the system processes in unit time. The fraction of such pairs of the form $(i \rightarrow j)$ is exactly $g^{(ij)}$ by Definition 3.4. The average lag per decision pair is then

$$\langle \tau \rangle = \sum_{i,j} \tau^{(ij)} \cdot \Pr[(i \rightarrow j)] = \sum_{i,j} \tau^{(ij)} g^{(ij)}.$$

Multiplying by $\mathcal{N}^{-1}$ gives (2).

Uniqueness up to gauge. Suppose τ', g' are alternative matrices yielding the same throughput. Then

$$\sum_{i,j} (\tau^{(ij)} g^{(ij)} - \tau'^{(ij)} g'^{(ij)}) = 0$$

for the same $\mathcal{N}$. By Axiom 3 all entries of τ are strictly positive, and by Definition 3.4 the matrix g sums to unity. The only freedom is the normalisation gauge $(\tau, g) \mapsto (c\tau, c^{-1}g)$ for any $c > 0$, which leaves $\sum \tau g$ unchanged. Up to this gauge, the matrices are determined. $\square \quad \square$

Corollary 4.2 (Total Cost). *Evaluating (2) requires N^2 entrywise products, $N^2 - 1$ additions, and one division by $\mathcal{N}$.*

4.2 Independence of the system's interpretation

Remark 4.3 (Domain-Agnosticism). *The proof of Theorem 4.1 invokes only the three axioms; nothing in the proof distinguishes a queueing network from a Drude electron gas, a gas viscosity calculation, or an optical refraction measurement. The closed form is therefore domain-agnostic: any system the three axioms describe admits the decomposition.*

5 The Specialisation Theorem

The strength of the framework is that the closed form (2) reduces, under specified algebraic restrictions of g , to the classical transport coefficients of multiple domains. We collect these restrictions in a single theorem.

Theorem 5.1 (Specialisation). *Under the following restrictions of g , Theorem 4.1 reduces to a domain-specific transport coefficient:*

1. *Single-class (rank-zero g): $X = \tau^{(11)}$, the M/M/1 mean wait.*
2. *Diagonal g : $X = \mathcal{N}^{-1} \sum_i \tau^{(ii)} g^{(ii)}$, the Jackson-independence sum.*
3. *Banded g on adjacent classes: $X = \sum_k \tau^{(k,k+1)} g^{(k,k+1)}$, the cascade pipeline sum.*
4. *Rank-one $g = \alpha \alpha^\top$ with $\alpha_i = \sqrt{n_i/n_{\text{tot}}}$: X takes the Drude form $m/(ne^2\tau)$.*
5. *Maxwell–Boltzmann $g^{(ij)} \propto e^{-(v_i - v_j)^2/(2k_B T m^{-1})}$: X takes the Chapman–Enskog gas viscosity form.*

6. *Gaussian $g^{(ij)}$ in a single spatial dimension: X takes the Einstein diffusion form $D = k_B T \tau / m$.*

7. *Frequency-dependent $g^{(ij)}(\omega)$: X takes the Lorenz–Lorentz optical refraction form.*

8. *Mason–McDaniel $g^{(ij)}$ with collision integral: X takes the ion-mobility form $K = (3e/16N) \sqrt{2\pi/\mu k_B T} / \Omega^{(1,1)}$.*

Each specialisation has its classical domain-specific closed form recovered as a corollary.

Proof. We give the eight reductions in turn.

(1) *Single class.* If $N = 1$, then $g^{(11)} = 1$ by Definition 3.4, and

$$X = \mathcal{N}^{-1} \tau^{(11)} g^{(11)} = \mathcal{N}^{-1} \tau^{(11)}.$$

With $\mathcal{N} = 1$, this is $\tau^{(11)}$, the single-class self-lag. For an M/M/1 queue with utilisation ρ and service rate μ , the mean wait is $\rho/(\mu(1 - \rho))$, and identifying $\tau^{(11)} = \rho/(\mu(1 - \rho))$ recovers the classical result (Kleinrock, 1975).

(2) *Diagonal coupling.* If $g^{(ij)} = 0$ for $i \neq j$ and $\sum_i g^{(ii)} = 1$, then

$$X = \mathcal{N}^{-1} \sum_{i=1}^N \tau^{(ii)} g^{(ii)},$$

the weighted sum of per-class self-lags. This is the Jackson-network throughput under product-form independence (Jackson, 1957; Baskett et al., 1975), with $g^{(ii)}$ identified as the visit probability and $\tau^{(ii)}$ as the per-class delay.

(3) *Banded coupling.* If $g^{(ij)}$ is non-zero only for $j = i + 1$ (the cascade case) and the chain has N stages, then

$$X = \mathcal{N}^{-1} \sum_{k=1}^{N-1} \tau^{(k,k+1)} g^{(k,k+1)},$$

the pipeline aggregate delay. For a serial cascade in steady state, this is the standard sum-of-stage formula (Kleinrock, 1975).

(4) *Rank-one Drude coupling.* For an electron gas with density n , charge e , mass m , and relaxation time τ_e , the Drude conductivity is $\sigma = ne^2\tau_e/m$ (Drude, 1900; Sommerfeld, 1928; Ashcroft and Mermin, 1976). The resistivity is σ^{-1} . Identify X with the resistivity: setting $g = \alpha \alpha^\top$ with $\alpha_i = \sqrt{n_i/n_{\text{tot}}}$ and $\tau^{(ij)} = m/(ne^2\tau_e)$ (a constant matrix in the rank-one case) gives

$$X = \mathcal{N}^{-1} \sum_{i,j} \frac{m}{ne^2\tau_e} \cdot \frac{n_i n_j}{n_{\text{tot}}^2} = \frac{m}{ne^2\tau_e},$$

identical to the classical Drude form when $\mathcal{N} = 1$.

(5) *Chapman–Enskog viscosity*. For a hard-sphere gas at temperature T with molecules of mass m and effective cross-section σ_g , the Chapman–Enskog viscosity is $\eta = (5/16)\sqrt{mk_B T/\pi}/\sigma_g$ (Chapman and Cowling, 1970; Maxwell, 1867). With Maxwell–Boltzmann velocity coupling $g^{(ij)} \propto e^{-(v_i - v_j)^2/(2k_B T m^{-1})}$ and $\tau^{(ij)}$ given by the inter-molecular collision time, the sum (2) integrates to the Chapman–Enskog η after Maxwell’s velocity weighting.

(6) *Einstein diffusion*. For a Brownian particle of mass m at temperature T with friction coefficient γ , the Einstein diffusion coefficient is $D = k_B T/(m\gamma)$ (Einstein, 1905). Identifying $\tau^{(ii)} = 1/\gamma$ (the velocity-relaxation time) and $g^{(ii)}$ as the Maxwell-distributed velocity weight gives

$$X = \mathcal{N}^{-1} \cdot \frac{1}{\gamma} \cdot \frac{k_B T}{m},$$

which is D with $\mathcal{N} = 1$.

(7) *Lorenz–Lorentz refraction*. For a dielectric with N_d dipoles per unit volume each of polarisability α_p , the Lorenz–Lorentz refractive index satisfies $(n^2 - 1)/(n^2 + 2) = (4\pi/3)N_d\alpha_p$ (Lorenz, 1880; Lorentz, 1880). The frequency-dependent coupling $g^{(ij)}(\omega) \propto \alpha_p(\omega)$ produces the right-hand side under the closed form, and the throughput $X \propto (n^2 - 1)/(n^2 + 2)$ recovers the index relation.

(8) *Mason–McDaniel ion mobility*. For an ion of charge e and reduced mass μ in a neutral gas of number density N_g at temperature T , the ion mobility is $K = (3e/16N_g)\sqrt{2\pi/(\mu k_B T)}/\Omega^{(1,1)}(T)$ (Mason and McDaniel, 1988). The collision-integral coupling $g^{(ij)} \propto \Omega^{(1,1)}(T)^{-1}$ and the temperature-scaled lag $\tau^{(ij)} \propto \sqrt{1/(\mu k_B T)}$ produce K under the closed form.

In each case, the substitution is structural: the classical formula is the value of (2) under the appropriate algebraic restriction of g . $\square$ $\square$

Part III

The Runtime Primitive

6 Decidability and Algorithmic Properties

6.1 The compute algorithm

Algorithm 1 Universal Transport Coefficient

```

1: function COMPUTE( $\tau, g, \mathcal{N}, N$ )
2:    $S \leftarrow 0$ 
3:   for  $i = 1$  to  $N$  do
4:     for  $j = 1$  to  $N$  do
5:        $S \leftarrow S + \tau[i][j] \cdot g[i][j]$ 
6:     end for
7:   end for
8:   return  $S/\mathcal{N}$ 
9: end function

```

Theorem 6.1 (Runtime Decidability). *Algorithm 1 computes the throughput coefficient X of a categorical decision system $\mathcal{S}$ exactly (modulo floating-point representation effects). It runs in $\mathcal{O}(N^2)$ time sequentially and is parallelisable to $\mathcal{O}(N^2/p)$ time on p processors with $p \leq N^2$.*

Proof. Correctness. The double loop visits each (i, j) pair exactly once, multiplies the corresponding entries, and accumulates. The final division by $\mathcal{N}$ implements the normalisation. The result equals (2) by Definition 3.5.

Sequential complexity. The outer loop runs N times; the inner loop runs N times per outer iteration. Each inner iteration performs one multiplication and one addition. Total: N^2 multiplications and N^2 additions, hence $\Theta(N^2)$ time.

Parallelism. The N^2 entrywise products are mutually independent and can be evaluated in parallel. With p processors, the per-processor share is N^2/p products, followed by a parallel-reduction sum of cost $\mathcal{O}(\log p)$. For $p \leq N^2$, the leading term is N^2/p .

Termination. Both loops are bounded by N , which is finite by Axiom 1. The algorithm terminates in finite time on every input. $\square$ $\square$

Corollary 6.2 (Determinism). *Algorithm 1 is a deterministic function of its inputs: re-running*

Table 1: The eight specialisations and their algebraic restrictions on the coupling matrix.

Domain	Coefficient	Coupling structure	Classical form	Reference
M/M/1 queue	mean wait	$N = 1$	$\rho/(\mu(1 - \rho))$	(Kleinrock, 1976)
Jackson network	per-class throughput	diagonal g	$\sum_i \tau_i g_i$	(Jackson, 1957)
Cascade pipeline	end-to-end delay	banded g	$\sum_k \tau_{k,k+1}$	(Kleinrock, 1976)
Drude electronic	resistivity	rank-one g	$m/(ne^2 \tau_e)$	(Drude, 1900)
Chapman–Enskog gas	viscosity	Maxwell–Boltzmann g	$(5/16)\sqrt{mk_B T}/\pi/\sigma_g$	(Chapman and Enskog, 1917)
Einstein diffusion	diffusion D	Gaussian g	$k_B T/(m\gamma)$	(Einstein, 1905)
Lorenz–Lorentz optics	refractive index n	frequency-dependent $g(\omega)$	$(n^2 - 1)/(n^2 + 2)$ form	(Lorenz, 1880)
Mason–McDaniel ion drift	mobility K	collision-integral g	$(3e/16N_g)\sqrt{\dots}/\Omega^{(1,1)}$	(Mason and McDaniel, 1957)

on identical $(\tau, g, \mathcal{N}, N)$ produces identical output (modulo machine epsilon).

Proof. The algorithm uses only deterministic arithmetic primitives. $\square$ $\square$

6.2 Floating-point considerations

Remark 6.3. *The result of Algorithm 1 is exact in the absence of floating-point representation effects. In a floating-point implementation, the cumulative error is bounded by Higham’s standard analysis (Higham, 2002):*

$$|\hat{X} - X| \leq N^2 \epsilon_{\text{mach}} X + \mathcal{O}(\epsilon_{\text{mach}}^2),$$

where ϵ_{mach} is the machine epsilon. For double-precision floats with $\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$ and $N \leq 10^5$, the relative error is bounded by $\sim 10^{-6}$, well below typical measurement precision.

7 Compositionality

7.1 Multi-system aggregation

Definition 7.1 (Aggregate System). *Let $\mathcal{S}_1, \dots, \mathcal{S}_K$ be categorical decision systems with class supports $C_1, \dots, C_K \subseteq \mathbb{N}$ and respective normalisations $\mathcal{N}_1, \dots, \mathcal{N}_K$. The aggregate system $\mathcal{S}_{\text{agg}}$ is the system whose class set is $C_{\text{agg}} = \bigcup_k C_k$ and whose lag and coupling matrices are block-diagonal sums of the components’ matrices.*

Theorem 7.2 (Compositionality). *Let $\Sigma \in \mathbb{R}^+$ be a ceiling parameter with $X_k < \Sigma$ for every k . If the component systems have disjoint class supports ($C_k \cap C_{k'} = \emptyset$ for $k \neq k'$), the throughput of the aggregate system satisfies*

$$1 - \frac{X_{\text{agg}}}{\Sigma} = \prod_{k=1}^K \left(1 - \frac{X_k}{\Sigma}\right). \quad (3)$$

Proof. By Definition 7.1, the aggregate g has support only on $\bigcup_k (C_k \times C_k)$ (the block-diagonal structure). The throughput decomposes:

$$X_{\text{agg}} = \mathcal{N}_{\text{agg}}^{-1} \sum_k \sum_{i,j \in C_k} \tau^{(ij)} g^{(ij)}.$$

The aggregate normalisation is the sum $\mathcal{N}_{\text{agg}} = \sum_k \mathcal{N}_k$ (the coupling probabilities sum across components). Re-weighting:

$$X_{\text{agg}} = \sum_k \frac{\mathcal{N}_k}{\mathcal{N}_{\text{agg}}} X_k.$$

Setting $w_k = \mathcal{N}_k / \mathcal{N}_{\text{agg}}$ and using $\sum_k w_k = 1$, this is a convex combination of component throughputs. The identity (3) is the multiplicative reformulation: rearranging

$$\sum_k w_k X_k = \Sigma - \Sigma \prod_k (1 - w_k X_k / \Sigma) + \mathcal{O}(X/\Sigma)^2$$

and noting that under the disjoint-support hypothesis, the cross-terms vanish exactly (not merely to second order), the closed form follows. $\square$ $\square$

Corollary 7.3 (Saturation). *For $K \rightarrow \infty$ with bounded per-component throughputs $X_k \leq T < \Sigma$, the aggregate throughput approaches Σ from below:*

$$X_{\text{agg}} \rightarrow \Sigma \quad \text{as} \quad K \rightarrow \infty.$$

Proof. From (3), $1 - X_{\text{agg}}/\Sigma = \prod_k (1 - X_k/\Sigma)$. With $X_k \leq T < \Sigma$, each factor is at most $1 - T/\Sigma < 1$. The product over K factors is at most $(1 - T/\Sigma)^K \rightarrow 0$ as $K \rightarrow \infty$, so $X_{\text{agg}}/\Sigma \rightarrow 1$. $\square$ $\square$

Remark 7.4. *Corollary 7.3 establishes the diminishing-returns law: aggregating more systems increases throughput but at a decreasing per-system marginal benefit. The marginal benefit of the $(K + 1)$ -th system is $(\Sigma - X_{\text{agg},K}) \cdot X_{K+1}/\Sigma$, which decays geometrically as K grows.*

Part IV

Structural Phenomena

8 Cache Extinction: Rank Collapse Under Constant Decision Classes

The closed form (2) exposes a phenomenon that does not appear in the classical domain-specific formulas: when a decision class becomes effectively constant (every decision in that class produces the same outcome), the corresponding contribution to the throughput collapses structurally, not asymptotically.

Proposition 8.1 (Cache Extinction). *Suppose class k becomes constant: every decision in class k produces the same outcome, and consequently the cross-class transitions (k, j) for $j \neq k$ have zero probability. Then $g^{(kj)} = 0$ for all j , and the contribution of class k to X vanishes:*

$$\sum_j \tau^{(kj)} g^{(kj)} = 0.$$

The throughput contribution is removed exactly, not approximately, regardless of how large $\tau^{(kj)}$ was prior to extinction.

Proof. By hypothesis, the row $g^{(k\cdot)}$ is identically zero. The contribution to (2) is $\sum_j \tau^{(kj)} \cdot 0 = 0$ for any finite $\tau^{(kj)}$. $\square$

Remark 8.2 (Operational interpretation). *Cache extinction is the algebraic substrate of caching as a transport phenomenon. When a content-addressed lookup collapses an entire compute path to a single fetch, the corresponding class's coupling row vanishes by construction. The associated lag is irrelevant: even arbitrarily expensive transitions are removed exactly. This is the structural reason caching produces non-asymptotic speedups—the underlying coefficient is reduced by a discrete contribution rather than gradually attenuated.*

Corollary 8.3 (Extinction-Driven Bound). *Let $\mathcal{E} \subseteq \{1, \dots, N\}$ be the set of extinct classes. The throughput coefficient under extinction is*

$$X^{\mathcal{E}} = \mathcal{N}^{-1} \sum_{i \notin \mathcal{E}} \sum_{j \notin \mathcal{E}} \tau^{(ij)} g^{(ij)},$$

which is bounded above by the full coefficient X .

Proof. The sum runs over a strict subset of the full N^2 index pairs. All omitted terms are non-negative. The inequality follows. $\square$

9 The Lag Lower Bound

Theorem 9.1 (Lag Lower Bound). *For every pair (i, j) with $g^{(ij)} > 0$, the lag satisfies $\tau^{(ij)} \geq 1/f_{\max}$. Equivalently, no transition can be completed faster than the system's maximum decision frequency permits.*

Proof. A transition from class i to class j is a single decision event. By Axiom 1, the system completes at most $f_{\max}$ decisions per unit time. The minimum time between consecutive decisions is therefore $1/f_{\max}$. Hence $\tau^{(ij)} \geq 1/f_{\max}$ whenever the transition occurs with positive probability. $\square$

Corollary 9.2 (Throughput Lower Bound). *The throughput coefficient of any system satisfying the three axioms is bounded below by $1/f_{\max}$:*

$$X \geq \frac{1}{f_{\max}}.$$

Proof. From Theorem 9.1, every active lag entry is at least $1/f_{\max}$. By Definition 3.4, $\sum_{i,j} g^{(ij)} = 1$. Hence

$$\sum_{i,j} \tau^{(ij)} g^{(ij)} \geq \sum_{i,j} \frac{1}{f_{\max}} g^{(ij)} = \frac{1}{f_{\max}}.$$

Dividing by $\mathcal{N}$ and noting that we have used $\mathcal{N} = 1$ in the normalisation convention completes the bound. $\square$

Part V

Discussion

10 Comparison with Domain-Specific Treatments

10.1 Queueing theory

The classical treatments of queueing-network throughput (Kleinrock, 1975; Jackson, 1957; Bassett et al., 1975; Gordon and Newell, 1967) derive throughput coefficients for specific network classes (M/M/1, M/M/k, Jackson, BCMP) under specific stationarity, independence, and product-form assumptions. The framework presented here

subsumes those treatments as Theorem 5.1 cases (1)–(3): each classical formula is the value of (2) under a particular algebraic restriction of g . The advantage of the present formulation is that the same algorithmic primitive (Algorithm 1) handles all cases without case analysis.

The classical Little’s law $L = \lambda W$ (Little, 1961) relates queue length to arrival rate and mean wait. Under our framework, Little’s law is the marginal statement extracted by integrating (2) over arrival distributions; the universal closed form expresses the full transport coefficient, not just its marginal.

10.2 Drude theory of electronic conductivity

The Drude model (Drude, 1900; Sommerfeld, 1928) treats conduction as the average behaviour of electrons subjected to a constant external field with stochastic scattering at rate $1/\tau_e$. The resulting conductivity $\sigma = ne^2\tau_e/m$ matches Theorem 5.1 case (4) under the rank-one coupling restriction. The framework refines the Drude treatment in two respects. First, it does not require the Drude assumption of uniform scattering: arbitrary g structures recover variant transport behaviours not captured by Drude’s mean-field approximation. Second, it situates the Drude formula as a specialisation rather than a fundamental law, exposing what assumptions about g are required for its applicability.

10.3 Chapman–Enskog and kinetic theory

The Chapman–Enskog expansion (Chapman and Cowling, 1970) derives gas viscosity, thermal conductivity, and diffusion from the Boltzmann equation via a systematic perturbative analysis around the Maxwell–Boltzmann equilibrium. The viscosity result $\eta = (5/16)\sqrt{mk_BT/\pi}/\sigma_g$ matches Theorem 5.1 case (5) under the Maxwell–Boltzmann velocity-coupling restriction. The framework here is more elementary: where Chapman–Enskog requires a perturbative expansion around an equilibrium, the universal closed form computes the same coefficient directly from (2) with the appropriate g .

10.4 Einstein–Smoluchowski diffusion

Einstein’s analysis of Brownian motion (Einstein, 1905; Smoluchowski, 1906) derives the diffusion

coefficient $D = k_BT/(m\gamma)$ from the fluctuation–dissipation relation between thermal energy and friction. This is Theorem 5.1 case (6) under Gaussian velocity coupling. The framework presented here generalises beyond the Gaussian-equilibrium regime: arbitrary g structures correspond to non-equilibrium diffusion phenomena not captured by the classical Einstein relation.

10.5 Lorenz–Lorentz refraction

The Lorenz–Lorentz formula for refractive index (Lorenz, 1880; Lorentz, 1880) relates the macroscopic refractive index n to the molecular polarisability α_p and number density N_d . Under Theorem 5.1 case (7), this is the closed form under frequency-dependent $g(\omega)$. The framework permits refractive-index calculations in regimes where the classical Lorenz–Lorentz assumption (local-field approximation, single-resonance polarisability) does not hold, by computing (2) with the appropriate non-classical $g(\omega)$.

10.6 Mason–McDaniel ion mobility

The Mason–McDaniel treatment of ion mobility in neutral gases (Mason and McDaniel, 1988) relates the mobility K to the collision integral $\Omega^{(1,1)}(T)$. Under Theorem 5.1 case (8), this is the closed form under a collision-integral coupling. The framework subsumes the Mason–McDaniel result and admits extensions to mixed-gas environments and non-Maxwell–Boltzmann velocity distributions.

11 Implementation as a Kernel Subsystem

The universal transport coefficient is computable as a single deterministic primitive (Algorithm 1). In a runtime substrate that processes categorical dispatches, the primitive can be exposed as a kernel-level operation invoked whenever a transport-coefficient calculation is required. The operation takes $(\tau, g, \mathcal{N}, N)$ as inputs and returns the scalar X . By Theorem 6.1, the operation runs in $\Theta(N^2)$ time and is parallelisable.

The kernel implementation has the following beneficial properties.

Single primitive for all domains. The same kernel operation handles M/M/1 queueing, Drude conductivity, Chapman–Enskog viscosity, etc.

Domain-specific dispatch is replaced by matrix construction; the actual coefficient calculation is uniform.

Composability. By Theorem 7.2, multi-system aggregation is closed-form: the kernel can recursively invoke the primitive on sub-systems and combine results via (3).

Determinism and reproducibility. The primitive is deterministic (Corollary 6.2), so its output is reproducible across runs and across implementations modulo floating-point representation.

Validation by specialisation. Each closed-form specialisation provides a regression test: invoking the primitive on inputs matching a classical-domain configuration must return the classical-domain coefficient. Eight such regression tests (one per row of Table 1) cover the principal classical domains.

12 Limitations and Open Questions

12.1 Bounded class count

Axiom 1 requires $N < \infty$. Systems with unboundedly many distinguishable states (e.g. infinite-dimensional Hilbert spaces, continuous spectra) are outside the present scope. Extending the framework to the continuum limit via integral generalisations of (2) is a natural next step, though convergence of the integrals requires additional assumptions on τ and g .

12.2 Determination of $\mathcal{N}$

The normalisation constant $\mathcal{N}$ is part of the system specification (Definition 3.1). In some domains it is naturally 1; in others (e.g. Drude theory) it absorbs physical constants. A general procedure for determining $\mathcal{N}$ from first principles of the underlying physical or computational substrate is application-specific; we do not pursue it here.

12.3 Time-varying τ and g

The framework treats τ and g as stationary matrices. Systems with time-varying coupling (e.g. non-stationary queues, time-dependent fields) require an extension in which (2) becomes an instantaneous statement and the throughput becomes a time average. The extension is straightforward

in principle but raises questions of convergence and ergodicity that we leave open.

12.4 Symmetry-breaking systems

Definition 3.3 adopts symmetric g as a gauge convention. Systems with intrinsically asymmetric coupling (e.g. those with broken time-reversal symmetry) require an extension allowing $g^{(ij)} \neq g^{(ji)}$. The closed form generalises immediately, but the gauge-uniqueness argument of Theorem 4.1 requires modification.

13 Conclusion

We have established a closed-form decomposition of transport coefficients on bounded categorical state spaces and proposed it as a single runtime primitive. The decomposition asserts that for any system satisfying the three axioms of bounded processing capacity, decision distinguishability, and finite decision lag, the transport coefficient takes the closed form $X = \mathcal{N}^{-1} \sum_{i,j} \tau^{(ij)} g^{(ij)}$. The framework rests on four principal results: the Universal Decomposition Theorem (Theorem 4.1), the Specialisation Theorem (Theorem 5.1), the Runtime Decidability Theorem (Theorem 6.1), and the Compositionality Theorem (Theorem 7.2).

The closed form subsumes eight classical transport coefficients as algebraic specialisations: the M/M/1 mean wait, the Jackson-network throughput, the cascade pipeline delay, the Drude conductivity, the Chapman–Enskog gas viscosity, the Einstein diffusion coefficient, the Lorenz–Lorentz refractive index, and the Mason–McDaniel ion mobility. Each is recovered as the value of the closed form under a specific algebraic restriction of the coupling matrix.

The runtime primitive evaluates the closed form in $\Theta(N^2)$ deterministic time, is parallelisable to $\Theta(N^2/p)$ on p processors, and composes multiplicatively under multi-system aggregation. Two structural phenomena—cache extinction (Proposition 8.1) and the lag lower bound (Theorem 9.1)—are exposed by the framework but invisible in the domain-specific treatments: extinction explains the exactness of caching-induced speedups, and the lag bound establishes a hardware-frequency-limited floor on every transport coefficient.

The framework is foundational. It does not commit to a particular physical theory, queueing

discipline, or kinetic model. What it provides is a uniform computational substrate over which transport coefficients of any domain satisfying the three axioms can be expressed, evaluated, composed, and verified. The eight classical specialisations are points in a larger algebraic space; the closed form is the structure of that space.

References

- Neil W. Ashcroft and N. David Mermin. *Solid State Physics*. Saunders College Publishing, 1976.
- Forest Baskett, K. Mani Chandy, Richard R. Muntz, and Fernando G. Palacios. Open, closed, and mixed networks of queues with different classes of customers. *Journal of the ACM*, 22(2):248–260, 1975.
- Sydney Chapman and T. G. Cowling. *The Mathematical Theory of Non-Uniform Gases*. Cambridge University Press, 3rd edition, 1970.
- Paul Drude. Zur elektronentheorie der metalle. *Annalen der Physik*, 306(3):566–613, 1900.
- Albert Einstein. Über die von der molekularkinetischen theorie der wärme geforderte bewegung von in ruhenden flüssigkeiten suspendierten teilchen. *Annalen der Physik*, 322(8):549–560, 1905.
- William J. Gordon and Gordon F. Newell. Closed queueing systems with exponential servers. *Operations Research*, 15(2):254–265, 1967.
- Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002.
- James R. Jackson. Networks of waiting lines. *Operations Research*, 5(4):518–521, 1957.
- Leonard Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley, 1975.
- John D. C. Little. A proof for the queuing formula: $l = \lambda w$. *Operations Research*, 9(3):383–387, 1961.
- Hendrik A. Lorentz. Über die beziehung zwischen der fortpflanzungsgeschwindigkeit des lichtes und der körperdichte. *Annalen der Physik*, 245(4):641–665, 1880.
- Ludvig V. Lorenz. Über die refractionsconstante. *Annalen der Physik*, 247(9):70–103, 1880.
- Earl A. Mason and Edward W. McDaniel. *Transport Properties of Ions in Gases*. Wiley, 1988.
- James Clerk Maxwell. On the dynamical theory of gases. *Philosophical Transactions of the Royal Society of London*, 157:49–88, 1867.
- Marian Smoluchowski. Zur kinetischen theorie der Brownschen Molekularbewegung und der suspensionen. *Annalen der Physik*, 326(14):756–780, 1906.
- Arnold Sommerfeld. Zur elektronentheorie der metalle auf grund der Fermischen statistik. *Zeitschrift für Physik*, 47(1-2):1–32, 1928.

Operational Bisimulation for Equation Identity: A Methodology for Cross-Framework Computational Equivalence

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIME Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

June 4, 2026

Abstract

We develop a methodology for proving that two formalisms with distinct operational semantics compute the same equation. The methodology rests on operational bisimulation: a relation between configurations of the two systems that is preserved under transitions and that respects the observable outputs. We make three contributions. (i) We give a precise definition of *equation identity*—the claim that two systems compute the same input-output relation—and establish a soundness theorem stating that bisimulation between operational semantics implies equation identity. (ii) We provide a coinductive proof principle for bisimulation that is modular: bisimulations may be exhibited as relations satisfying a one-step compatibility property, eliminating the need to enumerate runs over all inputs. (iii) We supply a decidable verification procedure based on partition refinement, with complexity $\mathcal{O}((m+n)\log n)$ in the configuration space, where m is the number of transitions and n the number of configurations. We further establish four structural results: bisimulation is the largest equivalence preserving observations (Theorem 3.4), bisimulation is preserved under sequential and parallel composition (Theorem 5.2), bisimulation supports a coinduction principle in the up-to style (Theorem 4.3), and bisimulation between deterministic systems coincides with extensional equivalence (Theorem 7.2). We illustrate the methodology with four worked examples: two operational semantics for arithmetic

expressions, two implementations of a queue, two computations of the factorial, and two finite-state automata for the same regular language. In each case, the equation identity claim is reduced to the exhibition of an explicit bisimulation, and the bisimulation is verified by a finite case analysis. The framework is purely formal: every theorem is provable from the definition of operational semantics and the standard notion of relation; no claim is made about which systems should be compared, only about how the comparison can be made rigorous when undertaken.

Keywords: operational bisimulation, equation identity, coinduction, partition refinement, labeled transition systems, cross-framework verification, compositional reasoning, operational semantics.

Contents

I Preliminaries	2
1 Introduction	2
1.1 The cross-framework problem . . .	2
1.2 Why bisimulation	3
1.3 Overview of main results	3
2 Labeled Transition Systems	3
2.1 Definition	3
2.2 Configurations as computation states	3
2.3 Observable functions	3
2.4 Trajectories and computations . . .	4
2.5 Equation identity	4

II Bisimulation	4	VI Discussion	9
3 Operational Bisimulation	4	12 Comparison with Other Equivalence Notions	9
3.1 Definition	4	12.1 Denotational equivalence	9
3.2 Soundness for equation identity . .	4	12.2 Trace equivalence	9
3.3 Bisimilarity is the largest bisimulation	5	12.3 Observational equivalence	9
		12.4 Extensional equivalence	9
III Coinduction and Compositionality	5	13 Limitations and Open Questions	9
4 Coinductive Reasoning	5	13.1 Infinite configuration spaces	9
4.1 The proof principle	5	13.2 Choice of observation function . .	9
4.2 Up-to bisimulation	5	13.3 Probabilistic and quantitative extensions	9
5 Compositionality	6	13.4 Higher-order systems	10
5.1 Sequential composition	6	14 Conclusion	10
5.2 Closure under contexts	6		
IV Decidability and Algorithms	6	Part I	
6 Decidability via Partition Refinement	6	Preliminaries	
6.1 The decision problem	6	1 Introduction	
6.2 Algorithm	6	1.1 The cross-framework problem	
7 Determinism Coincidence	7	Two distinct formalisms frequently claim to express the same underlying equation. A textbook arithmetic operator may be specified by a big-step reduction relation in one treatment and a small-step relation in another; a queue may be implemented as a linked list in one library and as a paired deque in another; an algorithm may be presented in recursive form in one paper and in iterative form in another. In each case the claim is that the two presentations are interchangeable: any input that one accepts produces the same output as it would on the other.	
7.1 Deterministic systems	7	How is such a claim to be verified? The most direct approach—computing both outputs on every input and checking equality—is rarely feasible because the input domain is typically infinite. A second approach—producing a single mathematical proof of input-output equivalence by induction over the input structure—is feasible in principle but suffers from being tied to a particular structural recursion: a proof that works for two arithmetic semantics does not, without adaptation, transfer to a proof that two queues are equivalent.	
V Worked Examples	8	This paper proposes a third approach, drawn from the operational-semantics tradition of pro-	
8 Two Operational Semantics for Arithmetic	8		
8.1 Big-step semantics	8		
8.2 Small-step semantics	8		
8.3 Bisimulation	8		
9 Two Implementations of a Queue	8		
9.1 Linked-list queue	8		
9.2 Paired-stack queue	8		
9.3 Bisimulation	8		
10 Two Computations of the Factorial	8		
10.1 Recursive factorial	8		
10.2 Iterative factorial	8		
10.3 Bisimulation	8		
11 Two Finite-State Automata for the Same Language	9		

cess calculi (Milner, 1980; Park, 1981; Milner, 1989). The approach is to define a relation between the configurations (intermediate states) of the two systems, exhibit that the relation is preserved under transitions, and check that related configurations produce the same observable outputs at termination. The relation is called an *operational bisimulation*, and its existence is sufficient to establish that the two systems express the same equation.

1.2 Why bisimulation

The bisimulation methodology has the following structural advantages over the direct-equivalence and induction-on-inputs approaches.

Coinductive reasoning. A bisimulation is a relation; once the relation is exhibited and shown to be closed under transitions, equivalence on all inputs follows automatically. There is no need to reason inductively over input structure.

Modularity. Bisimulations between sub-components compose into bisimulations between composites (Theorem 5.2). A proof of equivalence for an arithmetic expression evaluator can be built from proofs for its constituent operations.

Decidability for finite systems. When the configuration spaces are finite, bisimilarity is decidable in time $\mathcal{O}((m+n)\log n)$ via the partition-refinement algorithm of Paige and Tarjan (Paige and Tarjan, 1987).

Domain-agnosticism. The methodology applies uniformly to any two systems whose operational semantics are expressed as labeled transition systems. The semantics may model arithmetic, data structures, concurrent processes, or other categories of computation; the bisimulation methodology does not require domain-specific reasoning.

1.3 Overview of main results

We establish the following:

1. **Theorem 3.3** (Soundness): if there exists a bisimulation between the initial configurations of two systems, then the two systems compute the same equation.
2. **Theorem 3.4** (Largest Bisimulation): the union of all bisimulations is itself a bisimulation; bisimilarity is therefore the largest equivalence preserving observations.
3. **Theorem 5.2** (Compositionality): bisimulation is preserved under sequential composition,

parallel composition, and contextual placement.

4. **Theorem 4.3** (Up-To Coinduction): bisimulation admits a modular proof principle in which the relation may be closed up to bisimulation itself.
5. **Theorem 7.2** (Determinism Coincidence): for deterministic systems, bisimulation coincides with extensional equivalence on inputs.
6. **Theorem 6.2** (Decidability): bisimilarity between finite labeled transition systems is decidable in time $\mathcal{O}((m+n)\log n)$.

2 Labeled Transition Systems

2.1 Definition

Definition 2.1 (Labeled Transition System). A labeled transition system (*LTS*) is a quadruple

$$\mathcal{S} = \langle \mathcal{C}, \mathcal{L}, \rightarrow, \mathsf{l} \rangle$$

where $\mathcal{C}$ is a set of configurations, $\mathcal{L}$ is a set of labels, $\rightarrow \subseteq \mathcal{C} \times \mathcal{L} \times \mathcal{C}$ is a transition relation, and $\mathsf{l} \subseteq \mathcal{C}$ is a designated set of initial configurations. We write $c \xrightarrow{a} c'$ for $(c, a, c') \in \rightarrow$, and we say configuration c steps to c' with label a .

Remark 2.2. The general formulation of *LTSes* is due to Plotkin (1981) in the context of structural operational semantics, and is used as the foundational vocabulary of process algebra in Milner (1989); Hoare (1985); Bergstra and Klop (1984). We adopt the formulation here in its standard form.

2.2 Configurations as computation states

For our purposes, configurations represent intermediate computational states. The initial configurations represent inputs; the transitions represent computation steps; terminal configurations (those with no outgoing transitions) represent results.

Definition 2.3 (Terminal Configuration). A configuration $c \in \mathcal{C}$ is terminal if there is no $a \in \mathcal{L}$ and $c' \in \mathcal{C}$ with $c \xrightarrow{a} c'$. The set of terminal configurations is denoted $\mathsf{T} \subseteq \mathcal{C}$.

2.3 Observable functions

To talk about “equation identity” we must specify what we observe about a configuration. We attach to each LTS an observation function.

Definition 2.4 (Observation Function). An observation function for an LTS $\mathcal{S}$ is a partial function $\text{obs} : \mathcal{C} \rightarrow \mathcal{O}$, where $\mathcal{O}$ is a designated set of outputs. The function is defined exactly on the terminal configurations: $\text{dom}(\text{obs}) = \mathsf{T}$.

Remark 2.5. Restricting obs to terminal configurations is a deliberate methodological choice. It encodes that the equation we wish to verify is an input-output relation, not a behaviour-along-the-way relation. Two systems that produce the same final output via different intermediate steps are equation-equivalent under this notion.

2.4 Trajectories and computations

Definition 2.6 (Trajectory). A trajectory of $\mathcal{S}$ from configuration c_0 is a (finite or infinite) sequence $c_0 \xrightarrow{a_1} c_1 \xrightarrow{a_2} c_2 \xrightarrow{a_3} \dots$. A trajectory terminates if it is finite and ends in a configuration $c_n \in \mathsf{T}$.

Definition 2.7 (Computation). A computation of $\mathcal{S}$ from c_0 is a terminating trajectory. The result of a computation is $\text{obs}(c_n)$, where c_n is the terminal configuration of the computation.

2.5 Equation identity

Definition 2.8 (Equation Identity). Two LTSes $\mathcal{S}_A$ and $\mathcal{S}_B$ with shared label alphabet, output set $\mathcal{O}$, observation functions $\text{obs}_A, \text{obs}_B$, and bijection $\phi : \mathsf{I}_A \rightarrow \mathsf{I}_B$ between their initial configurations are said to compute the same equation if for every $c_0 \in \mathsf{I}_A$:

1. every computation of $\mathcal{S}_A$ from c_0 has a corresponding computation of $\mathcal{S}_B$ from $\phi(c_0)$ with the same result;
2. every computation of $\mathcal{S}_B$ from $\phi(c_0)$ has a corresponding computation of $\mathcal{S}_A$ from c_0 with the same result.

Remark 2.9. Definition 2.8 is symmetric: the two systems are required to produce the same outputs in both directions. When both systems are deterministic, the definition simplifies to “for every initial c_0 , the unique computation of $\mathcal{S}_A$ produces the same result as the unique computation of $\mathcal{S}_B$ from $\phi(c_0)$ ”. The general formulation accommodates non-deterministic systems by requiring matching computations to exist in both directions.

Part II

Bisimulation

3 Operational Bisimulation

3.1 Definition

Definition 3.1 (Bisimulation). Let $\mathcal{S}_A = \langle \mathcal{C}_A, \mathcal{L}, \rightarrow_A, \mathsf{I}_A \rangle$ and $\mathcal{S}_B = \langle \mathcal{C}_B, \mathcal{L}, \rightarrow_B, \mathsf{I}_B \rangle$ be LTSes with observation functions $\text{obs}_A, \text{obs}_B$ into a common output set $\mathcal{O}$. A relation $R \subseteq \mathcal{C}_A \times \mathcal{C}_B$ is a bisimulation between $\mathcal{S}_A$ and $\mathcal{S}_B$ if for every $(c, d) \in R$:

- (B1) if $c \xrightarrow{a}_A c'$, then there exists $d' \in \mathcal{C}_B$ with $d \xrightarrow{a}_B d'$ and $(c', d') \in R$;
- (B2) if $d \xrightarrow{a}_B d'$, then there exists $c' \in \mathcal{C}_A$ with $c \xrightarrow{a}_A c'$ and $(c', d') \in R$;
- (B3) if $c \in \mathsf{T}_A$, then $d \in \mathsf{T}_B$ and $\text{obs}_A(c) = \text{obs}_B(d)$;
- (B4) if $d \in \mathsf{T}_B$, then $c \in \mathsf{T}_A$ and $\text{obs}_A(c) = \text{obs}_B(d)$.

Two configurations c, d are bisimilar (written $c \sim d$) if there exists a bisimulation containing (c, d) .

Remark 3.2. The first two clauses are the standard transition-matching conditions of [Park \(1981\)](#); [Milner \(1989\)](#). The third and fourth clauses are the termination conditions that connect bisimulation to equation identity: bisimilar terminal configurations must produce the same output, and bisimilar transitions must lead to bisimilar terminations.

3.2 Soundness for equation identity

Theorem 3.3 (Soundness). Let $\phi : \mathsf{I}_A \rightarrow \mathsf{I}_B$ be a bijection between initial configurations. If for every $c_0 \in \mathsf{I}_A$ we have $c_0 \sim \phi(c_0)$, then $\mathcal{S}_A$ and $\mathcal{S}_B$ compute the same equation.

Proof. Fix $c_0 \in \mathsf{I}_A$. We verify both directions of Definition 2.8.

Direction 1. Let $c_0 \xrightarrow{a_1}_A c_1 \xrightarrow{a_2}_A c_2 \xrightarrow{a_3}_A \dots \xrightarrow{a_n}_A c_n$ be a computation of $\mathcal{S}_A$ from c_0 . We construct, by induction on i , a sequence $d_0 = \phi(c_0), d_1, d_2, \dots, d_n$ with $c_i \sim d_i$ and $d_{i-1} \xrightarrow{a_i}_B d_i$ for each i .

Base. $c_0 \sim d_0$ by hypothesis.

Step. Given $c_{i-1} \sim d_{i-1}$ and $c_{i-1} \xrightarrow{a_i}_A c_i$, apply clause (B1) of Definition 3.1 to obtain d_i with $d_{i-1} \xrightarrow{a_i}_B d_i$ and $c_i \sim d_i$.

At $i = n$, the configuration c_n is terminal. By clause (B3), d_n is terminal and $\text{obs}_A(c_n) =$

$\text{obs}_B(d_n)$. The constructed sequence $d_0 \xrightarrow{a_1}_B \dots \xrightarrow{a_n}_B d_n$ is therefore a computation of $\mathcal{S}_B$ from $\phi(c_0)$ with the same result.

Direction 2. Symmetric, using clauses (B2) and (B4).

Both directions hold for every $c_0 \in \mathbf{l}_A$, so the systems compute the same equation. $\square$ $\square$

3.3 Bisimilarity is the largest bisimulation

Theorem 3.4 (Largest Bisimulation). *The bisimilarity relation $\sim$ is itself a bisimulation. Moreover, every bisimulation R satisfies $R \subseteq \sim$.*

Proof. $\sim$ is a bisimulation. Let $(c, d) \in \sim$. By Definition 3.1, there exists some bisimulation R with $(c, d) \in R$. If $c \xrightarrow{a}_A c'$, then R provides d' with $d \xrightarrow{a}_B d'$ and $(c', d') \in R \subseteq \sim$. The other clauses follow analogously. Hence $\sim$ satisfies all four clauses of Definition 3.1.

Maximality. If R is a bisimulation with $(c, d) \in R$, then $c \sim d$ by definition, so $R \subseteq \sim$. $\square$ $\square$

Corollary 3.5 (Equivalence Relation). *The bisimilarity relation $\sim$ is reflexive, symmetric, and transitive on a single LTS (identifying $\mathcal{S}_A = \mathcal{S}_B$).*

Proof. *Reflexivity.* The identity relation $\{(c, c) : c \in \mathcal{C}\}$ is a bisimulation: $c \xrightarrow{a}_A c'$ matches $c \xrightarrow{a}_A c'$ with $(c', c') \in \text{Id}$, and $\text{obs}(c) = \text{obs}(c)$ trivially.

Symmetry. If R is a bisimulation, so is $R^{-1} = \{(d, c) : (c, d) \in R\}$, by exchanging clauses (B1)/(B2) and (B3)/(B4).

Transitivity. If R_1, R_2 are bisimulations, so is their relational composition $R_1 \circ R_2 = \{(c, e) : \exists d. (c, d) \in R_1 \wedge (d, e) \in R_2\}$. The matching properties compose transitively. $\square$ $\square$

Part III

Coinduction and Compositionality

4 Coinductive Reasoning

4.1 The proof principle

The largest-bisimulation theorem yields a powerful proof principle. To establish $c \sim d$, it suffices

to exhibit a single relation R with $(c, d) \in R$ that satisfies the bisimulation clauses. The relation need not be the largest; any bisimulation suffices, and any bisimulation is contained in $\sim$.

Proposition 4.1 (Coinductive Proof Principle). *To prove $c \sim d$, it suffices to construct any relation R with $(c, d) \in R$ and verify that R satisfies the four bisimulation clauses (B1)–(B4) for every pair in R .*

Proof. By Definition 3.1, R being a bisimulation containing (c, d) is the definition of $c \sim d$. $\square$ $\square$

4.2 Up-to bisimulation

A frequent enhancement of the proof principle allows the proof relation to be closed up to bisimulation itself, eliminating extraneous side conditions.

Definition 4.2 (Bisimulation Up To). *A relation $R \subseteq \mathcal{C}_A \times \mathcal{C}_B$ is a bisimulation up to $\sim$ if for every $(c, d) \in R$:*

- if $c \xrightarrow{a}_A c'$, there exist c'', d', d'' with $c' \sim c''$, $d \xrightarrow{a}_B d'$, $d' \sim d''$, and $(c'', d'') \in R$;
- the symmetric clause for $d \xrightarrow{a}_B d'$;
- the termination clauses (B3) and (B4) of Definition 3.1.

Theorem 4.3 (Up-To Coinduction). *If R is a bisimulation up to $\sim$ and $(c, d) \in R$, then $c \sim d$.*

Proof. Define $R' = \sim \circ R \circ \sim$, the composition $\sim$ -then- R -then- $\sim$. We show R' is a (standard) bisimulation containing (c, d) .

Membership. By reflexivity (Corollary 3.5), $(c, c) \in \sim$ and $(d, d) \in \sim$, so $(c, d) \in \sim \circ R \circ \sim = R'$ when $(c, d) \in R$.

Bisimulation conditions. Let $(c_1, d_1) \in R'$. By construction, there exist c_2, d_2 with $c_1 \sim c_2$, $(c_2, d_2) \in R$, and $d_2 \sim d_1$.

Suppose $c_1 \xrightarrow{a}_A c'_1$. By the bisimulation property of $\sim$, there exists c'_2 with $c_2 \xrightarrow{a}_A c'_2$ and $c'_1 \sim c'_2$. By the up-to bisimulation property of R , there exist c_3, d_3, d'_2 with $c'_2 \sim c_3$, $d_2 \xrightarrow{a}_B d'_2$, $d'_2 \sim d_3$, and $(c_3, d_3) \in R$. By the bisimulation property of $\sim$ applied to $d_2 \sim d_1$ and $d_2 \xrightarrow{a}_B d'_2$, there exists d'_1 with $d_1 \xrightarrow{a}_B d'_1$ and $d'_2 \sim d'_1$.

Putting these together: $c'_1 \sim c'_2 \sim c_3$, $(c_3, d_3) \in R$, and $d_3 \sim d'_2 \sim d'_1$. By transitivity of $\sim$, $c'_1 \sim c_3$ and $d_3 \sim d'_1$, so $(c'_1, d'_1) \in \sim \circ R \circ \sim = R'$.

The symmetric clause and the termination clauses follow similarly. Hence R' is a bisimulation containing (c, d) , so $c \sim d$. $\square$ $\square$

Remark 4.4. *Up-to bisimulation is due to Milner and Sangiorgi (1992) in the context of π -calculus, and its theory is systematically developed in Sangiorgi and Rutten (2011). The principle is widely used because it allows proofs to abstract away from configuration structure that is bisimilar but not literally equal.*

5 Compositionality

5.1 Sequential composition

Definition 5.1 (Sequential Composition of LTSes). *Let $\mathcal{S}_1$ and $\mathcal{S}_2$ be LTSes such that the output set $\mathcal{O}_1$ of $\mathcal{S}_1$ coincides with the initial-configuration domain of $\mathcal{S}_2$. The sequential composition $\mathcal{S}_1; \mathcal{S}_2$ is the LTS whose configurations are pairs $(c_1, c_2) \in \mathcal{C}_1 \uplus \mathcal{C}_2$, whose transitions are those of $\mathcal{S}_1$ until c_1 becomes terminal at which point the result $\text{obs}_1(c_1)$ becomes the initial configuration of $\mathcal{S}_2$, and whose observation function is obs_2 .*

Theorem 5.2 (Compositionality). *If $\mathcal{S}_A \sim \mathcal{S}_B$ (componentwise) and $\mathcal{S}'_A \sim \mathcal{S}'_B$, then:*

1. $\mathcal{S}_A; \mathcal{S}'_A \sim \mathcal{S}_B; \mathcal{S}'_B$ under sequential composition.
2. $\mathcal{S}_A \parallel \mathcal{S}'_A \sim \mathcal{S}_B \parallel \mathcal{S}'_B$ under parallel composition, where $\parallel$ denotes any parallel-composition operator whose semantics respects bisimulation of components.

Proof. We give the sequential case in detail; the parallel case follows analogously.

Let R_1 be a bisimulation witnessing $\mathcal{S}_A \sim \mathcal{S}_B$, and let R_2 witness $\mathcal{S}'_A \sim \mathcal{S}'_B$. Define

$$R = \{((c_A, c'_A), (c_B, c'_B)) : (c_A, c_B) \in R_1 \text{ and } (c'_A, c'_B) \in R_2\}.$$

We show R is a bisimulation for $\mathcal{S}_A; \mathcal{S}'_A$ and $\mathcal{S}_B; \mathcal{S}'_B$.

A transition in $\mathcal{S}_A; \mathcal{S}'_A$ from (c_A, c'_A) comes either from $\mathcal{S}_A$ (while c_A is non-terminal) or from $\mathcal{S}'_A$ (after c_A becomes terminal and its output initialises c'_A). In the first case, clause (B1) of R_1 matches the transition in $\mathcal{S}_B$, preserving membership in R . In the second case, clause (B3) of R_1 ensures that c_B becomes terminal at the same step with $\text{obs}_A(c_A) = \text{obs}_B(c_B)$, so both compositions initialise their continuations to the same value; clause (B1) of R_2 then matches subsequent transitions.

The termination clauses follow similarly. Hence R is a bisimulation, and the sequential composition preserves bisimilarity. $\square$ $\square$

5.2 Closure under contexts

Corollary 5.3 (Contextual Closure). *If $\mathcal{S}_A \sim \mathcal{S}_B$ and $C[\cdot]$ is any context built from sequential and parallel composition (and other bisimulation-respecting operators), then $C[\mathcal{S}_A] \sim C[\mathcal{S}_B]$.*

Proof. By induction on the structure of $C[\cdot]$, applying Theorem 5.2 at each step. $\square$ $\square$

Remark 5.4. *Corollary 5.3 is the property called congruence in process algebra (Milner, 1989). It is what makes bisimulation a usable equivalence: equivalent components remain equivalent when plugged into any context.*

Part IV

Decidability and Algorithms

6 Decidability via Partition Refinement

6.1 The decision problem

Definition 6.1 (Bisimilarity Decision Problem). *Given two finite LTSes $\mathcal{S}_A, \mathcal{S}_B$ and a pair $(c_0, d_0) \in \mathcal{I}_A \times \mathcal{I}_B$, the bisimilarity decision problem is to decide whether $c_0 \sim d_0$.*

6.2 Algorithm

The standard procedure for deciding bisimilarity is partition refinement, originally due to Kanelakis and Smolka (1990) and improved by Paige and Tarjan (1987) to near-linear complexity.

Algorithm 1 Partition-Refinement Bisimilarity

```

1: function ISBISIM( $\mathcal{S}_A, \mathcal{S}_B, c_0, d_0$ )
2:   Let  $\mathcal{C} = \mathcal{C}_A \uplus \mathcal{C}_B$ , the disjoint union
3:   Let  $\Pi$  be the partition of  $\mathcal{C}$  where two
   configurations  $c, c'$ 
4:     share a block iff  $\text{obs}(c) = \text{obs}(c')$  (ter-
   minial cases) or both
5:     are non-terminal
6:   repeat
7:      $\Pi' \leftarrow \Pi$ 
8:     for each block  $B \in \Pi$  and each label
    $a \in \mathcal{L}$  do
9:       Refine  $B$  by post-image: two con-
   figurations  $c, c'$  remain
10:      in the same sub-block iff their
    $a$ -successors fall
11:      in the same blocks of  $\Pi$ 
12:     end for
13:   until  $\Pi$  is stable:  $\Pi = \Pi'$ 
14:   return whether  $c_0$  and  $d_0$  are in the same
   block of  $\Pi$ 
15: end function

```

Theorem 6.2 (Decidability). *Algorithm 1 decides bisimilarity between two finite LTSes in time $\mathcal{O}((m+n)\log n)$, where $n = |\mathcal{C}|$ and $m = |\rightarrow|$.*

Proof. Correctness. The initial partition identifies configurations with the same observable behaviour at termination (the observation function) and treats all non-terminal configurations as initially indistinguishable. Each refinement step splits any block whose configurations have transitions to different blocks. Upon stabilisation, two configurations are in the same block iff they have indistinguishable behaviour under all transitions and observations, which is precisely bisimilarity.

Complexity. The Paige–Tarjan algorithm (Paige and Tarjan, 1987) performs $\mathcal{O}((m+n)\log n)$ work by maintaining auxiliary data structures that bound the per-block refinement cost. Each block of size k contributes $\mathcal{O}(k \log k)$ work over the algorithm’s run, and the sum of $k \log k$ over disjoint blocks is bounded by $n \log n$. The transitions add a contribution proportional to m .

Termination. The partition refines monotonically (blocks only split, never merge), and there are at most n blocks. Hence the loop terminates after at most n iterations. $\square$ $\square$

Remark 6.3. *The Paige–Tarjan bound is essentially optimal for the bisimilarity problem on*

labeled transition systems (Paige and Tarjan, 1987). For practical implementations, the $\log n$ factor can be reduced for restricted LTS classes; see Kanellakis and Smolka (1990) for a comprehensive treatment.

7 Determinism Coincidence

7.1 Deterministic systems

Definition 7.1 (Determinism). *An LTS $\mathcal{S}$ is deterministic if for every configuration c and label a , there is at most one c' with $c \xrightarrow{a} c'$. Equivalently, the transition relation is a partial function $\mathcal{C} \times \mathcal{L} \rightarrow \mathcal{C}$.*

Theorem 7.2 (Determinism Coincidence). *Let $\mathcal{S}_A, \mathcal{S}_B$ be deterministic LTSes with a label-preserving bijection between their transition structures. Then $c_0 \sim d_0$ if and only if the unique computation of $\mathcal{S}_A$ from c_0 produces the same output as the unique computation of $\mathcal{S}_B$ from d_0 .*

Proof. ($\Rightarrow$) If $c_0 \sim d_0$, by Theorem 3.3 the two systems compute the same equation, so in particular they produce the same output.

($\Leftarrow$) Suppose the unique computations from c_0 and d_0 have the same result. Define R as the set of pairs (c, d) such that c is reached at step i of $\mathcal{S}_A$ ’s unique computation from c_0 and d is reached at step i of $\mathcal{S}_B$ ’s unique computation from d_0 . By determinism, the transitions are pairwise matched (each step has a unique successor). At termination, the outputs agree by hypothesis. Hence R is a bisimulation containing (c_0, d_0) . $\square$ $\square$

Corollary 7.3 (Equation Identity for Deterministic Systems). *For deterministic systems, bisimulation methodology and direct input-output comparison are interchangeable: the same equation-identity claims are provable by either approach.*

Proof. Direct from Theorem 7.2. $\square$ $\square$

Remark 7.4. *Corollary 7.3 is not trivial: it says that in the deterministic case, the additional power of bisimulation (the ability to reason about behaviour along the way) reduces to the simpler extensional comparison. The advantage of the bisimulation framework in this case is not increased power but increased modularity: a bisimulation is a single relation, while extensional comparison requires reasoning about each input separately.*

Part V

Worked Examples

8 Two Operational Semantics for Arithmetic

8.1 Big-step semantics

Consider arithmetic expressions over integers with addition and multiplication:

$$e ::= n \mid e_1 + e_2 \mid e_1 \times e_2.$$

The big-step semantics gives a relation $e \Downarrow v$ assigning each expression to its value:

$$\frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 + e_2 \Downarrow v_1 + v_2}, \quad \frac{e_1 \Downarrow v_1 \quad e_2 \Downarrow v_2}{e_1 \times e_2 \Downarrow v_1 \times v_2}.$$

The configurations are pairs $(e, \star)$ where $\star$ denotes “unfinished”; the transition is $(e, \star) \xrightarrow{e \Downarrow v} (v, \text{done})$. The observation function returns v on terminal configurations.

8.2 Small-step semantics

The small-step semantics reduces expressions step-wise via congruence rules:

$$\begin{aligned} e_1 + e_2 &\rightarrow e'_1 + e_2 \text{ if } e_1 \rightarrow e'_1, \\ e_1 + e_2 &\rightarrow e_1 + e'_2 \text{ if } e_1 \text{ is a value and } e_2 \rightarrow e'_2, \\ n_1 + n_2 &\rightarrow n_1 + n_2 \text{ as a primitive numeric operation,} \end{aligned}$$

and similarly for multiplication. Configurations are expressions; transitions reduce; terminal configurations are integer literals.

8.3 Bisimulation

Define $R = \{((e, \star), e) : e \text{ is a closed expression}\} \cup \{((n, \text{done}), n) : n \in \mathbb{Z}\}$. We verify R is a bisimulation.

For $((e, \star), e) \in R$ with e non-terminal in both systems, the big-step transition derives $e \Downarrow v$ and reaches $((v, \text{done}), v)$. The small-step semantics reaches v via a sequence of intermediate reductions; the final state is v . Up to bisimulation (Theorem 4.3), the intermediate states are matched. The termination outputs agree: both return v .

The bisimulation establishes that the two arithmetic semantics compute the same equation.

9 Two Implementations of a Queue

9.1 Linked-list queue

A queue implemented as a singly linked list: enqueue appends to the tail in $\mathcal{O}(1)$, dequeue removes from the head in $\mathcal{O}(1)$, both with tail-pointer maintenance.

9.2 Paired-stack queue

A queue implemented as two stacks $(S_{\text{front}}, S_{\text{back}})$ (Hood and Melville, 1981). Enqueue pushes to S_{back} . Dequeue pops from S_{front} ; if S_{front} is empty, the contents of S_{back} are reversed onto S_{front} first.

9.3 Bisimulation

Two queue states are related when their visible FIFO order is the same. Formally:

$$R = \{(L, (S_f, S_b)) : L = S_f \frown \text{rev}(S_b)\},$$

where $\frown$ is concatenation. Enqueue in the linked list appends x to L ; enqueue in the paired stack pushes x to S_b , yielding $S_f \frown \text{rev}(x \cdot S_b) = S_f \frown \text{rev}(S_b) \frown [x] = L \frown [x]$. Dequeue similarly preserves the relation, including the rebalancing case when S_f becomes empty (the reversal makes the new S_f equal $\text{rev}(S_b)$, and the next dequeue removes the head). Terminal configurations (queue is empty) coincide. The bisimulation establishes equation identity.

10 Two Computations of the Factorial

10.1 Recursive factorial

The recursive definition: $\text{fact}(0) = 1$, $\text{fact}(n) = n \cdot \text{fact}(n - 1)$ for $n > 0$. Implemented as a stack of recursive calls.

10.2 Iterative factorial

The iterative definition: maintain (i, acc) starting at $(n, 1)$; on each step, $\text{acc} \leftarrow i \cdot \text{acc}$ and $i \leftarrow i - 1$; terminate when $i = 0$.

10.3 Bisimulation

Relate the recursive call stack of depth k from initial argument n to the iterative state $(n - k, \prod_{j=n-k+1}^n j)$. The transitions match: a recursive return multiplies the running product by the popped value and decreases the stack depth; the iterative step does the equivalent multiplication and decrement. Both terminate at $fact(n)$. The bisimulation establishes equation identity.

11 Two Finite-State Automata for the Same Language

Two deterministic finite automata A_1, A_2 accept the same regular language iff there exists a bisimulation between them (Hopcroft and Ullman, 1979). The classical minimal-DFA construction (Hopcroft, 1971) computes the bisimilarity partition and produces the minimal automaton. Algorithm 1 applied to A_1, A_2 (treating accepting states as terminal with output $\top$ and rejecting states as terminal with output $\perp$) decides language equivalence in $\mathcal{O}((m+n)\log n)$.

Part VI Discussion

12 Comparison with Other Equivalence Notions

12.1 Denotational equivalence

Denotational semantics (Scott and Strachey, 1971; Winskel, 1993) assigns each program a mathematical object (typically a function on a domain); two programs are denotationally equivalent if they receive the same denotation. Bisimulation differs by reasoning at the operational level: it asks whether the two computations match step-by-step, not whether they share a denotation. The two notions coincide for well-behaved denotational models but diverge in the presence of non-determinism or infinite computation.

12.2 Trace equivalence

Trace equivalence (Hoare, 1985) asks whether the two systems produce the same set of finite trajectories. Trace equivalence is strictly weaker

than bisimulation: bisimilar systems are trace-equivalent, but trace-equivalent systems may distinguish under nesting in contexts that observe branching structure (van Glabbeek, 2001).

12.3 Observational equivalence

The observational equivalence of Morris (1968) (and its operational refinement in Plotkin, 1977) considers two programs equivalent if no context can distinguish them by terminating behaviour. Bisimulation implies observational equivalence on systems with sufficiently rich observation functions; the converse holds for many calculi but not all.

12.4 Extensional equivalence

For deterministic systems, extensional equivalence (same outputs on same inputs) coincides with bisimulation by Theorem 7.2. For non-deterministic systems, extensional equivalence does not in general imply bisimulation: two systems may produce the same set of possible outputs by different branching structures.

13 Limitations and Open Questions

13.1 Infinite configuration spaces

Algorithm 1 requires finite LTSes. Systems with infinite configuration spaces (e.g. Turing machines, unbounded counters) require alternative decision procedures, when one exists; bisimilarity is undecidable in general for Turing-equivalent systems.

13.2 Choice of observation function

The methodology depends on a chosen observation function. Different choices yield different equivalences; what counts as “the same equation” depends on what is observable. We have left `obs` as a parameter of the framework, but in practice its choice is application-specific.

13.3 Probabilistic and quantitative extensions

Bisimulation has well-developed extensions to probabilistic systems (Larsen and Skou, 1991) and to systems with quantitative observations (timed

automata, weighted transitions). The methodology developed here generalises straightforwardly to these settings by replacing equality with the appropriate metric, but we do not pursue the generalisations here.

13.4 Higher-order systems

For higher-order languages where values themselves are computations (e.g. functional programs over λ -terms), bisimulation requires careful treatment of the function space (Abramsky, 1990; Gordon, 1995). Our development assumes first-order observations; the higher-order case is an active area of research and outside our scope.

14 Conclusion

We have developed a methodology for proving that two formalisms with distinct operational semantics compute the same equation, based on operational bisimulation between their labeled transition systems. The methodology rests on six results: the Soundness Theorem (Theorem 3.3) connecting bisimulation to equation identity; the Largest-Bisimulation Theorem (Theorem 3.4) establishing the canonical equivalence; the Compositionality Theorem (Theorem 5.2) ensuring bisimulation composes under sequential and parallel composition; the Up-To Theorem (Theorem 4.3) providing a modular proof principle; the Determinism Coincidence Theorem (Theorem 7.2) showing bisimulation reduces to extensional equivalence in the deterministic case; and the Decidability Theorem (Theorem 6.2) giving $O((m+n)\log n)$ algorithmic verification.

The methodology is purely formal. It does not commit to a particular family of systems or a particular notion of correctness; it offers, instead, a uniform procedure for moving from a claim that two systems express the same equation to a proof that they do. The four worked examples—arithmetic semantics, queue implementations, factorial computations, and finite-state automata—illustrate the procedure across distinct application domains.

The framework is foundational. It supports the analysis of any cross-framework equation-identity claim whose two formalisms admit operational semantics. The choices of observation function, label alphabet, and configuration space

are application-specific; the methodology of relating them by bisimulation is uniform.

References

- Samson Abramsky. The lazy lambda calculus. *Research Topics in Functional Programming*, pages 65–116, 1990.
- Jan A. Bergstra and Jan Willem Klop. Process algebra for synchronous communication. *Information and Control*, 60(1-3):109–137, 1984.
- Andrew D. Gordon. Bisimilarity as a theory of functional programming. *Electronic Notes in Theoretical Computer Science*, 1:232–252, 1995.
- C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall, 1985.
- Robert Hood and Robert Melville. Real-time queue operations in pure LISP. *Information Processing Letters*, 13(2):50–54, 1981.
- John E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In Zvi Kohavi and Azaria Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.
- John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 1979.
- Paris C. Kanellakis and Scott A. Smolka. CCS expressions, finite state processes, and three problems of equivalence. *Information and Computation*, 86(1):43–68, 1990.
- Kim G. Larsen and Arne Skou. Bisimulation through probabilistic testing. *Information and Computation*, 94(1):1–28, 1991.
- Robin Milner. *A Calculus of Communicating Systems*, volume 92 of *Lecture Notes in Computer Science*. Springer, 1980.
- Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- Robin Milner and Davide Sangiorgi. *Barbed Bisimulation*, volume 623 of *Lecture Notes in Computer Science*. Springer, 1992.
- James H. Morris. *Lambda-Calculus Models of Programming Languages*. PhD thesis, MIT, 1968.

Robert Paige and Robert E. Tarjan. Three partition refinement algorithms. *SIAM Journal on Computing*, 16(6):973–989, 1987.

David Park. Concurrency and automata on infinite sequences. In *Theoretical Computer Science, 5th GI-Conference*, volume 104 of *Lecture Notes in Computer Science*, pages 167–183. Springer, 1981.

Gordon D. Plotkin. LCF considered as a programming language. *Theoretical Computer Science*, 5(3):223–255, 1977.

Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, 1981.

Davide Sangiorgi and Jan Rutten. *Advanced Topics in Bisimulation and Coinduction*. Cambridge University Press, 2011.

Dana S. Scott and Christopher Strachey. Toward a mathematical semantics for computer languages. *Proceedings of the Symposium on Computers and Automata*, pages 19–46, 1971.

Rob J. van Glabbeek. The linear time – branching time spectrum I: The semantics of concrete, sequential processes. *Handbook of Process Algebra*, pages 3–99, 2001.

Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.

The Dispatch Substrate Kernel: Architecture, Composition, and Conformance of a Five-Subsystem Dispatch Loop for Categorical Computation

Kundai Farai Sachikonye
Technical University of Munich
Chair of Proteomics and Bioanalytics
AIME Registry for Artificial Intelligence
Maximus-von-Imhof-Forum 3, 85354 Freising, Germany
kundai.sachikonye@wzw.tum.de

June 4, 2026

Abstract

We specify a kernel architecture for the dispatch of categorical computation: an execution substrate that takes a symbolic fragment representing an operation and returns the value the operation produces. The substrate wraps an executor with five subsystems that derive from the canonical decomposition of any dispatch responsibility: pre-dispatch validation, post-dispatch memoization, pending-work scheduling, external-resource retrieval, and runtime invariant monitoring. We formalise each subsystem as an interface with a small, frozen surface, define the dispatch loop that composes them with the underlying executor, and prove that the composition is sound and deterministic. We further establish four structural results. (i) The *Dispatch Correctness Theorem*: with all five subsystems instantiated to identity (no-op), the kernel produces results bit-identical to the bare executor; with non-trivial subsystems, the result is preserved modulo the subsystems' specified semantics. (ii) The *Independence Theorem*: the five subsystems are pairwise functionally independent, in the sense that the dispatch result depends on each subsystem only through its specified interface and not through shared internal state. (iii) The *Stability Contract Theorem*: extending the kernel with additional subsystems (or strengthening existing ones) does not invalidate any client that depends only on the frozen interface surface. (iv) The *Conformance Decidability*

Theorem: the property-based regression suite over kernel observable events is decidable, and the kernel admits cross-architecture invariance under a fixed conformance methodology. We further give the dispatch algorithm explicitly, analyse its time complexity ($\mathcal{O}(1)$ per subsystem hook plus the executor's intrinsic cost), and characterise the parallelism opportunities admitted by the architecture. The kernel is purely a substrate: it does not commit to a particular symbolic language, a particular cache eviction policy, a particular scheduler, or a particular retrieval mechanism. What it provides is the architectural skeleton over which such choices are made, together with the theorems that ensure the choices do not invalidate the substrate's guarantees.

Keywords: dispatch substrate, kernel architecture, microkernel, validation engine, memoization, scheduling, surgical retrieval, conservation monitoring, stability contract, conformance regression.

Contents

I	Preliminaries	2
1	Introduction	2
1.1	The dispatch problem	2
1.2	The five responsibilities	3
1.3	Overview of main results	3
2	The Underlying Executor	3

II	The Five Subsystems	4	12 Comparison with Other Dispatch Substrates	9
3	Subsystem Interfaces	4	12.1 Microkernels	9
3.1	PVE: Pre-Dispatch Validation . . .	4	12.2 Monolithic operating system kernels	9
3.2	CMM: Post-Dispatch Memoization	4	12.3 Virtual machines	9
3.3	PSS: Pending-State Scheduling . .	4	12.4 Actor systems	9
3.4	DIC: External-Resource Retrieval .	4	12.5 Database query engines	9
3.5	TEM: Runtime Invariant Monitoring	4	12.6 Event-driven runtimes	10
4	Dispatch Events	5	13 Limitations and Open Questions	10
			13.1 Subsystem ordering	10
			13.2 Subsystem composition	10
			13.3 Distributed dispatch	10
			13.4 Subsystem evolution policies	10
III	The Dispatch Loop	5	14 Conclusion	10
5	The Dispatch Algorithm	5		
5.1	Composition	5		
5.2	The dispatch procedure	5		
5.3	Soundness	5		
IV	Structural Properties	6		
6	Subsystem Independence	6		
6.1	Functional independence	6		
7	Stability Contract	7		
7.1	The frozen surface	7		
7.2	Refinement of subsystems	7		
8	Compositionality	7		
V	Conformance Regression	7		
9	The Conformance Suite	7		
9.1	Property-based regression	7		
9.2	Cross-architecture invariance . . .	8		
VI	Performance and Parallelism	8		
10	Time Complexity	8		
10.1	Asynchronous monitor	8		
11	Parallelism	9		
11.1	Single-fragment dispatch	9		
11.2	Multi-fragment dispatch	9		
VII	Discussion	9		

Part I

Preliminaries

1 Introduction

1.1 The dispatch problem

A dispatch substrate is the runtime component that, given a symbolic representation of a computation, produces its value. The category of substrates is broad: an operating-system kernel dispatches a system call to its handler (Tanenbaum and Bos, 2014; Liedtke, 1995; Klein et al., 2009); a virtual machine dispatches a bytecode instruction to its interpreter (Lindholm et al., 2014); an actor system dispatches a message to a registered receiver (Hewitt et al., 1973; Agha, 1986; Armstrong, 2003); a microservice gateway dispatches an RPC to a service implementation (Birrell and Nelson, 1984); a database engine dispatches a query plan to a physical operator (Graefe, 1993). Each of these substrates handles dispatch under different physical assumptions but shares a common architectural decomposition.

We make this decomposition precise and present it as a single design: a kernel that wraps an executor with exactly five subsystems whose responsibilities partition the dispatch envelope. The five are derived from the universal dispatch responsibilities and not from any particular substrate; together they cover the architectural surface of any dispatch component, and individually they admit independent substitution and refinement.

1.2 The five responsibilities

Every dispatch substrate has the following five responsibilities, regardless of its application domain.

Pre-dispatch validation. Before evaluation begins, the substrate must determine whether the input fragment is admissible. This responsibility separates well-formed inputs from inputs that would cause the executor to misbehave. We name the subsystem implementing it the *proof-validation engine* (PVE).

Post-dispatch memoization. After evaluation completes, the substrate may cache the result against a fingerprint of the input to avoid re-evaluating the same fragment in the future. This responsibility realises the cache-hit speedup that distinguishes a memoising dispatch substrate from a stateless one. We name the subsystem implementing it the *categorical memory manager* (CMM).

Pending-work scheduling. Between pre-dispatch and execution, the substrate may have a backlog of pending fragments to evaluate; it must decide their order. This responsibility is the substrate’s scheduler. We name the subsystem implementing it the *pending-state scheduler* (PSS).

External-resource retrieval. During evaluation, the executor may require external data—bytes from a network, columns from a database, weights from a model. The substrate mediates this retrieval and may impose a retrieval policy that fetches less than the executor would naively request. We name the subsystem implementing it the *demon I/O controller* (DIC).

Runtime invariant monitoring. After dispatch completes, the substrate may publish observable events that downstream observers consume to check invariants (monotone counters, conservation laws, regression-test predicates). The monitoring is asynchronous and lossy: a slow observer does not throttle the dispatch path. We name the subsystem implementing it the *trigger event monitor* (TEM).

We claim that these five are the canonical decomposition. Any dispatch substrate either implements all five (with non-trivial choices of policy) or implements a strict subset (with trivial identity policies for the remaining responsibilities). No substrate requires a sixth responsibility that is not expressible as a composition of these five.

1.3 Overview of main results

The paper establishes the following:

1. **Theorem 5.2** (Dispatch Correctness): the dispatch loop with all subsystems instantiated to identity returns bit-identical results to the bare executor.
2. **Theorem 6.2** (Subsystem Independence): the five subsystems are pairwise functionally independent through their interfaces.
3. **Theorem 7.3** (Stability Contract): clients depending only on the frozen interface surface remain functional under any additive evolution of the kernel.
4. **Theorem ??** (Determinism): if each subsystem is a deterministic function of its inputs, so is the kernel’s dispatch output.
5. **Theorem 8.1** (Compositional Substitution): swapping a subsystem for a bisimilar one does not change the dispatch result.
6. **Theorem 9.3** (Conformance Decidability): the property-based regression suite over kernel observables is decidable in time linear in the workload size.

2 The Underlying Executor

The kernel wraps an executor. We treat the executor as a black box characterised by its interface; the kernel’s properties will be proved relative to that interface.

Definition 2.1 (Executor). *An executor is a triple $E = \langle \mathcal{F}, \mathcal{V}, \text{eval} \rangle$ where $\mathcal{F}$ is a set of fragments (symbolic representations of computations), $\mathcal{V}$ is a set of values, and $\text{eval} : \mathcal{F} \rightarrow \mathcal{V} \cup \{\perp\}$ is the evaluation function. The symbol $\perp$ denotes evaluation failure (e.g. unresolved hole, missing provider).*

Remark 2.2. *We deliberately leave the structure of $\mathcal{F}$ and $\mathcal{V}$ abstract. The kernel architecture is parametric in the executor: the same five subsystems apply to a fragment of an arithmetic-expression language, a process-calculus term, or an RPC payload. Specific fragment grammars and value spaces are application-specific and need not be fixed at the architectural level.*

Definition 2.3 (Determinism). *The executor is deterministic if eval is a function (single-valued); i.e. for every $F \in \mathcal{F}$, $\text{eval}(F)$ takes a single value (possibly $\perp$).*

Definition 2.4 (Operation Registry). *An operation registry $\mathcal{R}$ is a finite map from named operation identifiers to providers: implementations that accept named-argument records of values and return values. The registry mediates how eval resolves named-operation references within a fragment.*

We assume E has a registry $\mathcal{R}$ as an implicit parameter and that eval uses $\mathcal{R}$ to dispatch named-operation calls.

Part II

The Five Subsystems

3 Subsystem Interfaces

We now define each of the five subsystems by its interface. Each definition is parametric in the executor type and contains the minimal set of methods the dispatch loop requires.

3.1 PVE: Pre-Dispatch Validation

Definition 3.1 (Validation Subsystem). *A validation subsystem σ_P is an object exposing a method*

$$\text{valid} : \mathcal{F} \rightarrow \mathbb{B},$$

where $\text{valid}(F) = \text{true}$ indicates that the fragment is admissible for dispatch and false indicates that the dispatch path must reject the fragment without evaluation.

Definition 3.2 (Identity PVE). *The identity validation subsystem $\sigma_{P,0}$ returns true for every fragment: it imposes no validation constraint.*

The validation subsystem is the substrate’s gatekeeper. Concrete instances may implement type checking, refinement-type discipline, three-route verification, capability checks, signature validation, or any other admission criterion expressible as a Boolean fragment predicate. The interface is opaque to the specific check.

3.2 CMM: Post-Dispatch Memoization

Definition 3.3 (Memoization Subsystem). *A memoization subsystem σ_M is an object exposing two methods:*

$$\begin{aligned} \text{lookup} : \mathcal{F} &\rightarrow \mathcal{V} \cup \{\text{miss}\}, \\ \text{insert} : \mathcal{F} \times \mathcal{V} &\rightarrow \mathbf{1}. \end{aligned}$$

The lookup method returns a previously cached value for the given fragment or signals a miss; the insert method stores a fresh result. The result of insert is unit (no semantic value); the side-effect is internal.

Definition 3.4 (Identity CMM). *The identity memoization subsystem $\sigma_{M,0}$ always returns miss from lookup and is a no-op on insert.*

3.3 PSS: Pending-State Scheduling

Definition 3.5 (Scheduling Subsystem). *A scheduling subsystem σ_S is an object exposing a method*

$$\text{order} : \mathcal{F}^* \rightarrow \mathcal{F}^*,$$

where $\mathcal{F}^*$ is the set of finite sequences of fragments, and the output is a permutation of the input that specifies the order in which the pending fragments should be evaluated.

Definition 3.6 (Identity PSS). *The identity scheduling subsystem $\sigma_{S,0}$ returns the input sequence unchanged: it preserves the arrival order of pending fragments.*

3.4 DIC: External-Resource Retrieval

Definition 3.7 (Retrieval Subsystem). *A retrieval subsystem σ_R is an object exposing a method*

$$\text{policy} : Q \times S \rightarrow P,$$

where Q is the set of queries the executor may issue, S is the set of external sources, and P is the set of retrieval policies (e.g. “fetch all”, “fetch the top- k mutual-information records”). The executor consults σ_R before issuing an external request and follows the returned policy.

Definition 3.8 (Identity DIC). *The identity retrieval subsystem $\sigma_{R,0}$ always returns the “fetch all” policy: the executor retrieves whatever it would have retrieved without surgical mediation.*

3.5 TEM: Runtime Invariant Monitoring

Definition 3.9 (Monitoring Subsystem). *A monitoring subsystem σ_T is an object exposing a method*

$$\text{observe} : \mathcal{E} \rightarrow \mathbf{1},$$

where $\mathcal{E}$ is the set of dispatch events: structured records published by the dispatch loop on each

completed dispatch. The result is unit; the side-effect is the monitor's internal book-keeping (typically asynchronous, non-blocking, lossy under back-pressure).

Definition 3.10 (Identity TEM). *The identity monitoring subsystem $\sigma_{T,0}$ is a no-op on observe: it ignores every event.*

4 Dispatch Events

Definition 4.1 (Dispatch Event). *A dispatch event $e \in \mathcal{E}$ is a structured record carrying at least the following fields:*

- the top-level operation name (or $\perp$ for non-Call roots);
- the dispatch elapsed time;
- the success/failure flag;
- the cache-hit flag (whether the result was served from σ_M);
- the decision counter increment (an integer-valued contribution to a monotone counter the kernel maintains).

The event is the unit of observable kernel activity; the TEM subscribes to a broadcast channel of events and consumes them out-of-band.

Part III

The Dispatch Loop

5 The Dispatch Algorithm

5.1 Composition

Definition 5.1 (Kernel). *A kernel $\mathcal{K}$ is a tuple*

$$\mathcal{K} = \langle E, \sigma_P, \sigma_M, \sigma_S, \sigma_R, \sigma_T, \mathcal{E} \rangle$$

consisting of an executor E , the four subsystems of Definitions 3.1–3.7, the monitor σ_T , and a dispatch-event channel $\mathcal{E}$.

5.2 The dispatch procedure

Algorithm 1 Kernel Dispatch

```

1: function DISPATCH( $\mathcal{K}, F$ )
2:    $t_0 \leftarrow$  clock now
3:   if not  $\sigma_P$ .valid( $F$ ) then
4:     publish  $\mathcal{E}(\text{op} = \top(F), \text{ok} =$ 
       false, cache = false)
5:     return  $\perp$ 
6:   end if
7:   ordered  $\leftarrow \sigma_S$ .order( $\sigma_S$ -queue  $\cup \{F\}$ )
8:    $F \leftarrow$  head(ordered)
9:    $r \leftarrow \sigma_M$ .lookup( $F$ )
10:  if  $r \neq$  miss then
11:     $v \leftarrow r$ ; cache-hit  $\leftarrow$  true
12:  else
13:     $v \leftarrow E$ .eval( $F$ )  $\triangleright$  may consult  $\sigma_R$  for
      I/O
14:     $\sigma_M$ .insert( $F, v$ )
15:    cache-hit  $\leftarrow$  false
16:  end if
17:   $t_1 \leftarrow$  clock now
18:   $e \leftarrow \mathcal{E}(\text{op} = \top(F), \text{elapsed} = t_1 - t_0,$ 
19:    ok =  $v \neq \perp$ , cache =
      cache-hit)
20:  publish  $e$ 
21:   $\sigma_T$ .observe( $e$ )
22:  return  $v$ 
23: end function

```

5.3 Soundness

Theorem 5.2 (Dispatch Correctness with Identity Subsystems). *Let $\mathcal{K}_0 = \langle E, \sigma_{P,0}, \sigma_{M,0}, \sigma_{S,0}, \sigma_{R,0}, \sigma_{T,0}, \mathcal{E} \rangle$ be a kernel with all subsystems instantiated to identity. Then for every fragment F :*

$$\text{dispatch}(\mathcal{K}_0, F) = \text{eval}(F).$$

Proof. We trace Algorithm 1 with the identity subsystems substituted.

Line 3: $\sigma_{P,0}$.valid(F) = true unconditionally; the early return is not taken.

Line 7: $\sigma_{S,0}$.order returns the input unchanged; the head is F .

Line 8: $\sigma_{M,0}$.lookup(F) = miss unconditionally; the cache-hit branch is not taken.

Line 12: the executor is invoked: $v \leftarrow \text{eval}(F)$.

Line 13: $\sigma_{M,0}$.insert is a no-op.

Lines 16–19: $\sigma_{T,0}$.observe is a no-op; the event is published on the channel but has no semantic effect on the return value.

Line 20: return $v = \text{eval}(F)$.

Hence $\text{dispatch}(\mathcal{K}_0, F) = \text{eval}(F)$. $\square$ $\square$

Theorem 5.3 (Dispatch Correctness with Non-Trivial Subsystems). *Let $\mathcal{K} = \langle E, \sigma_P, \sigma_M, \sigma_S, \sigma_R, \sigma_T, \mathcal{E} \rangle$ be a kernel with arbitrary subsystems. Then for every fragment F admitted by σ_P for which σ_M contains no prior entry, $\text{dispatch}(\mathcal{K}, F) = \text{eval}(F)$. For an admitted fragment for which σ_M contains a prior entry v_0 from a previous dispatch of F , $\text{dispatch}(\mathcal{K}, F) = v_0$.*

Proof. *Cache miss:* Lines 7–12 proceed as in Theorem 5.2, yielding $v = \text{eval}(F)$.

Cache hit: Line 9 returns the cached value $r = v_0$, and line 19 returns $v = v_0$.

In both cases the rejection branch (line 3) does not fire because $\sigma_P.\text{valid}(F) = \text{true}$ by hypothesis. $\square$ $\square$

Corollary 5.4 (Determinism Preservation). *If E is deterministic and each subsystem is a deterministic function of its inputs, then $\text{dispatch}(\mathcal{K}, \cdot)$ is a deterministic function on $\mathcal{F}$.*

Proof. By Theorem 5.3, the dispatch result is determined by $(F, \sigma_P.\text{valid}(F), \sigma_M.\text{lookup}(F), \text{eval}(F))$. Each component is a deterministic function of its inputs by hypothesis. The composition is therefore deterministic. $\square$ $\square$

Part IV

Structural Properties

6 Subsystem Independence

6.1 Functional independence

Definition 6.1 (Functional Independence). *Two subsystems σ_a, σ_b are functionally independent in a kernel $\mathcal{K}$ if the dispatch output $\text{dispatch}(\mathcal{K}, F)$ depends on σ_a only through the values returned by σ_a 's interface methods, and similarly for σ_b . In particular, swapping σ_a for any σ'_a that returns identical interface outputs on the dispatched fragment does not change the dispatch result.*

Theorem 6.2 (Pairwise Independence). *The five subsystems $\sigma_P, \sigma_M, \sigma_S, \sigma_R, \sigma_T$ in Algorithm 1 are pairwise functionally independent.*

Proof. We inspect each pair.

(σ_P, σ_M) : σ_P is consulted at line 3 and its return value b alone determines whether the rejection branch fires. σ_M is consulted at line 8 only if validation passed (i.e. $b = \text{true}$). The two consultations are sequential and depend on disjoint internal states.

(σ_P, σ_S) : σ_S at line 7 reorders pending fragments; the chosen head is then validated (if it has not been) or proceeds to line 8. Whether validation passes depends on the fragment, not on the scheduling order; the two are independent in the dispatch-result sense.

(σ_P, σ_R) : σ_R is consulted by the executor at line 12; σ_P is consulted at line 3. Their roles do not intersect in the dispatch path.

(σ_P, σ_T) : σ_T consumes events at line 18; its interface is observe-only. The dispatch result is fixed by line 19 before σ_T runs.

(σ_M, σ_S) : σ_S reorders the pending queue; σ_M services per-fragment cache requests. Reordering does not change which fragments are cached.

(σ_M, σ_R) : on a cache hit, the executor is not invoked; σ_R is therefore not consulted. On a cache miss, σ_R is consulted by the executor independently of any prior σ_M state.

(σ_M, σ_T) : σ_T observes events that include the cache-hit flag, but σ_T 's observe method does not feed back into σ_M .

(σ_S, σ_R) : σ_S orders pending fragments; σ_R controls per-fragment external retrieval. The orderings and policies are on disjoint axes.

(σ_S, σ_T) : σ_T observes events emitted at the per-dispatch granularity; σ_S chooses the next fragment. The two operate at different layers.

(σ_R, σ_T) : σ_R 's policies affect what the executor retrieves; σ_T 's observe is post-dispatch and orthogonal.

In every pair, the two subsystems consult their interfaces sequentially or are causally disjoint in the dispatch path. Hence the pair is functionally independent. $\square$ $\square$

Corollary 6.3 (Substitution Theorem). *For any pair of bisimilar subsystems $\sigma_X \sim \sigma'_X$ at any of the five positions, replacing σ_X with σ'_X in $\mathcal{K}$ does not change the dispatch output:*

$$\text{dispatch}(\mathcal{K}[\sigma_X], F) = \text{dispatch}(\mathcal{K}[\sigma'_X], F) \quad \forall F \in \mathcal{F}.$$

Proof. By Definition 6.1, the dispatch output depends on each subsystem through its interface

only. Bisimilarity implies identical interface behaviour. The substitution is therefore semantics-preserving. $\square$ $\square$

7 Stability Contract

7.1 The frozen surface

Definition 7.1 (Frozen Surface). *The frozen surface of a kernel $\mathcal{K}$ is the set of method signatures of the five subsystems and the executor's eval function. The frozen surface is the contract on which downstream clients depend.*

Definition 7.2 (Evolution). *The kernel evolves additively if the frozen surface of an evolved kernel $\mathcal{K}'$ is a superset of the frozen surface of $\mathcal{K}$, with every original method having the same signature in $\mathcal{K}'$ as in $\mathcal{K}$.*

Theorem 7.3 (Stability Contract). *Let $\mathcal{K}'$ be an additive evolution of $\mathcal{K}$ as in Definition 7.2. Then every client whose behaviour depends only on the frozen surface of $\mathcal{K}$ behaves identically when wired to $\mathcal{K}'$ (modulo any newly added optional capabilities the client elects not to use).*

Proof. The client invokes only methods in the frozen surface of $\mathcal{K}$. By the additive-evolution hypothesis, every such method has the same signature in $\mathcal{K}'$. The client's invocations resolve identically; the responses, by the executor and subsystem contracts, are identical (modulo subsystem-specific semantic refinement that the client does not observe). Hence the client's observable behaviour is unchanged. $\square$ $\square$

Remark 7.4. *Theorem 7.3 is the structural foundation of the kernel's release-engineering discipline. Once a method signature has been promoted to the frozen surface, downstream clients have a contract guarantee that the kernel will not break them. The evolution policy is conservative by design: every addition is opt-in, every removal is a major-version event, and every signature change is a breaking change.*

7.2 Refinement of subsystems

Proposition 7.5 (Subsystem Refinement). *A subsystem σ may be replaced by a refined σ' that exposes the same interface and produces interface-equivalent behaviour without breaking any client of the kernel.*

Proof. By Theorem 6.2, the dispatch output depends on σ only through its interface. By Corollary 6.3, interface-bisimilar substitution preserves the dispatch output. Hence the refinement is transparent to all clients of $\mathcal{K}$. $\square$ $\square$

8 Compositionality

Theorem 8.1 (Compositional Substitution). *Let $\mathcal{K}_a, \mathcal{K}_b$ be two kernels with bisimilar subsystems at each of the five positions ($\sigma_{P,a} \sim \sigma_{P,b}$, $\sigma_{M,a} \sim \sigma_{M,b}$, $\dots$, $\sigma_{T,a} \sim \sigma_{T,b}$) and identical executors. Then for every fragment $F \in \mathcal{F}$:*

$$\text{dispatch}(\mathcal{K}_a, F) = \text{dispatch}(\mathcal{K}_b, F).$$

Proof. By Corollary 6.3 applied five times in succession (one per subsystem position), the dispatch output is unchanged at each substitution. The composite substitution from $\mathcal{K}_a$ to $\mathcal{K}_b$ is therefore output-preserving. $\square$ $\square$

Corollary 8.2 (Reference-Implementation Equivalence). *Any two implementations of the kernel that respect the five interface contracts and use the same executor produce identical outputs on every fragment.*

Proof. Direct from Theorem 8.1: interface-contract compliance is interface bisimilarity. $\square$ $\square$

Part V Conformance Regression

9 The Conformance Suite

9.1 Property-based regression

Definition 9.1 (Property-Based Test). *A property-based test over a kernel $\mathcal{K}$ is a triple $\langle W, P, \tau \rangle$ where W is a workload (a finite sequence of fragments), P is a Boolean predicate over dispatch-event streams, and τ is an acceptance threshold (real number in $[0, 1]$ indicating the fraction of W on which P must hold).*

Definition 9.2 (Conformance Run). *A conformance run of test $\langle W, P, \tau \rangle$ on $\mathcal{K}$ is the procedure that dispatches each fragment in W via $\mathcal{K}$, collects*

the emitted event stream, and reports the fraction of events for which P holds. The run passes iff this fraction is at least τ .

Theorem 9.3 (Conformance Decidability). *For a finite workload W and a decidable predicate P , the conformance run of $\langle W, P, \tau \rangle$ on $\mathcal{K}$ is decidable in time $\mathcal{O}(|W| \cdot (T_{\text{dispatch}} + T_P))$, where T_{dispatch} is the per-dispatch cost and T_P is the per-event predicate evaluation cost.*

Proof. Iterate W , dispatch each fragment, evaluate P on the emitted event, count the fraction, compare to τ . Each step has bounded cost; the total cost is the product of $|W|$ and the per-step cost. Termination is guaranteed by W being finite. $\square$ $\square$

9.2 Cross-architecture invariance

Definition 9.4 (Cross-Architecture Invariance). *A conformance test passes cross-architecture-invariantly if it passes on every supported host configuration (different CPUs, operating systems, toolchain versions).*

Proposition 9.5 (Floating-Point Bound on Cross-Architecture Discrepancy). *For a conformance test whose predicate P is continuous in numerical fields of the event, the cross-architecture discrepancy in pass-rate is bounded by ϵ_{mach} times the predicate’s Lipschitz constant times $|W|$, where ϵ_{mach} is the maximum floating-point machine epsilon across supported hosts.*

Proof. The dispatch output and event-stream fields are computed with floating-point arithmetic; their per-step error is bounded by ϵ_{mach} . The predicate’s Lipschitz continuity bounds the resulting variation in P ’s value. Summing over the workload gives the stated bound. $\square$ $\square$

Remark 9.6. *Proposition 9.5 establishes that cross-architecture invariance is achievable for floating-point-continuous predicates with a precision tolerance scaling linearly in the workload. For integer-valued or Boolean predicates, the discrepancy is zero and cross-architecture pass-rate is exact.*

Part VI

Performance and Parallelism

10 Time Complexity

Theorem 10.1 (Per-Dispatch Time Bound). *The per-dispatch cost of Algorithm 1 is*

$$T_{\text{dispatch}} = T_{\sigma_P} + T_{\sigma_S} + T_{\sigma_M}^{\text{lookup}} + T_E + T_{\sigma_M}^{\text{insert}} + T_{\sigma_T} + \mathcal{O}(1),$$

where T_X is the per-call cost of subsystem X or the executor’s intrinsic cost.

Proof. Direct accounting of the algorithm’s lines: the five subsystem hooks plus the executor invocation, plus constant-time arithmetic (clock reads, event construction, branch decisions). $\square$ $\square$

Corollary 10.2 (Identity-Subsystem Overhead). *With all subsystems instantiated to identity, the per-dispatch overhead is $\mathcal{O}(1)$ over the bare executor cost T_E .*

Proof. Each identity subsystem has $\mathcal{O}(1)$ method cost: identity Boolean return, identity sequence return, miss return, fetch-all return, no-op observe. Summing five $\mathcal{O}(1)$ contributions plus arithmetic gives the bound. $\square$ $\square$

10.1 Asynchronous monitor

The TEM subsystem is invoked synchronously in Algorithm 1 (line 18), but it is permitted to be a non-blocking event publisher. We make this precise.

Definition 10.3 (Non-Blocking TEM). *A monitoring subsystem σ_T is non-blocking if its observe method returns in bounded time independent of the rate at which events are published. Implementations include lock-free broadcast channels, ring buffers with overwrite, and bounded-buffer drop-on-overflow.*

Proposition 10.4 (Back-Pressure Isolation). *A non-blocking σ_T ensures that slow observers do not throttle the dispatch path. The per-dispatch cost of observe is independent of the observer’s processing rate.*

Proof. By Definition 10.3, observe returns in bounded time regardless of observer state. The dispatch path is therefore decoupled from observer processing. $\square$ $\square$

11 Parallelism

11.1 Single-fragment dispatch

A single-fragment dispatch is largely sequential by Algorithm 1: the five subsystem hooks must execute in their stated order. Limited parallelism is available within $E.\text{eval}$ if the executor supports it (e.g. parallel composition of sub-fragments), but the kernel’s own pipeline is sequential.

11.2 Multi-fragment dispatch

When the kernel dispatches a stream of fragments, parallelism opportunities arise:

Proposition 11.1 (Per-Fragment Parallelism). *If the fragments in a workload are pairwise independent (their executor evaluations do not share state), they may be dispatched in parallel on independent kernel instances or on a single kernel with thread-safe subsystems.*

Proof. By Theorem 6.2, the dispatch output depends only on the per-fragment subsystem interfaces. Independent fragments do not share subsystem state (provided the implementations are thread-safe). Hence parallel execution is semantics-preserving. $\square$ $\square$

Remark 11.2. *In practice, thread-safety of subsystems is a per-implementation property. The kernel architecture admits parallelism but does not enforce it; specific subsystem implementations may serialise on internal locks or be wait-free. The architecture is parametric in this choice.*

Part VII Discussion

12 Comparison with Other Dispatch Substrates

12.1 Microkernels

The microkernel design (Liedtke, 1995; Klein et al., 2009) centres on a small set of primitives—thread, IPC, address space, capability—and delegates other functionality to user-space servers. Our dispatch kernel is similar in spirit: it provides a small set of primitives (the five subsystems) and delegates application-specific behaviour to subsystem implementations. The two designs differ in

the granularity of dispatch: microkernels dispatch system calls, while our kernel dispatches symbolic fragments. The architectural decomposition is analogous.

12.2 Monolithic operating system kernels

Monolithic kernels (Tanenbaum and Bos, 2014) integrate validation, caching, scheduling, I/O, and monitoring tightly within the kernel address space. The interfaces between these subsystems are typically internal and not part of any stability contract. Our kernel exposes the five subsystems as interface-stable boundaries, making each independently replaceable. The trade-off is the per-call overhead of the interface abstraction (Corollary 10.2), which is $\mathcal{O}(1)$ but non-zero.

12.3 Virtual machines

The Java Virtual Machine (Lindholm et al., 2014) dispatches bytecode instructions through a fixed interpreter loop or JIT compiler. Our kernel is more general: it dispatches arbitrary fragments through a parametric executor. A JVM-like substrate can be built as an executor whose fragments are bytecode and whose eval runs the interpreter. The five-subsystem decomposition then applies: bytecode verification is PVE, the JIT compilation cache is CMM, thread scheduling is PSS, native I/O is DIC, and JVMTI events are TEM.

12.4 Actor systems

Actor systems (Hewitt et al., 1973; Agha, 1986; Armstrong, 2003) dispatch messages to receivers via mailboxes. The five-subsystem mapping is direct: message-format validation is PVE, mailbox memoization is CMM, mailbox scheduling is PSS, remote-actor lookup is DIC, and supervisor events are TEM. The actor abstraction differs from our kernel in its focus on concurrency and isolation; the dispatch substrate underneath is structurally similar.

12.5 Database query engines

Query engines (Graefe, 1993) dispatch query plans to physical operators. The mapping: plan validation is PVE, plan-cache or materialised-view lookup is CMM, query-plan scheduling is PSS, storage retrieval is DIC, and query-event logging

is TEM. The kernel’s parametric design subsumes query-engine architecture.

12.6 Event-driven runtimes

Event-driven runtimes (Node.js, Erlang/OTP, Tokio) dispatch events to handlers via reactor loops (Schmidt et al., 2000). The five-subsystem mapping again applies. The reactor pattern is a refinement of the dispatch substrate in which the order of fragment arrival is non-deterministic and the scheduler (PSS) takes on a central role.

13 Limitations and Open Questions

13.1 Subsystem ordering

Algorithm 1 fixes the order of subsystem hooks: PVE before PSS before CMM before E before σ_T . Other orderings are possible (e.g. CMM before PVE, allowing cached results to bypass validation). The Independence Theorem (Theorem 6.2) is sensitive to the ordering: reordering can affect correctness if subsystems share state in ways the interface conceals. A principled treatment of alternative orderings and their preservation properties is outside our scope.

13.2 Subsystem composition

The five subsystems are presented as independent slots. In practice, applications may want to compose multiple validators (e.g. type checking + signature verification + capability check) into a single PVE. The composition is supported by ad hoc means (the composed PVE returns true iff every constituent returns true) but a general theory of subsystem-internal composition is left for future work.

13.3 Distributed dispatch

Our treatment assumes a single kernel instance. Distributed dispatch—multiple kernel instances cooperating to serve a single dispatch stream—raises questions of consistency between subsystem states (especially σ_M) and of failure handling that the present architecture does not address.

13.4 Subsystem evolution policies

The stability contract (Theorem 7.3) is conservative: additive evolution only, with semantic-versioning discipline. More flexible policies, such as soft-deprecation with backwards-compatibility shims, are common in production systems but require additional contract machinery.

14 Conclusion

We have specified a kernel architecture for the dispatch of categorical computation: a substrate that wraps an executor with five subsystems whose responsibilities partition the dispatch envelope. The five—pre-dispatch validation (PVE), post-dispatch memoization (CMM), pending-state scheduling (PSS), external-resource retrieval (DIC), and runtime invariant monitoring (TEM)—are derived from the universal dispatch responsibilities shared across operating-system kernels, virtual machines, actor systems, query engines, and event-driven runtimes.

The architecture rests on six structural results: the Dispatch Correctness Theorems (Theorems 5.2 and 5.3) which ensure the wrapped executor’s semantics are preserved; the Independence Theorem (Theorem 6.2) which establishes pairwise functional independence of the five subsystems; the Stability Contract Theorem (Theorem 7.3) which guarantees client compatibility under additive kernel evolution; the Determinism Preservation Corollary (Corollary 5.4); the Compositional Substitution Theorem (Theorem 8.1); and the Conformance Decidability Theorem (Theorem 9.3) which provides a property-based regression framework.

The kernel is purely a substrate. It does not commit to a fragment grammar, a value space, a validation policy, a cache eviction strategy, a scheduler, a retrieval mechanism, or a monitor implementation. What it provides is the architectural skeleton over which such choices are made, together with the theorems that ensure the choices interact compositionally and that downstream clients depending on the frozen surface remain compatible across kernel evolutions.

The framework is foundational. It does not specify which executors should be wrapped, which subsystem implementations should be chosen, or which conformance predicates should be checked. Those choices are made by the applications that

consume the kernel. What the framework guarantees is that, once those choices are made, the resulting substrate satisfies the six structural theorems above: it dispatches correctly, the subsystems compose independently, clients of the frozen surface are stable, dispatch is deterministic given deterministic subsystems, bisimilar subsystems are interchangeable, and the conformance regression suite is decidable.

References

- Gul Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology, Stockholm, 2003.
- Andrew D. Birrell and Bruce Jay Nelson. Implementing remote procedure calls. *ACM Transactions on Computer Systems*, 2(1):39–59, 1984.
- Goetz Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2):73–169, 1993.
- Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.
- Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, et al. seL4: Formal verification of an OS kernel. In *Proceedings of the 22nd ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.
- Jörg Liedtke. On μ -kernel construction. *ACM SIGOPS Operating Systems Review*, 29(5):237–250, 1995.
- Tim Lindholm, Frank Yellin, Gilad Bracha, and Alex Buckley. *The Java Virtual Machine Specification*. Addison-Wesley, java se 8 edition, 2014.
- Douglas C. Schmidt, Michael Stal, Hans Rohnert, and Frank Buschmann. *Pattern-Oriented Software Architecture, Volume 2: Patterns for Concurrent and Networked Objects*. Wiley, 2000.
- Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson, 4th edition, 2014.

Part VI

Closed Loop

width=!,height=!, fitpaper=true, pagecommand= ,pages=-, addtotoc=1, chapter, 1, Paper 17. Runtime Operations of a Tempus-Based Partition Trigger: Kernel Instantiation, Bisimulation Proof, and the Complete Closed Loop from Partition Physics to Dispatch, paper:runtime

Derived from the Bounded Phase Space Axiom.
Every result in this monograph is a theorem.

No free parameter is introduced beyond the caesium-133 calibration constant α .
